

AMRITA VISHWA VIDHYAPEETHAM
AMRITA SCHOOL OF COMPUTING
ACADEMIC YEAR: 2022-2023 -ODD SEMESTER

23CCE284 - DATABASE MANAGEMENT SYSTEMS LABORATORY

SEM:IV

YEAR:II

BATCH: 2023-2027

LAB MANUAL

NAME : GOWTHAM M

REG NO : CH.EN.U4CCE24018

LIST OF EXPERIMENTS

1. Data Definition Commands, Data Manipulation Commands for inserting, deleting, updating and retrieving Tables and Transaction Control statements.
2. Database Querying – Simple queries, Nested queries, Sub queries and Joins.
3. Views, Sequences, Synonyms.
4. Database Programming: Implicit and Explicit Cursors.
5. Procedures and Functions.
6. Triggers.
7. Exception Handling.
8. Database Design using ER modeling, normalization and Implementation for any application.
9. Database Connectivity with Front End Tools.
10. Case Study using real life database applications.

PREPARED BY

Dr.K.Venkatesan
Asst.Professor(Sl. Grade)
of CSE(AIE)

APPROVED BY

Program Head Dept.
Dept.of CCE **INDEX PAGE**

EX. NO	DATE	TITLE OF THE EXPERIMENT	MARK	SIGNATURE
1		Data Definition Language		
2		Data Manipulation Language		

3		Creation of a Database and Writing SQL Queries to Retrieve Information from the Database.		
4		Build in Functions		
5		Creation of View, Synonym, Sequence, Index		
6		Creating a Student Database to set various Constraints		
7		TCL Commands		
8		Creating Relationship between the Databases		
9		Creating Nested Queries		
10		Study of PL/SQL Block		
11		Write a PL/SQL block to satisfy some conditions by accepting input from the user		
12		Write a PL/SQL Block that handles all types of Exceptions		
13		Creation of Procedures		
14		Creation of Database Triggers and Functions		
15		Creation of Implicit and Explicit Cursor		
16		Student Database Application		
17		Sports Meet Application		

INTRODUCTION TO SQL

SQL (Structured Query Language) is a database computer language designed for the retrieval and management of data in relational database management systems (RDBMS), database schema creation and modification, and database object access control management. SQL is a standard supported by all the popular relational database management systems in the market place. The basis data structure in RDBMS is a table. SQL provides you the features to define tables, define constraints on tables, query for data in table, and change the data in table by adding, modifying, and removing data. SQL also supports grouping of data in multiple rows, combining tables and other features. All these put together, SQL is a high-level query language standard to access and alter data in RDBMS.

History of SQL:

The first version of SQL was developed at IBM by Donald D. Chamberlin and Raymond F. Boyce in the early 1970s. This version, initially called SEQUEL, was designed to manipulate and retrieve data stored in IBM's original relational database product, System R. IBM patented their version of SQL in 1985, while the SQL language was not formally standardized until 1986, by the American National Standards Institute (ANSI) as SQL-86. Subsequent versions of the SQL standard have been released by ANSI and as International Organization for Standardization (ISO) standards.

Originally designed as a declarative query and data manipulation language, variations of SQL have been created by SQL database management system (DBMS) vendors that add procedural constructs, control-of-flow statements, user-defined data types, and various other language extensions. With the release of the SQL:1999 standard, many such extensions were formally adopted as part of the SQL language via the SQL Persistent Stored Modules (SQL/PSM) portion of the standard.

Querying can be used to: To retrieve existing data from database.

- Get all data from the table
- Get selected columns from the table.
- Get selected rows from the table.
- Get selected columns of selected rows from the table.
- Get computed columns using char, number, data functions, general functions, and aggregating functions.
- Get data in multiple rows grouped on an aggregating function applied on one or more columns.
- Select specific aggregating data on multiple rows using having clause. Apply set operations like Union and Intersection on data sets of the same cardinality and type.
- Get data from multiple tables using Cartesian product, equality join, un-equal join, and outer join.
- Create views on physical data.

Various Data Types :

1. Character Datatypes:

- ☐ Char – fixed length character string that can varies between 1-2000 bytes
- ☐ Varchar / Varchar2 – variable length character string, size ranges from 1-4000 bytes.it saves the disk space(only length of the entered value will be assigned as the size of column)
- ☐ Long - variable length character string, maximum size is 2 GB

2. Number Datatypes : Can store +ve,-ve,zero,fixed point,floating point with 38 precission.

- ☐ Number – {p=38,s=0}
- ☐ Number(p) - fixed point
- ☐ Number(p,s) –floating point (p=1 to 38,s= -84 to 127)

3. Date Datatype: used to store date and time in the table.

- ☐ DB uses its own format of storing in fixed length of 7 bytes for century,date,month,year,hour,minutes,seconds.
- ☐ Default data type is “dd-mon-yy”

Exp. No.1

DATA DEFINITION LANGUAGE

Date:

AIM:

To Execute Data Definition Language (DDL) commands available in RDBMS by using SQL.

PROCEDURE:

It is used to communicate with database. DDL is used to:

o Create an object

o Alter the structure of an object

o

T

o

drop the object created.

The commands used are:

o Create

o Alter

o

Drop

o

Truncate

1. CREATING A TABLE

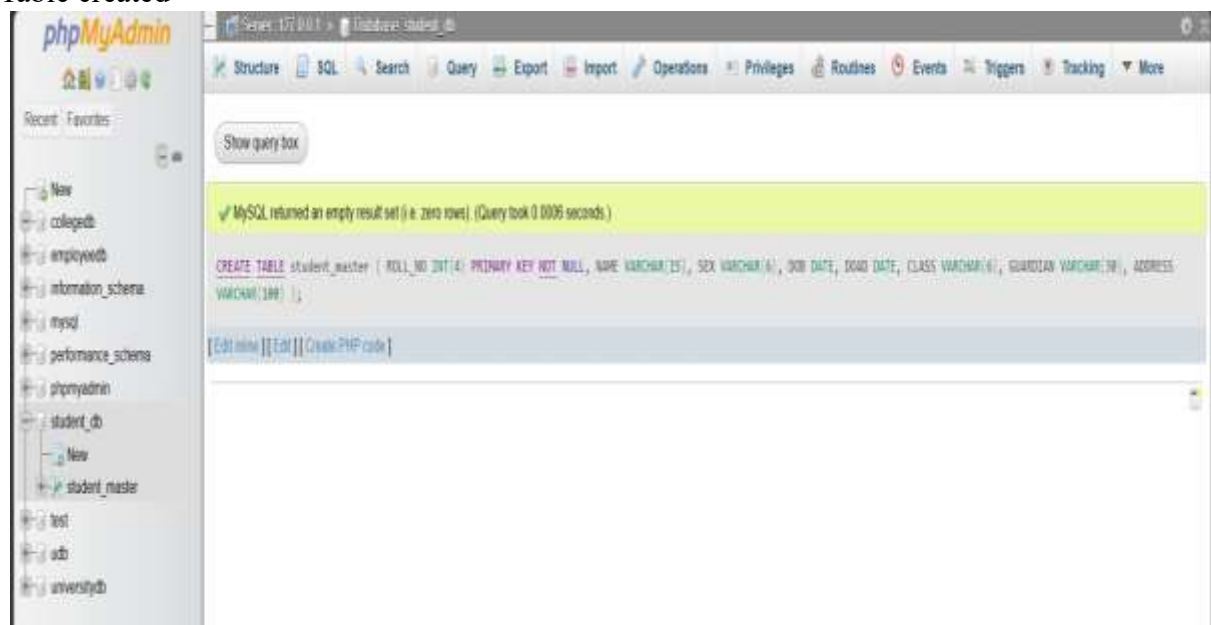
Syntax

CREATE TABLE <table name> (column1 datatype1,..., column datatypeN); **Example**

```
SQL> CREATE TABLE STUDENT_MASTER(ROLL_NO NUMBER(4), NAME VARCHAR2(15),SEX VARCHAR2(6),DOB DATE,DOAD DATE, CLASS VARCHAR2(6),GUARDIAN VARCHAR2(30), ADDRESS VARCHAR2(100));
```

```
CREATE TABLE STUDENT_MASTER ( ROLL_NO INT(4) PRIMARY KEY not null, NAME VARCHAR(15), SEX VARCHAR(6), DOB DATE,DOAD DATE, CLASS VARCHAR(6), GUARDIAN VARCHAR(30), ADDRESS VARCHAR(100) );
```

Table created



2. DESC COMMAND

If the user wants to view the structure of a table, this command can be used.

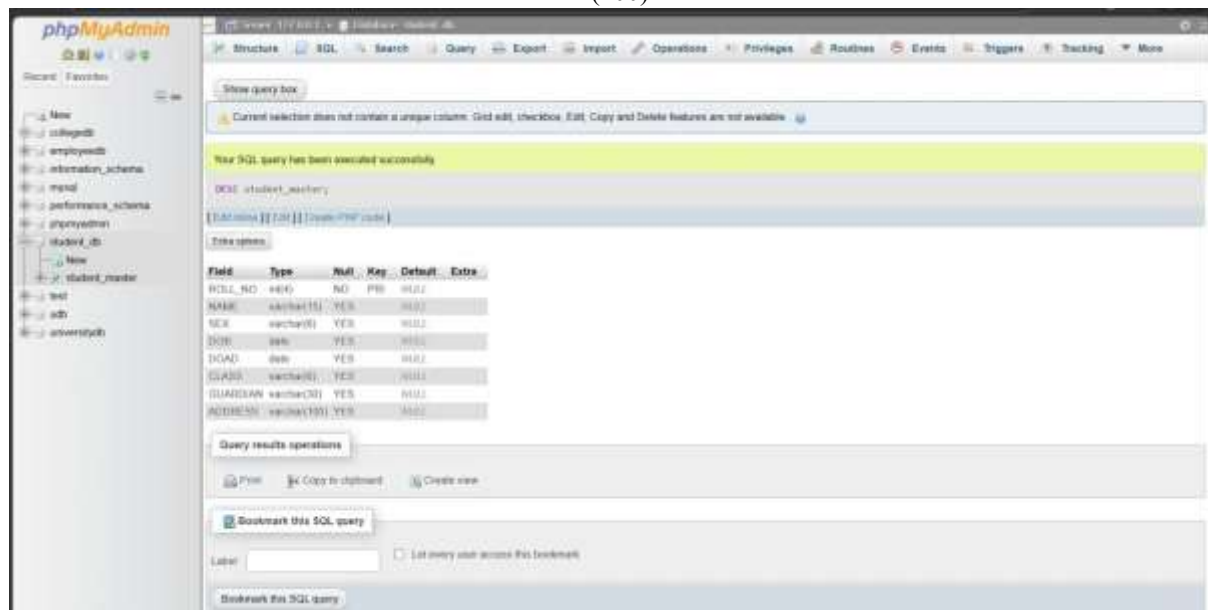
Syntax

Desc <table name>

Example

```
SQL>DESC STUDENT_MASTER;
```

Name	Null?	Type
ROLL_NO		NUMBER(4)
NAME		VARCHAR2(15)
Sex		VARCHAR2(6)
DOB		DATE
DOAD		DATE
CLASS		VARCHAR2(6)
GUARDIAN		VARCHAR2(30)
ADDRESS		VARCHAR2(100)



3. ALTERING A TABLE

The structure of a table can be changed using this command. Using this command we can do the following.

- Add a new column.
- Change the width of a data type
- Change the data type of a column
- Drop a column

Syntax

```
ALTER TABLE <table name> ADD (column datatype,...);
```

```
ALTER TABLE <table name> MODIFY (column datatype,...);
```

```
ALTER TABLE <table name> DROP (column);
```

Example 1

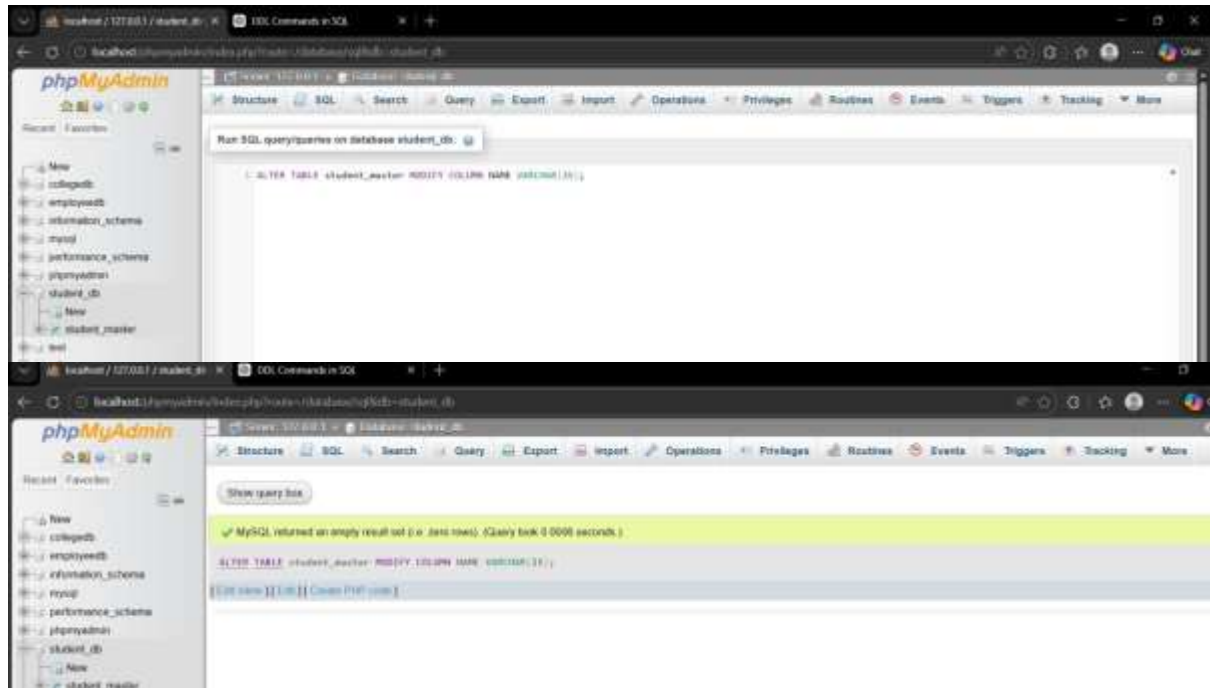
After creating the student_master table, the authority feels that the name of some students may exceed 15 characters. Hence it decides to change the width of the column to 25. The modification is carried out as follows.

```
SQL>ALTER TABLE STUDENT_MASTER MODIFY (NAME VARCHAR2 (25));
```

```
ALTER TABLE STUDENT_MASTER MODIFY COLUMN NAME VARCHAR(25);
```

```
SQL>DESC STUDENT_MASTER;
```

Name	Null?	Type
ROLL_NO		NUMBER(4)
NAME		VARCHAR2(25)
SEX		VARCHAR2(6)
DOB		DATE
DOAD		DATE
CLASS		VARCHAR2(6)
GUARDIAN		VARCHAR2(30)
ADDRESS		VARCHAR2(100)



Example 2

The head of the institution realizes that the phone number of each student is also necessary. This can be done by adding a new column to the existing student_master file.

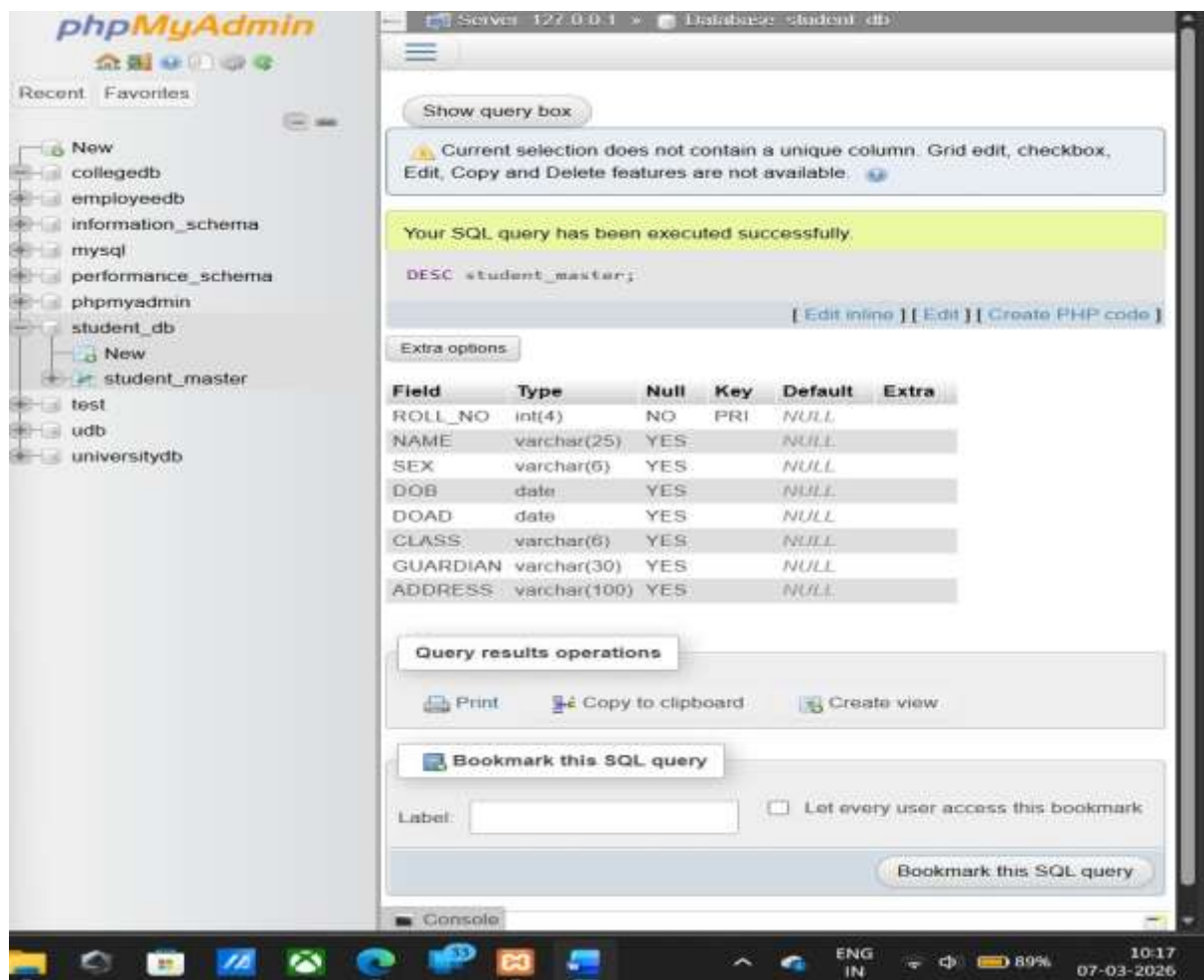
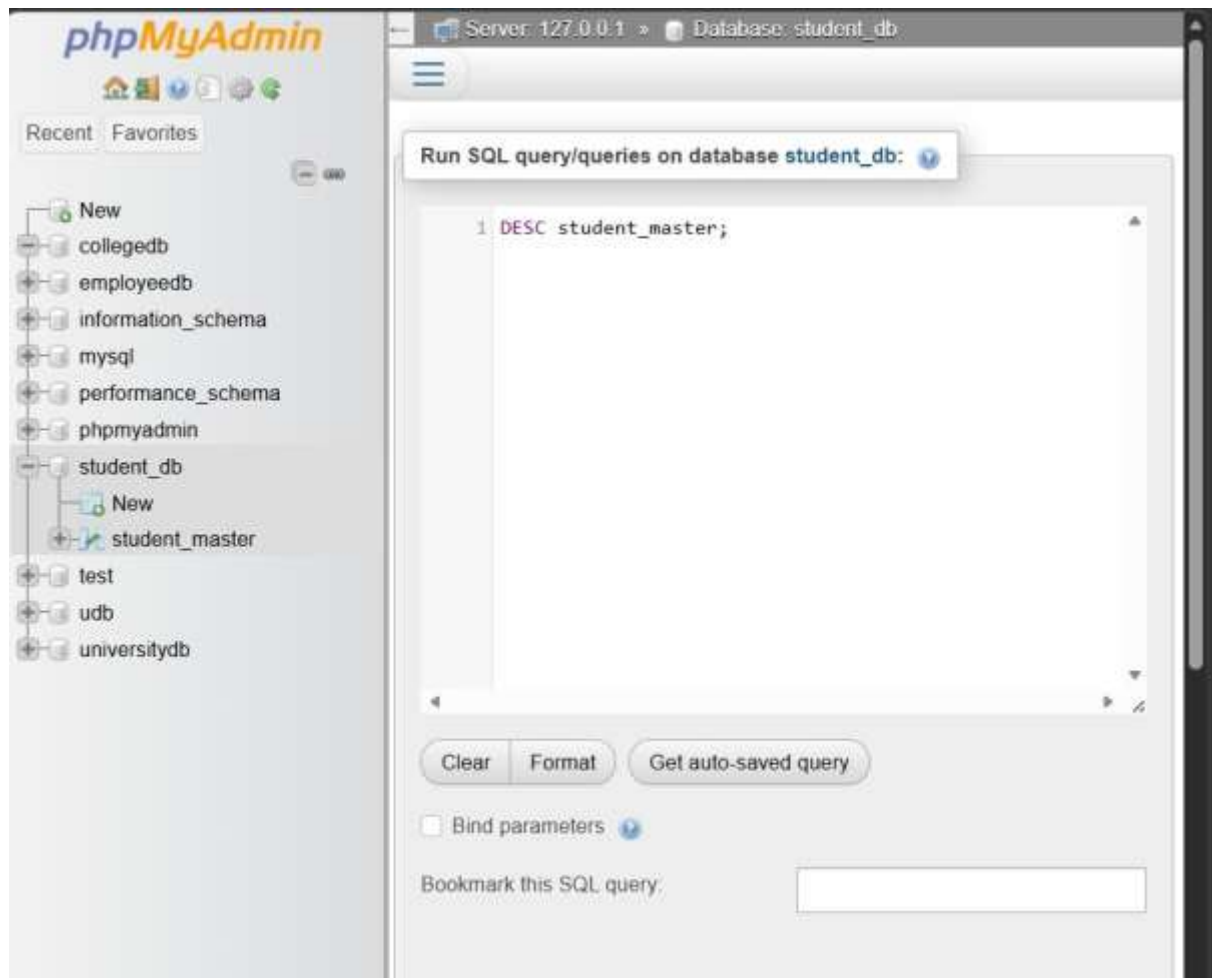
SQL>ALTER TABLE STUDENT_MASTER ADD(TEL_NO NUMBER(7));

ALTER TABLE STUDENT_MASTER ADD COLUMN TEL_NO INT(7); Table altered.

SQL>DESC STUDENT_MASTER;

Name	Null?	Type
ROLL_NO		NUMBER(4)
NAME		VARCHAR2(25)
SEX		VARCHAR2(6)
DOB		DATE
DOAD		DATE
CLASS		VARCHAR2(6)
GUARDIAN		VARCHAR2(30)
ADDRESS		VARCHAR2(100)
NUMBER(7)		TEL_NO

DESC student_master



Example 3

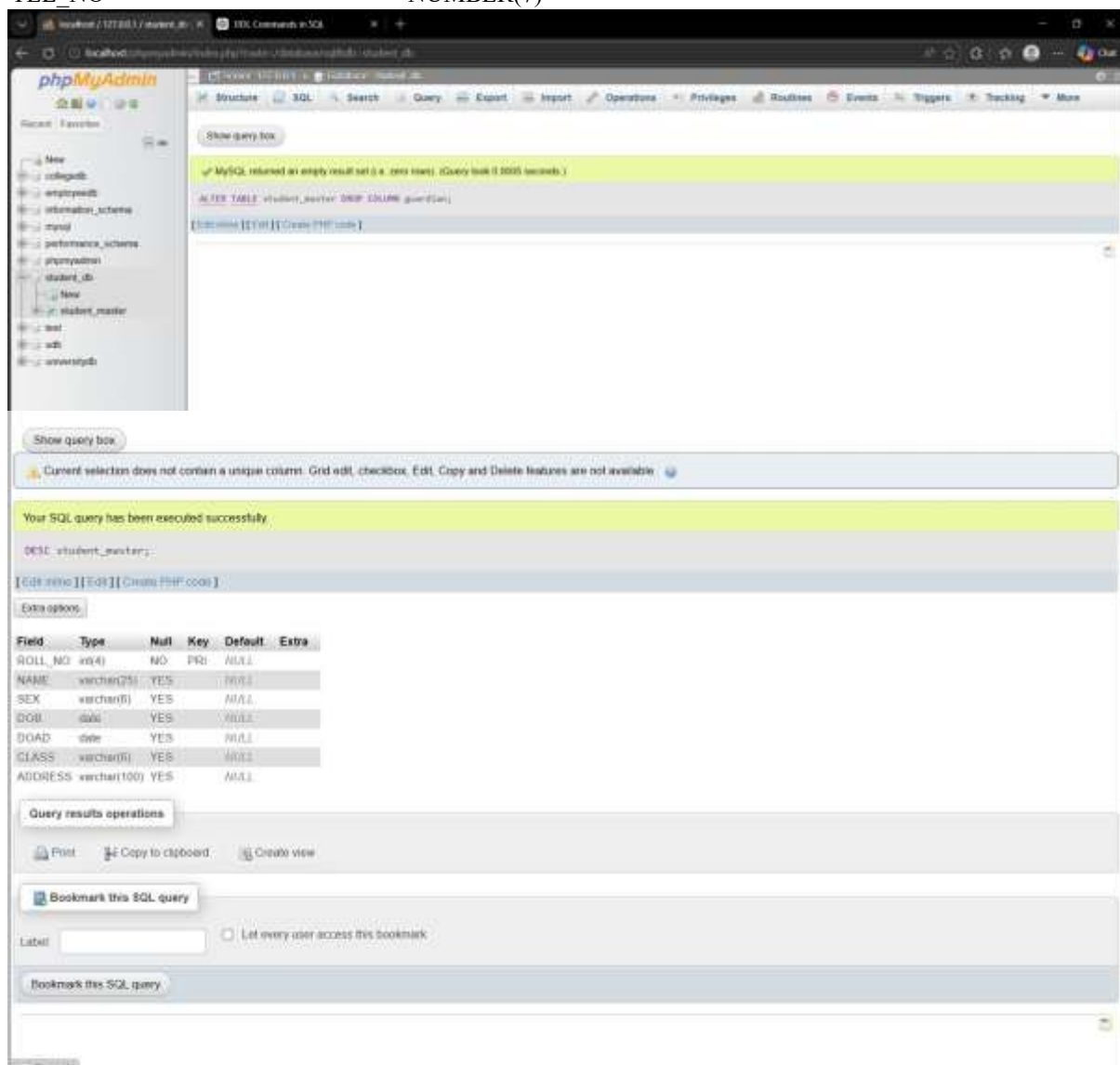
Possible that one of the column is also removed. This can be done by removing guardian attribute from the existing student_master file.

```
SQL>ALTER TABLE STUDENT_MASTER DROP(GUARDIAN);
```

ALTER TABLE STUDENT_MASTER DROP COLUMN GUARDIAN; Table altered.

```
SQL>DESC STUDENT_MASTER;
```

Name	Null?	Type
ROLL_NO		NUMBER(4)
NAME		VARCHAR2(25)
SEX		VARCHAR2(6)
DOB		DATE
DOAD		DATE
CLASS		VARCHAR2(6)
ADDRESS		VARCHAR2(100)
TEL_NO		NUMBER(7)



4. TRUNCATING A TABLE

This command is used to delete all records stored in a table. If we use this command, the structure of the table is retained.

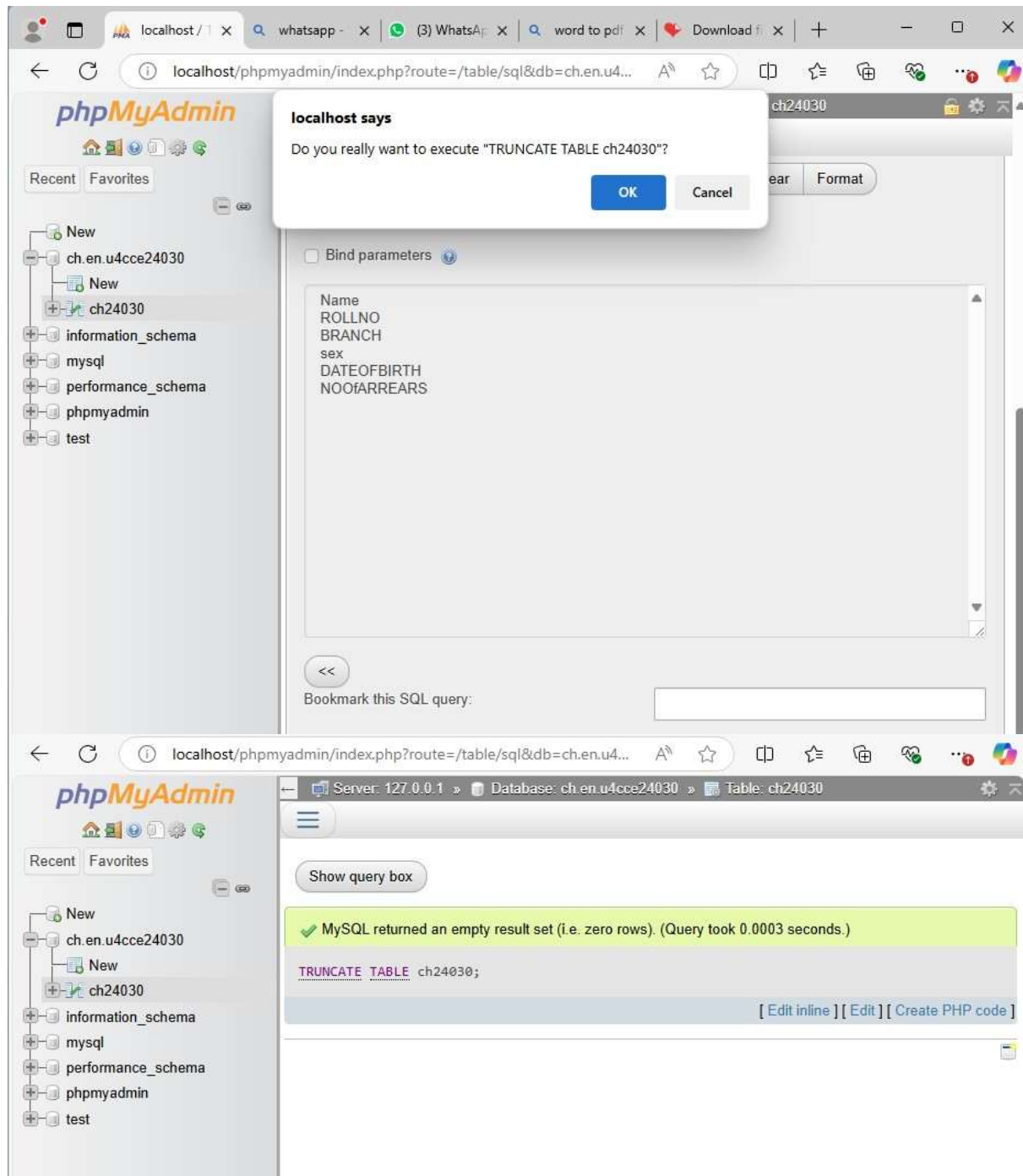
Syntax

```
TRUNCATE TABLE <table name>;
```

Example

SQL> TRUNCATE TABLE STUDENT_MASTER;

This command deletes all the rows in the table student_master.



5. DROPPING A TABLE

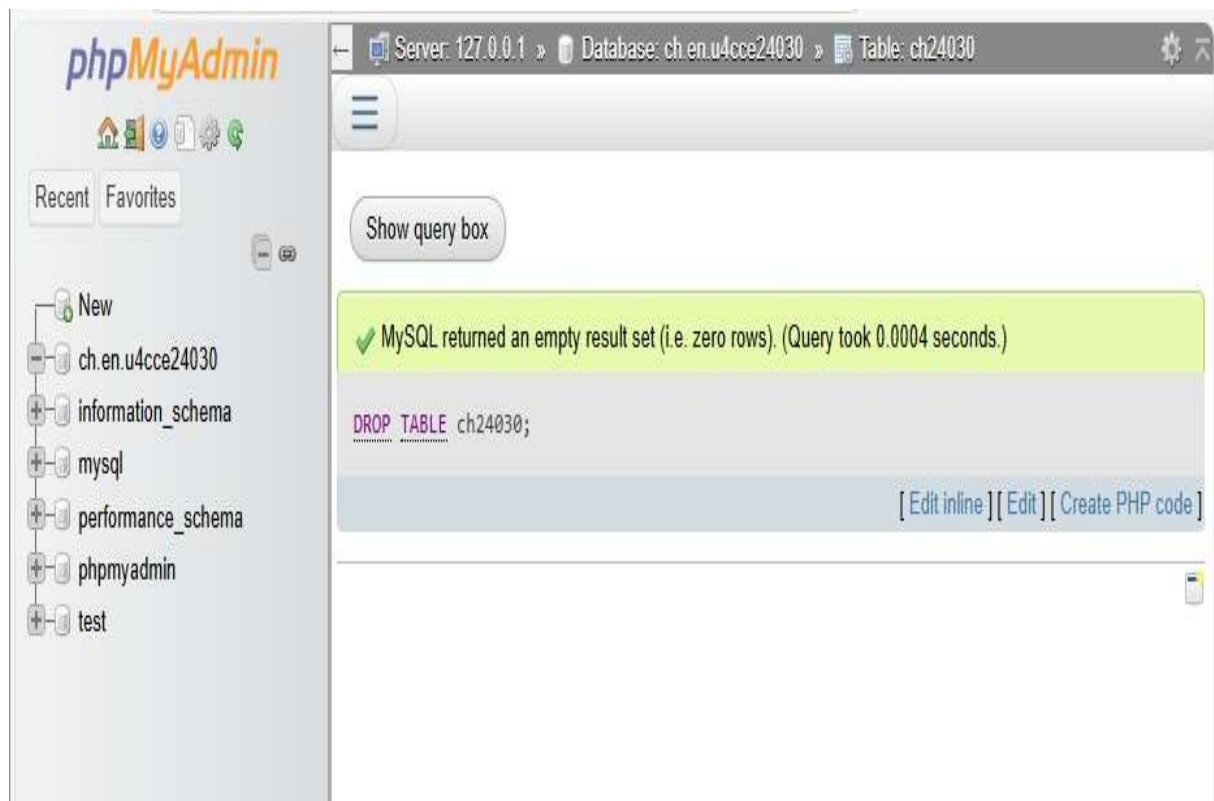
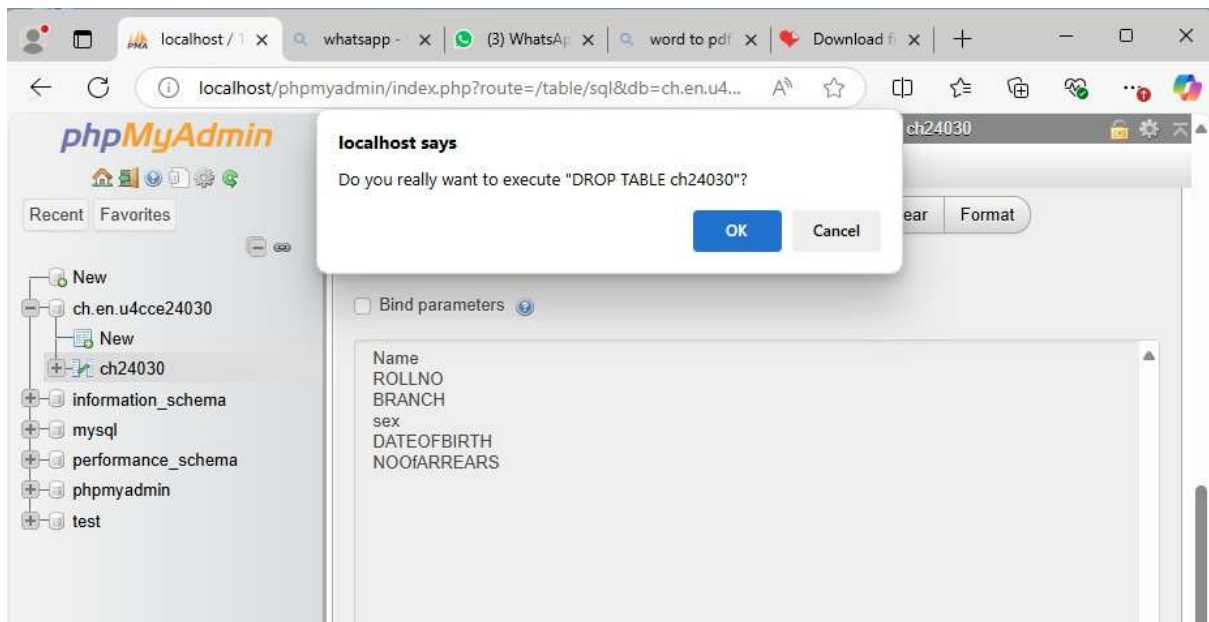
This command is used to remove a table from the database.

Syntax

DROP TABLE <table name>;

Example

SQL> DROP TABLE STUDENT_MASTER;



RESULT:

Thus the Data Definition Language (DDL) commands were studied and executed.

Exp. No.2

DATA MANIPULATION LANGUAGE

Date:

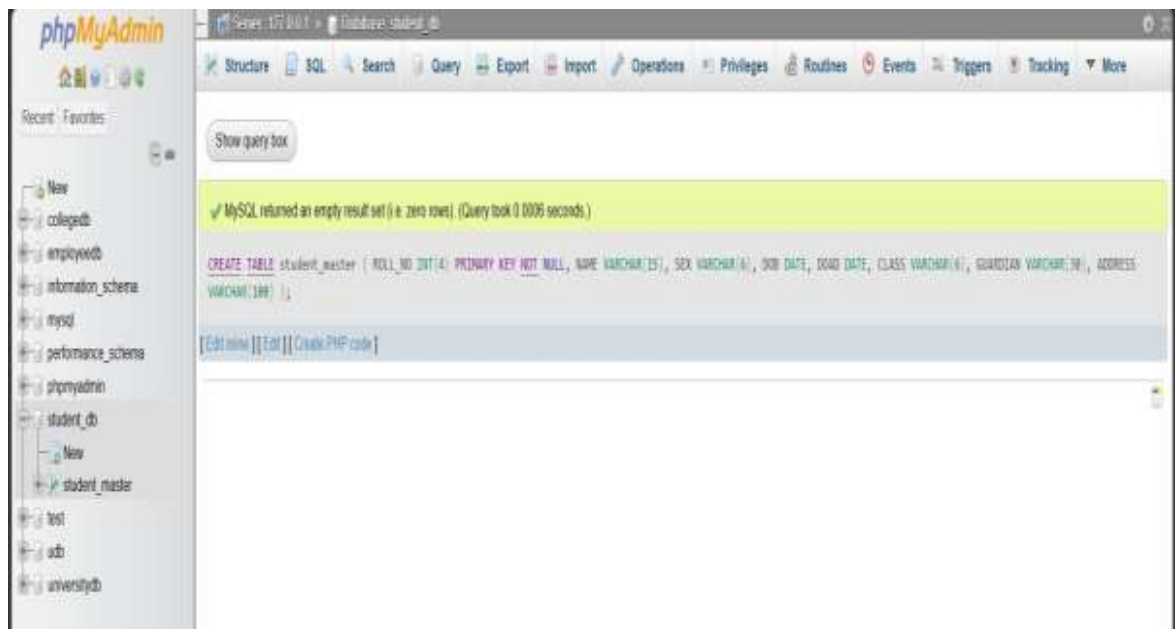
AIM:

To Execute Data Manipulation Language (DML) commands available in RDBMS by using SQL.

PROCEDURE:

DML commands are the most frequently used SQL commands and is used to query and manipulate the existing database objects. Some of the commands are

- o Insert
- o Select
- o Update
- o Delete



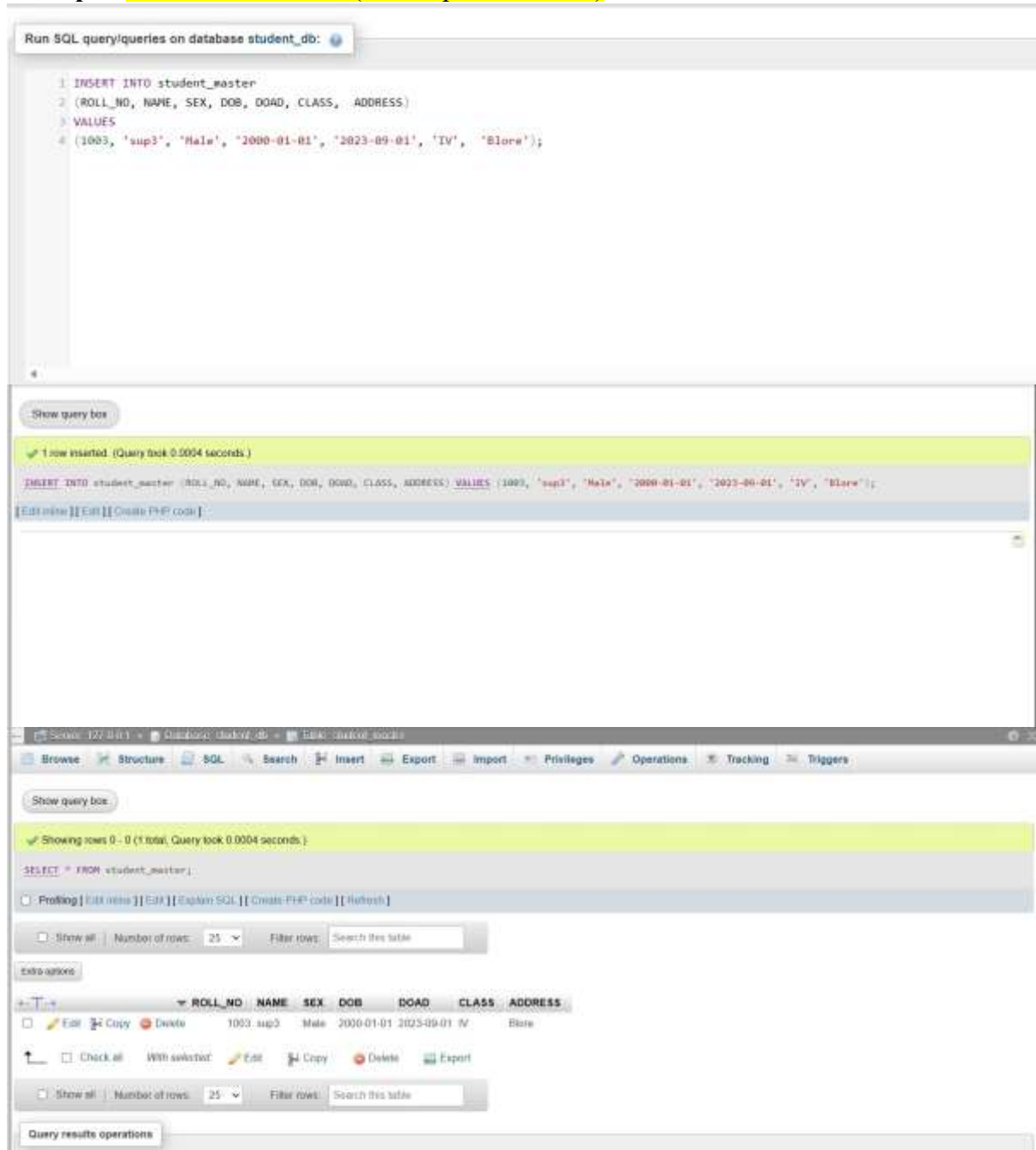
Insert Command

This is used to add one or more rows to a table. The values are separated by commas and the data types char and date are enclosed in apostrophes. The values must be entered in the same order as they are defined.

➤ Inserting a single row into a table:

Syntax: insert into <table name> values (value list)

Example: insert into stu values('s3','sup3','blore',10)



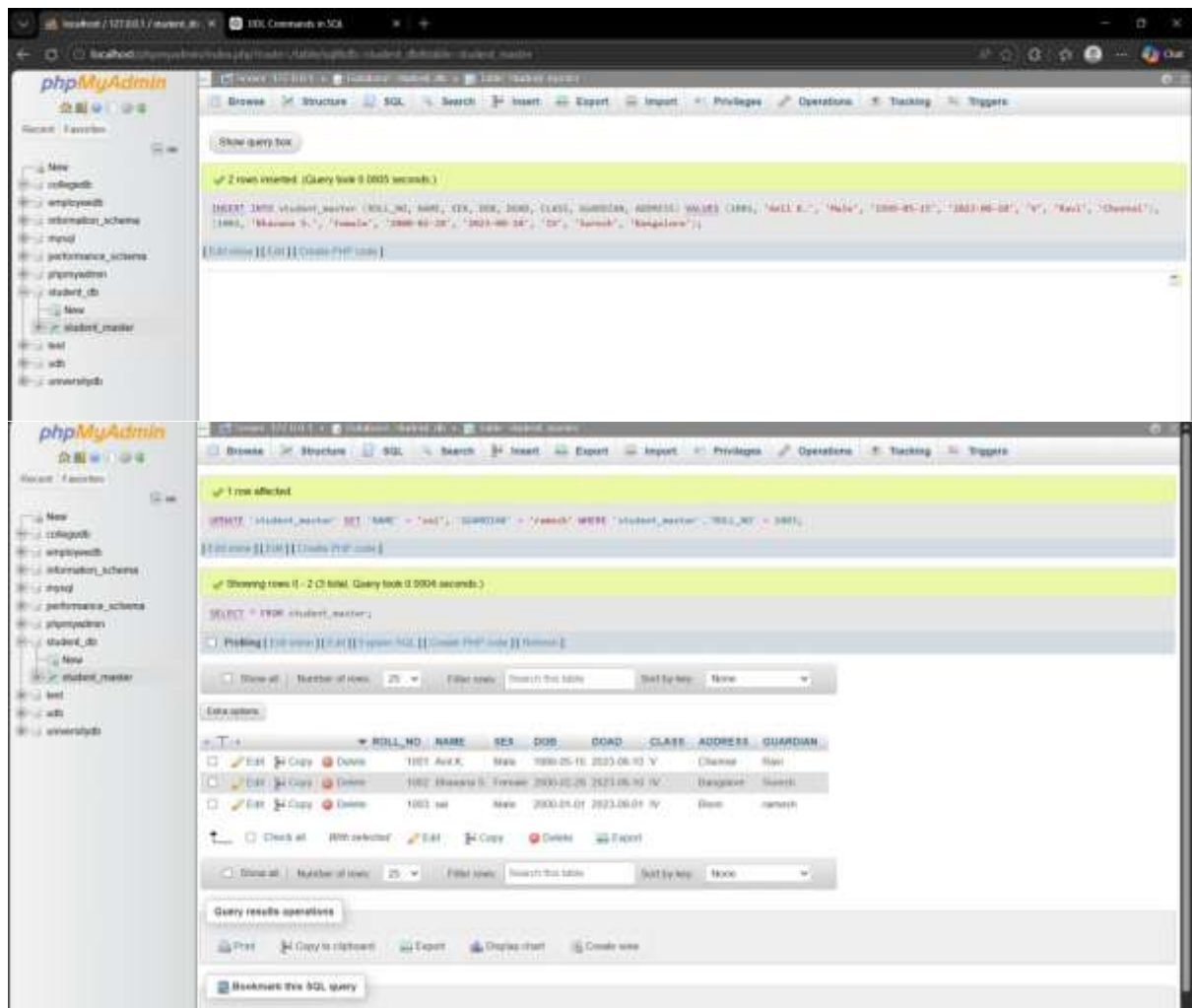
```
INSERT INTO STUDENT_MASTER (ROLL_NO, NAME, SEX, DOB, DOAD, CLASS, ADDRESS, TEL_NO) VALUES (1003, 'sup3', 'Male', '2000-01-01', '2023-09-01', 'IV', 'blore', 1234567);
```

➤ **Inserting more than one record using a single insert commands:**

Syntax: insert into <table name> values (&col1, &col2,)

Example: Insert into stud values(®, '&name', &percentage);

```
INSERT INTO STUDENT_MASTER (ROLL_NO, NAME, SEX, DOB, DOAD, CLASS, ADDRESS, TEL_NO) VALUES
(1001, 'Anil K.', 'Male', '1999-05-15', '2023-08-10', 'V', 'Chennai', 9876543),
(1002, 'Bhavana S.', 'Female', '2000-02-28', '2023-08-10', 'IV', 'Bangalore', 9876544),
(1003, 'Charles M.', 'Male', '1998-11-01', '2023-08-11', 'V', 'Kochi', 9876545);
```



➤ Skipping the fields while inserting:

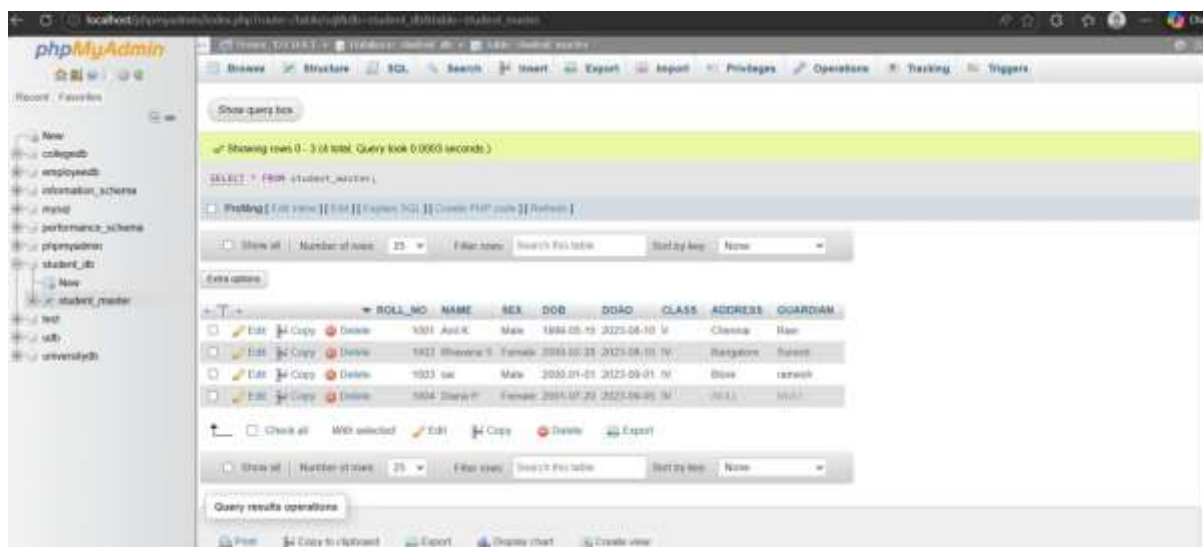
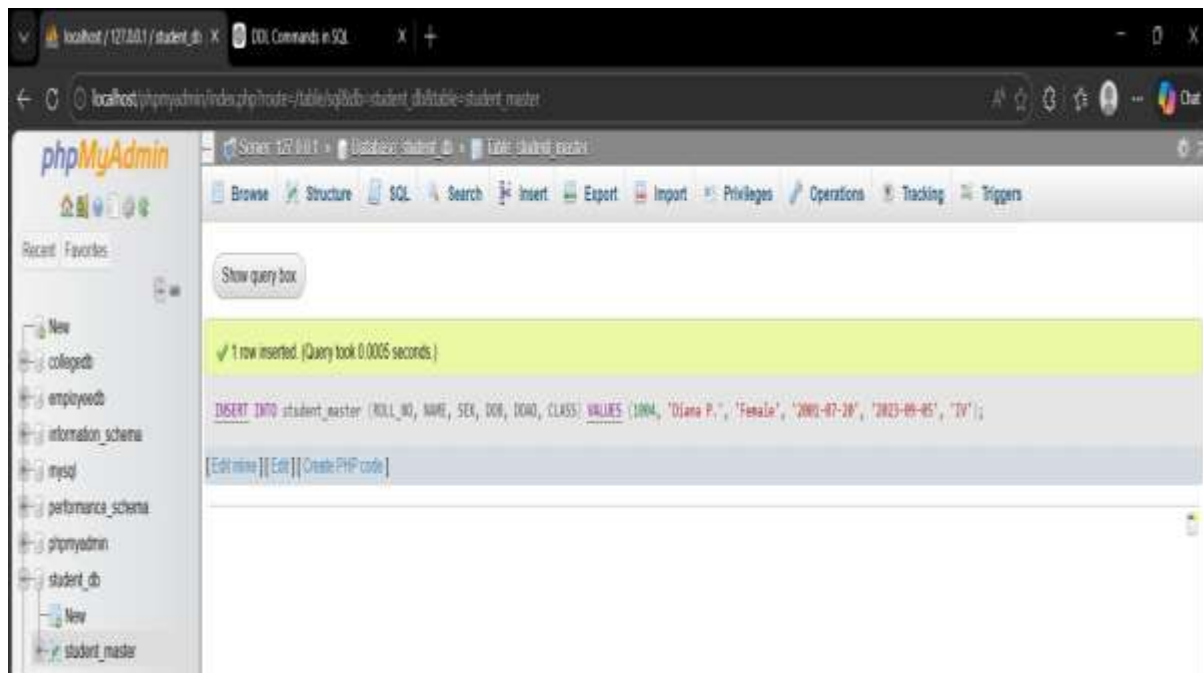
Syntax: Insert into <tablename> (coln names to which datas to be inserted) values (list of values);

Example: insert into stud (regno,name) values(121,'selvi');

INSERT INTO STUDENT_MASTER (ROLL_NO, NAME, SEX, DOB, DOAD, CLASS) VALUES (1004, 'Diana P.', 'Female', '2001-07-20', '2023-09-05', 'IV');

During insertion the skipped columns are taking null values.





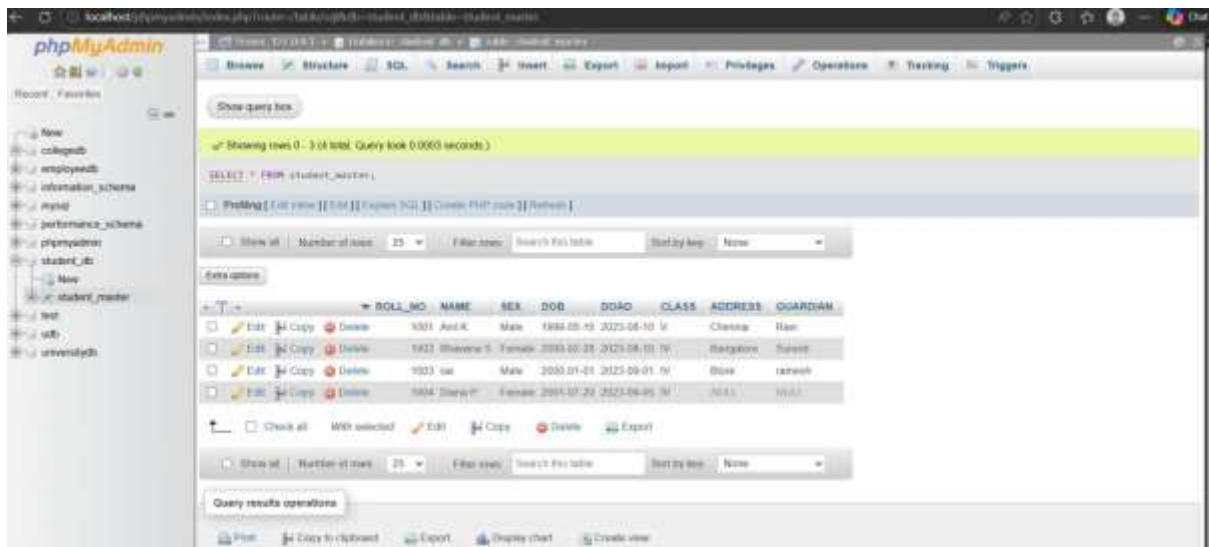
Select Commands

It is used to retrieve information from the table. It is generally referred to as querying the table. We can either display all columns in a table or only specify column from the table.

Syntax: Select * from tablename; // This query selects all rows from the table.

Example; Select * from IT;

SELECT * FROM STUDENT_MASTER;

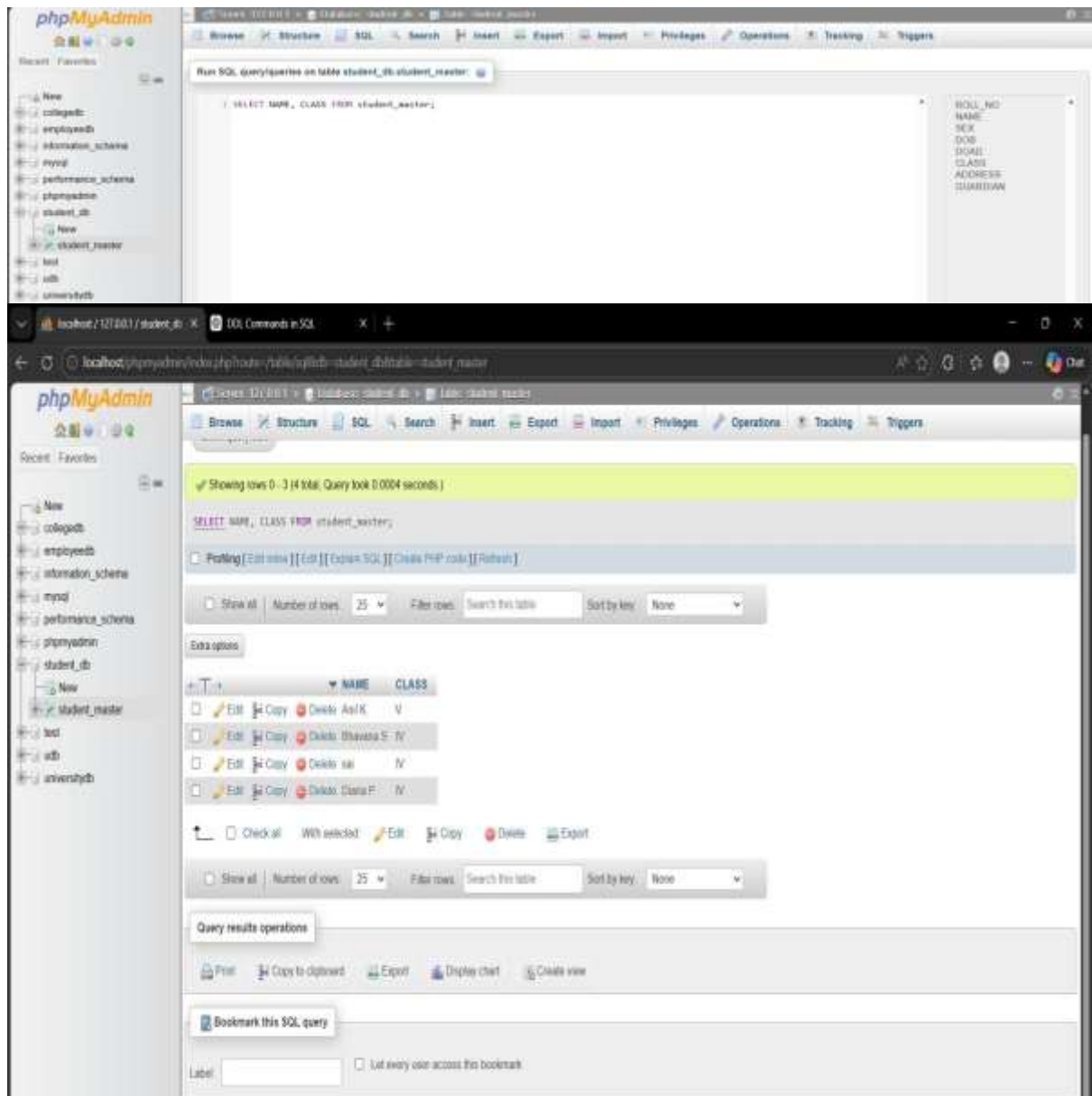


➤ The retrieval of **specific columns** from a table:

It retrieves the specified columns from the table

Syntax: Select column_name1,,column_namen from table name;

Example: Select empno, empname from emp;
 SELECT NAME, CLASS FROM STUDENT_MASTER;



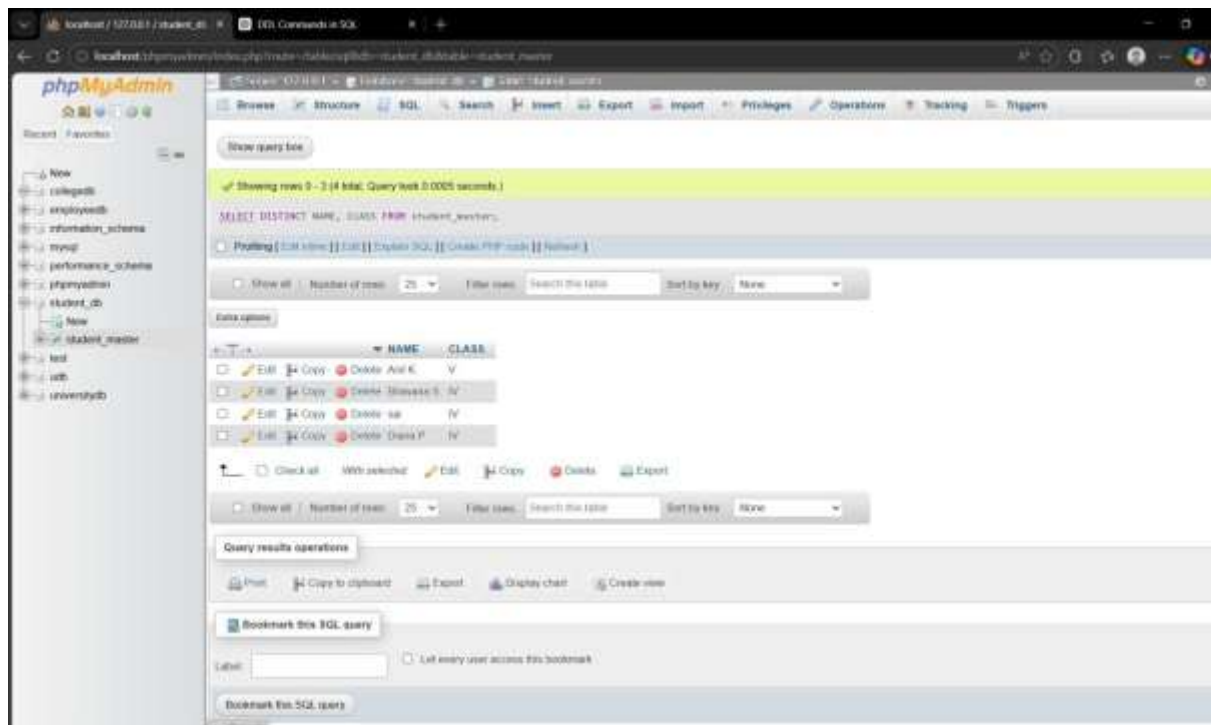
➤ Elimination of duplicates from the select clause:

It prevents retrieving the duplicated values .Distinct keyword is to be used. **Syntax:**

Select DISTINCT col1, col2 from table name;

Example: Select DISTINCT job from emp;

SELECT DISTINCT NAME, CLASS FROM STUDENT_MASTER;

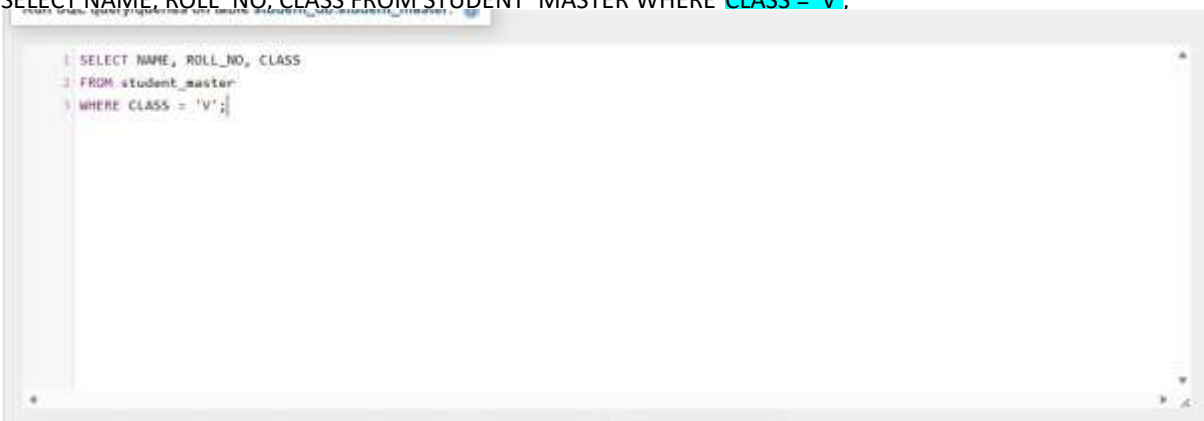


➤ Select command with **where clause**:

To select specific rows from a table we include 'where' clause in the select command. It can appear only after the 'from' clause.

Syntax: Select column_name1,,column_name from table name where condition;

Example: Select empno, empname from emp where **sal>4000**:
 SELECT NAME, ROLL_NO, CLASS FROM student_master WHERE **CLASS = 'V'**;



SELECT NAME, DOB FROM STUDENT_MASTER WHERE DOB > '2000-12-31';

The screenshot displays the phpMyAdmin web interface. The top navigation bar includes tabs for Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Tracking, and Triggers. The left sidebar shows a database structure with a tree view for 'student_db' containing tables like 'student_master' and 'student_detail'. The main content area is titled 'Run SQL queries on table student_db.student_master'. It contains a text box with the SQL query:

```
SELECT NAME, DOB
FROM student_master
WHERE DOB > '2000-12-31';
```

 Below the query box, a status bar indicates 'Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)'. The query is executed, and the results are displayed in a table with columns 'NAME' and 'DOB'. The table shows one row: 'Diana P' with '2001-07-20'. Below the table, there are options to 'Show all', 'Number of rows' (set to 25), and a 'Filter rows' search box. The 'Query results operations' section includes links for 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'. At the bottom, there is a 'Bookmark this SQL query' section with a 'Label' input field and a checkbox 'Let every user access this bookmark'.

SELECT * FROM STUDENT_MASTER WHERE ROLL_NO BETWEEN 1000 AND 1005;

The screenshot shows the phpMyAdmin interface. The SQL tab is active, displaying the following query:

```
SELECT *
FROM student_master
WHERE ROLL_NO BETWEEN 1000 AND 1005;
```

The query has been executed, showing 4 rows. The status bar indicates: "Showing rows 0 - 3 (4 total, Query took 0.0005 seconds)".

The results are displayed in a table with the following columns: ROLL_NO, NAME, SEX, DOB, DOAD, CLASS, ADDRESS, GUARDIAN.

ROLL_NO	NAME	SEX	DOB	DOAD	CLASS	ADDRESS	GUARDIAN
1001	Am K.	Male	1999-05-15	2023-08-10	V	Chennai	Ravi
1002	Bhavana S.	Female	2000-02-20	2023-08-10	IV	Bangalore	Suresh
1003	Kai	Male	2000-01-01	2023-08-01	IV	Bane	Ramesh
1004	Diana P.	Female	2001-07-20	2023-08-05	IV	MULL	NOT

Below the table, there are options for "Query results operations" including Print, Copy to clipboard, Export, Display chart, and Create view. A "Bookmark this SQL query" button is also present.

SELECT NAME, CLASS FROM STUDENT_MASTER WHERE NAME LIKE 'A%';

The screenshot shows the phpMyAdmin interface. The SQL tab is active, displaying the following query:

```
SELECT NAME, CLASS
FROM student_master
WHERE NAME LIKE 'A%';
```

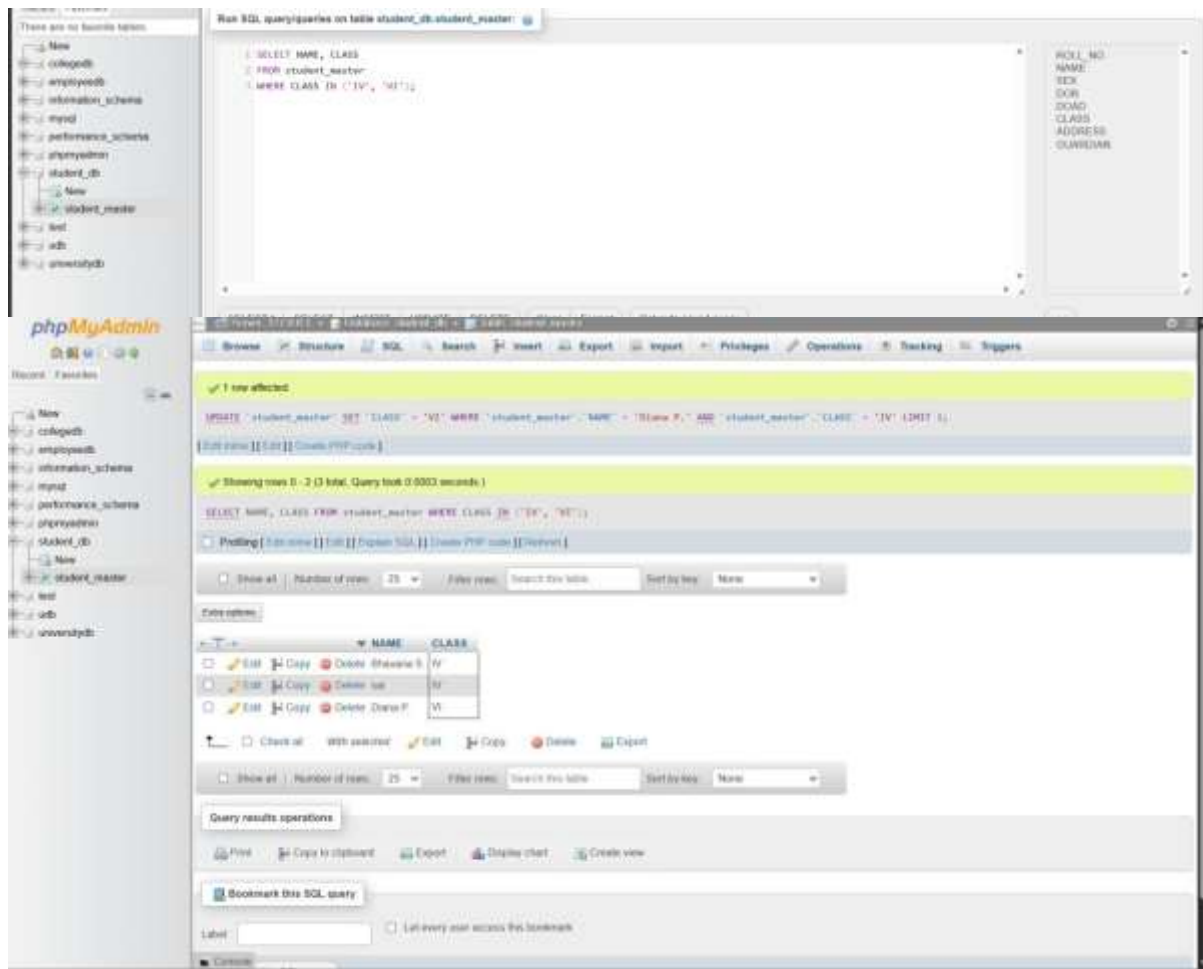
The query has been executed, showing 1 row. The status bar indicates: "Showing rows 0 - 0 (1 total, Query took 0.0004 seconds)".

The results are displayed in a table with the following columns: NAME, CLASS.

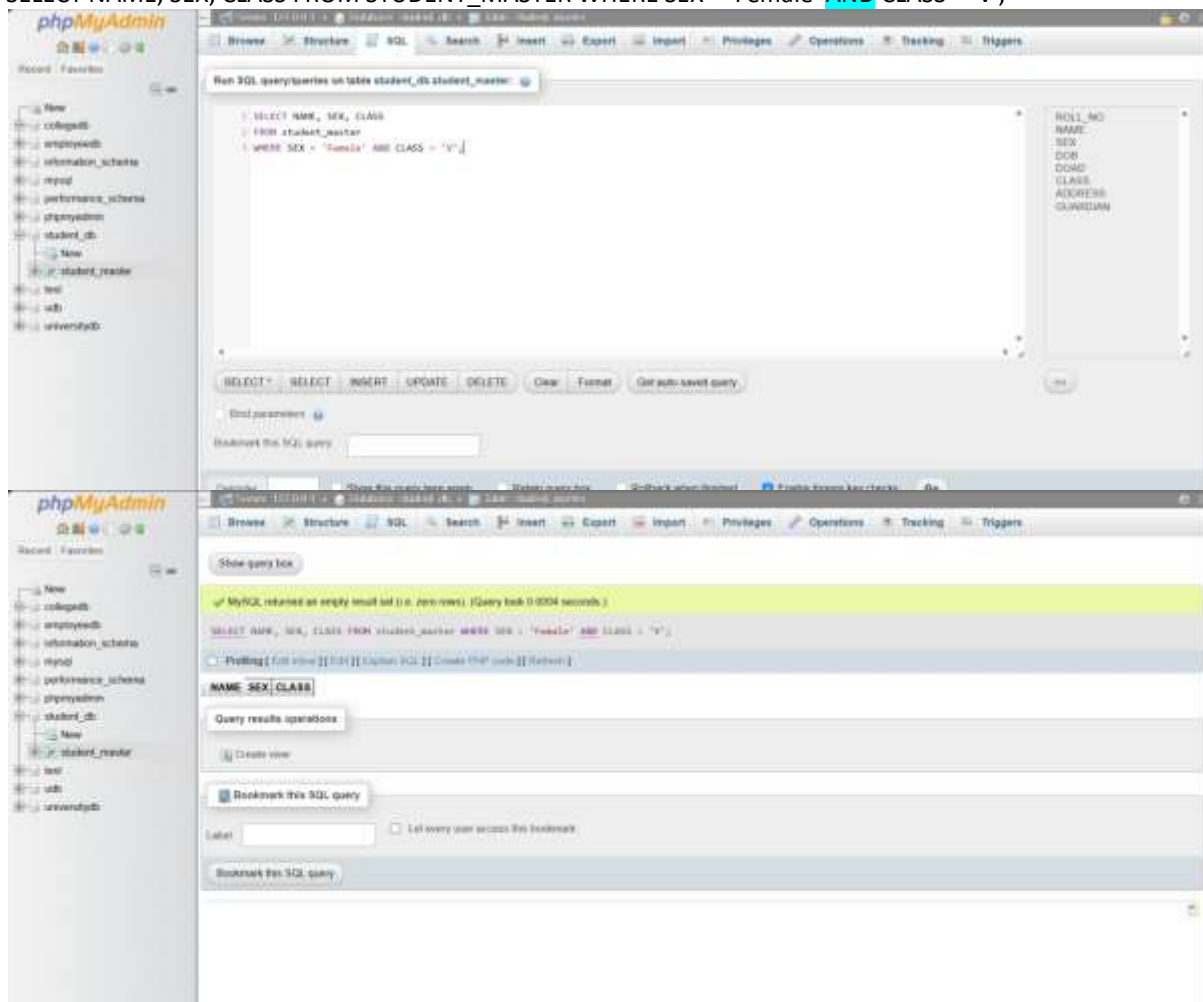
NAME	CLASS
Am K.	V

Below the table, there are options for "Query results operations" including Print, Copy to clipboard, Export, Display chart, and Create view. A "Show query box" button is also present.

SELECT NAME, CLASS FROM STUDENT_MASTER WHERE CLASS IN ('IV', 'VI');



SELECT NAME, SEX, CLASS FROM STUDENT_MASTER WHERE SEX = 'Female' AND CLASS = 'V';



➤ **Select command with order by clause:**

Select the values of the table with respect to some attribute in ascending or descending order.

Syntax: Select column_name1,,column_namen from table name where condition order by colmnname;

Example: *Select empno, empname from emp order by empno;*

SELECT NAME, CLASS FROM STUDENT_MASTER ORDER BY NAME ASC;

The screenshot shows the phpMyAdmin interface. The SQL query editor contains the following query:

```
SELECT NAME, CLASS  
FROM student_master  
ORDER BY NAME ASC;
```

The query has been executed successfully, showing 4 rows of data. The status bar indicates: "Showing rows 0 - 3 (4 total, Query took 0.0004 seconds.) [NAME: ANIL K.... - SAI...]"

The query text is: `SELECT NAME, CLASS FROM student_master ORDER BY NAME ASC;`

Below the query, there are controls for "Show all", "Number of rows: 25", "Filter rows: Search this table", and "Sort by key: None".

The "Extra options" section is expanded, showing a table with 2 columns: NAME and CLASS. The table contains 4 rows of data:

	NAME	CLASS
☐ Edit ☐ Copy ☐ Delete	Anil K.	V
☐ Edit ☐ Copy ☐ Delete	Bhavana S.	IV
☐ Edit ☐ Copy ☐ Delete	Diana P.	VI
☐ Edit ☐ Copy ☐ Delete	Sai	IV

Below the table, there are controls for "Check all", "With selected: Edit Copy Delete Export", and another set of controls for "Show all", "Number of rows: 25", "Filter rows: Search this table", and "Sort by key: None".

The "Query results operations" section is expanded, showing options for "Print", "Copy to clipboard", "Export", "Refresh", and "Create view".

SELECT NAME, DOAD FROM STUDENT_MASTER ORDER BY DOAD DESC;

The screenshot shows a SQL query editor with the following query:

```
1 SELECT NAME, DOAD  
2 FROM student_master  
3 ORDER BY DOAD DESC;
```

Show query box

Showing rows 0 - 3 (4 total, Query took 0.0004 seconds) [DOAD: 2023-09-05... - 2023-08-10...]

```
SELECT NAME, DOAD FROM student_master ORDER BY DOAD DESC;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	NAME	DOAD
☐ Edit Copy Delete	Diana P.	2023-09-05
☐ Edit Copy Delete	sai	2023-09-01
☐ Edit Copy Delete	Anil K.	2023-08-10
☐ Edit Copy Delete	Bhavana S.	2023-08-10

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

SELECT NAME, CLASS, ROLL_NO FROM STUDENT_MASTER ORDER BY CLASS ASC, NAME ASC;

Show query box

Showing rows 0 - 3 (4 total, Query took 0.0004 seconds) [CLASS: IV... - VI...][NAME: BHAVANA S... - DIANA P...]

```
SELECT NAME, CLASS, ROLL_NO FROM student_master ORDER BY CLASS ASC, NAME ASC;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	NAME	CLASS	ROLL_NO
☐ Edit Copy Delete	Bhavana S.	IV	1002
☐ Edit Copy Delete	sai	IV	1003
☐ Edit Copy Delete	Anil K.	V	1001
☐ Edit Copy Delete	Diana P.	VI	1004

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

SELECT NAME, ROLL_NO FROM STUDENT_MASTER WHERE CLASS = 'IV' ORDER BY NAME;

Showing rows 0 - 1 (2 total, Query took 0.0003 seconds) (NAME: BHAVANA S.... - SAI...)

```
SELECT NAME, ROLL_NO FROM student_master WHERE CLASS = 'IV' ORDER BY NAME;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	NAME	ROLL_NO
☐ Edit Copy Delete	Bhavana S	1002
☐ Edit Copy Delete	sai	1003

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Update Command

It is used to alter the column values in a table. A single column may be updated or more than one column could be updated.

Syntax: update tablename set field=values where condition;

Example: Update emp set sal = 10000 where empno=135;

Always include a **WHERE** clause. If you run the following command without it

UPDATE STUDENT_MASTER SET CLASS = 'V' WHERE ROLL_NO = 1003;

Run SQL query/queries on table student_db.student_master:

```
1 UPDATE student_master
2 SET CLASS = 'V'
3 WHERE ROLL_NO = 1003;
```

UPDATE STU

Show query box

✓ 1 row affected. (Query took 0.0004 seconds.)

```
UPDATE student_master SET CLASS = 'V' WHERE ROLL_NO = 1003;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

DENT_MASTER SET CLASS = 'VI', ADDRESS = 'Mumbai' WHERE NAME = 'Anil K.';

Run SQL query/queries on table `student_db.student_master`: 

```
1 UPDATE student_master
2 SET CLASS = 'VI', ADDRESS = 'Mumbai'
3 WHERE NAME = 'Anil K.';
```

Show query box

✓ 1 row affected. (Query took 0.0004 seconds.)

```
UPDATE student_master SET CLASS = 'VI', ADDRESS = 'Mumbai' WHERE NAME = 'Anil K.';
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

UPDATE STUDENT_MASTER SET TEL_NO = 9999999 WHERE CLASS = 'IV';

phpMyAdmin

Server: 127.0.0.1 • Database: student_db • Table: student_master

Recent Favorites

New

- collegedb
- employeedb
- information_schema
- mysql
- performance_schema
- phpmyadmin
- student_db
 - New
 - student_master
- test
- udb
- universityth

Run SQL query/queries on table student_db.student_master:

```

1 UPDATE student_master
2 SET TEL_NO = 9999999
3 WHERE CLASS = 'IV';

```

Show query box

✓ 1 row affected. (Query took 0.0004 seconds.)

UPDATE student_master SET TEL_NO = 9999999 WHERE CLASS = 'IV';

[Edit inline] [Edit] [Create PHP code]

✓ 1 row affected.

UPDATE `student\_master` SET `TEL\_NO` = '8765434' WHERE `student\_master`.`ROLL\_NO` = 1004;

[Edit inline] [Edit] [Create PHP code]

✓ Showing rows 0 - 3 (4 total. Query took 0.0002 seconds.)

SELECT * FROM student_master;

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

+T													
			▼	ROLL_NO	NAME	SEX	DOB	DOAD	CLASS	ADDRESS	GUARDIAN	TEL_NO	
☐					1001	Anil K.	Male	1999-05-15	2023-08-10	VI	Mumbai	Ravi	45678345
☐					1002	Bhavana S.	Female	2000-02-29	2023-08-10	IV	Bangalore	Suresh	9999999
☐					1003	sai	Male	2000-01-01	2023-09-01	V	Blore	ramesh	6789456
☐					1004	Diana P.	Female	2001-07-20	2023-08-05	VI	vijayawada	raju	8765434

⬆ ☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query

Delete command

After inserting row in a table we can also delete them if required. The delete command consists of a from clause followed by an optional where clause.

Syntax: Delete from table where conditions;

Example: delete from emp where empno=135;

```
DELETE FROM STUDENT_MASTER WHERE ROLL_NO = 1005;
```

Show query box

✓ 0 rows deleted. (Query took 0.0005 seconds.)

```
DELETE FROM student_master WHERE ROLL_NO = 1005;
```

[Edit inline] [Edit] [Create PHP code]

```
DELETE FROM STUDENT_MASTER WHERE CLASS = 'VI';
```

Show query box

✓ 2 rows deleted. (Query took 0.0011 seconds.)

```
DELETE FROM student_master WHERE CLASS = 'VI';
```

Edit inline] [Edit] [Create PHP code]

```
DELETE FROM STUDENT_MASTER WHERE DOAD < '2023-09-01';
```

Show query box

✓ 1 row deleted. (Query took 0.0004 seconds.)

```
DELETE FROM student_master WHERE DOAD < '2023-09-01';
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds)

```
SELECT * FROM student_master;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

	ROLL_NO	NAME	SEX	DOB	DOAD	CLASS	ADDRESS	GUARDIAN	TEL_NO			
☐	Edit	Copy	Delete	1003	sai	Male	2000-01-01	2023-09-01	V	Blore	ramesh	6789456

☐ Check all | With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 | Filter rows:

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Problem 1: Insert 5 records into dept table. The table has the following columns DNO, DNMAE, LOCATION

```
1 CREATE TABLE dept (  
2     DNO INT,  
3     DNAME VARCHAR(50),  
4     LOCATION VARCHAR(50)  
5 );
```

Show query box

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE dept ( DNO INT, DNAME VARCHAR(50), LOCATION VARCHAR(50) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

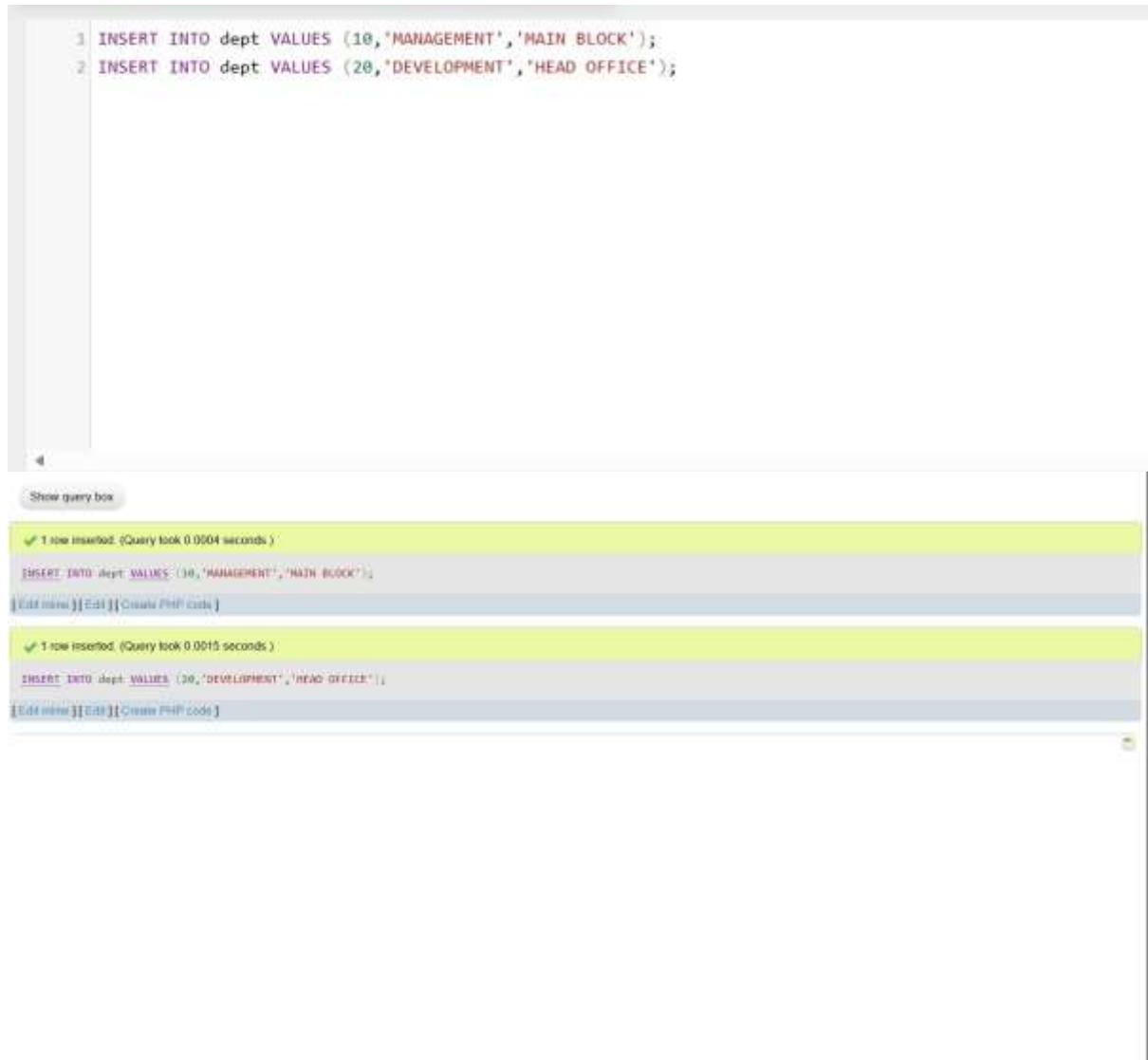
After Table Creation, we can insert first two rows by using **one row at a time syntax**.

SQL>insert into dept values (10,'MANAGEMENT','MAIN BLOCK');

1 row created.

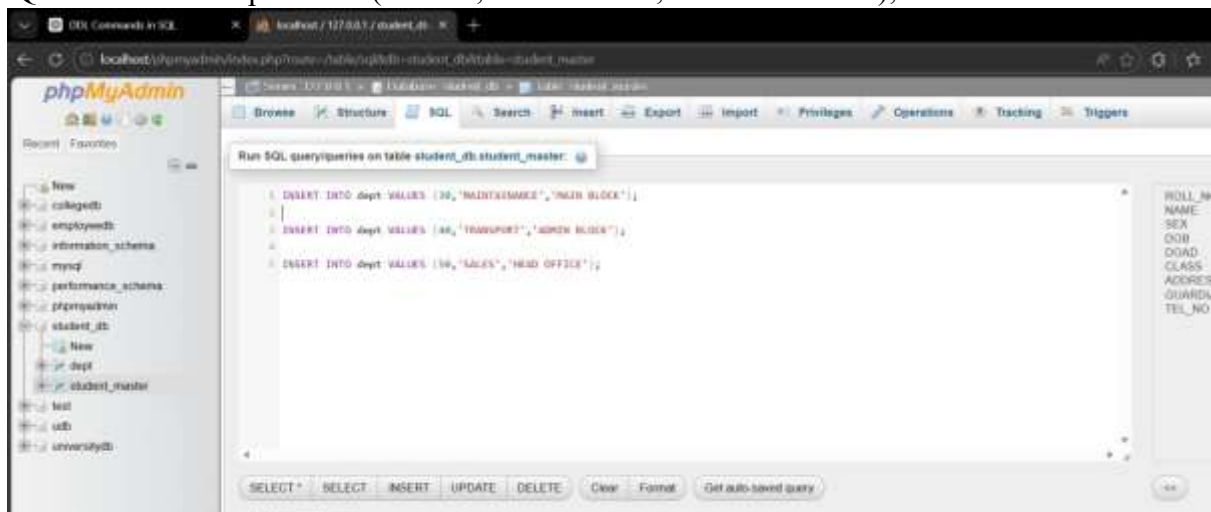
SQL>insert into dept values(20,'DEVELOPMENT', 'HEAD OFFICE');

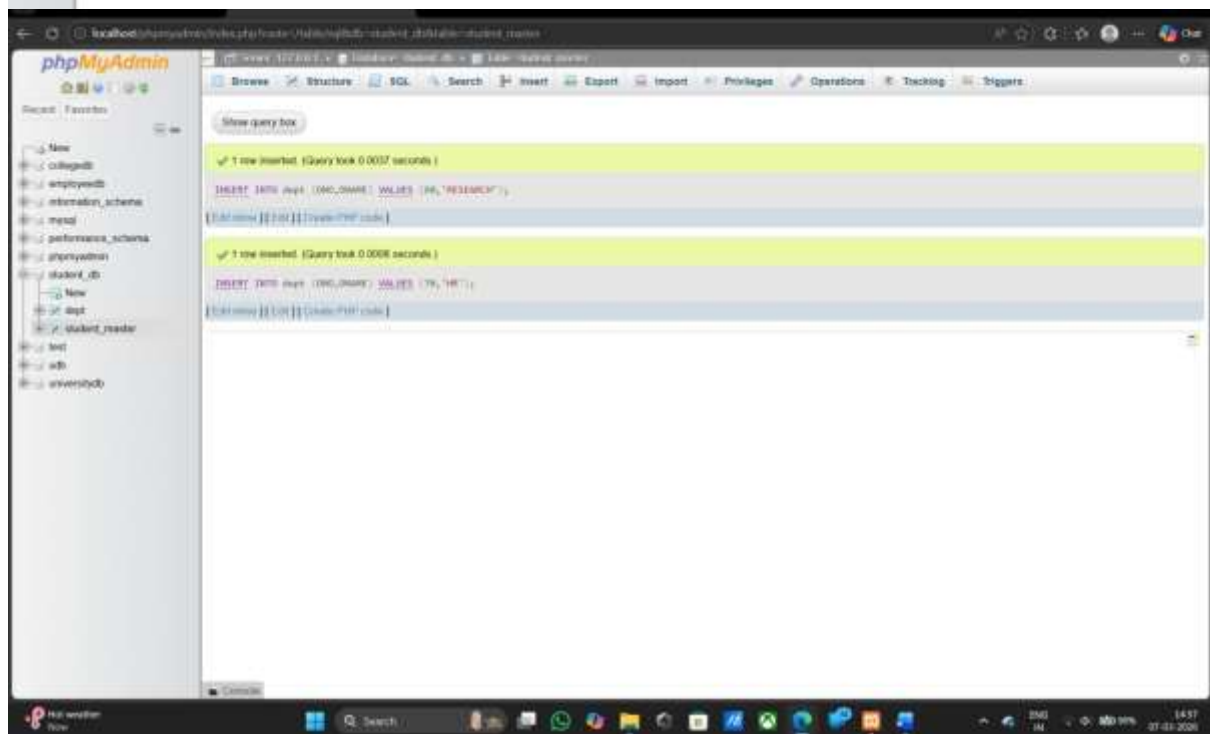
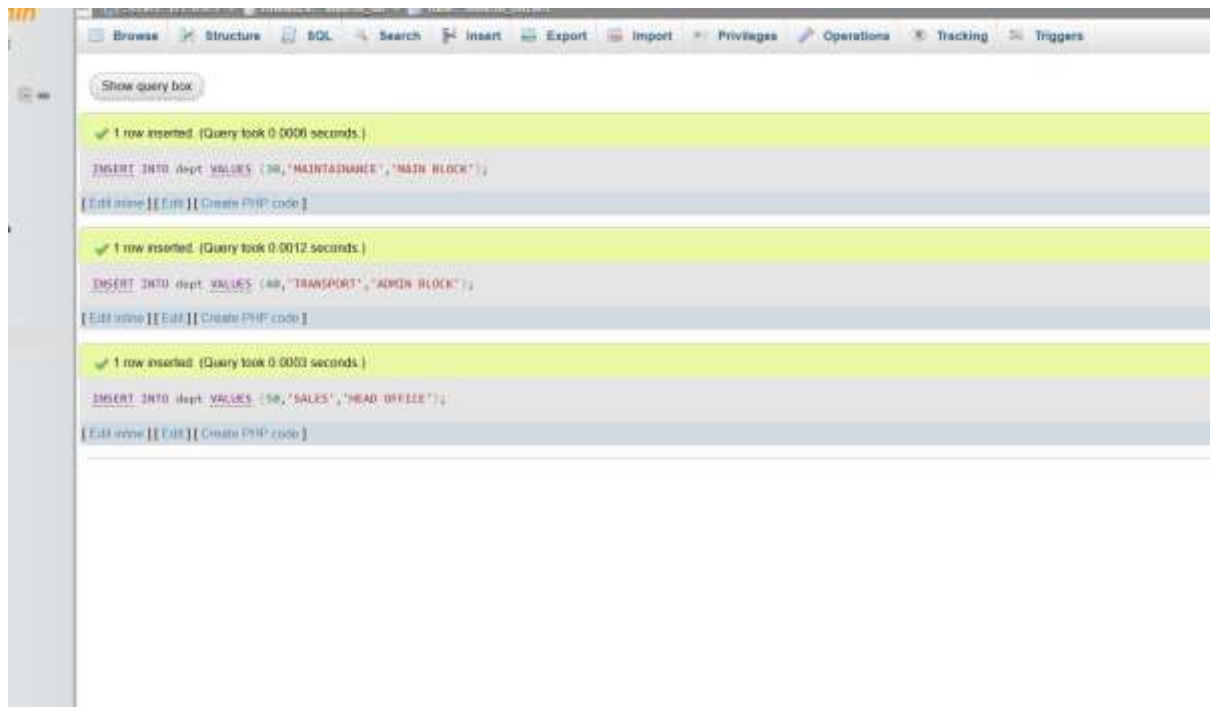
1 row created.



Followed three more rows are inserted by using **more than one row syntax**.

SQL >insert into dept values (&DNO, '&DNAME', '&LOCATION');





Enter value for DNO:30

Enter value for DNAME: MAINTAINANCE

Enter value for LOCATION: MAIN BLOCK
old 1 : Insert into dept values (&DNO, '&DNAME', '&LOCATION');

new 1: Insert into dept values (30,' MAINTAINANCE',' MAIN BLOCK');

1 row created.

SQL >/

Enter value for DNO:40

Enter value for DNAME: TRANSPORT

Enter value for LOCATION: ADMIN BLOCK

old 1 : Insert into dept values (&DNO, '&DNAME', '&LOCATION');

new 1: Insert into dept values (40,' TRANSPORT',' ADMIN BLOCK');

1 row created

SQL >/

Enter value for DNO:50

Enter value for DNAME: SALES Enter value for LOCATION: HEAD

OFFICE old 1 : Insert into dept values (&DNO, '&DNAME',
'&LOCATION'); new 1: Insert into dept values (40,' SALES ',' HEAD
OFFICE');

1 row created

Followed two more rows are inserted by using **specific column of the table**.

SQL >insert into dept (DNO,DNAME) values (60,'RESEARCH');

1 row created

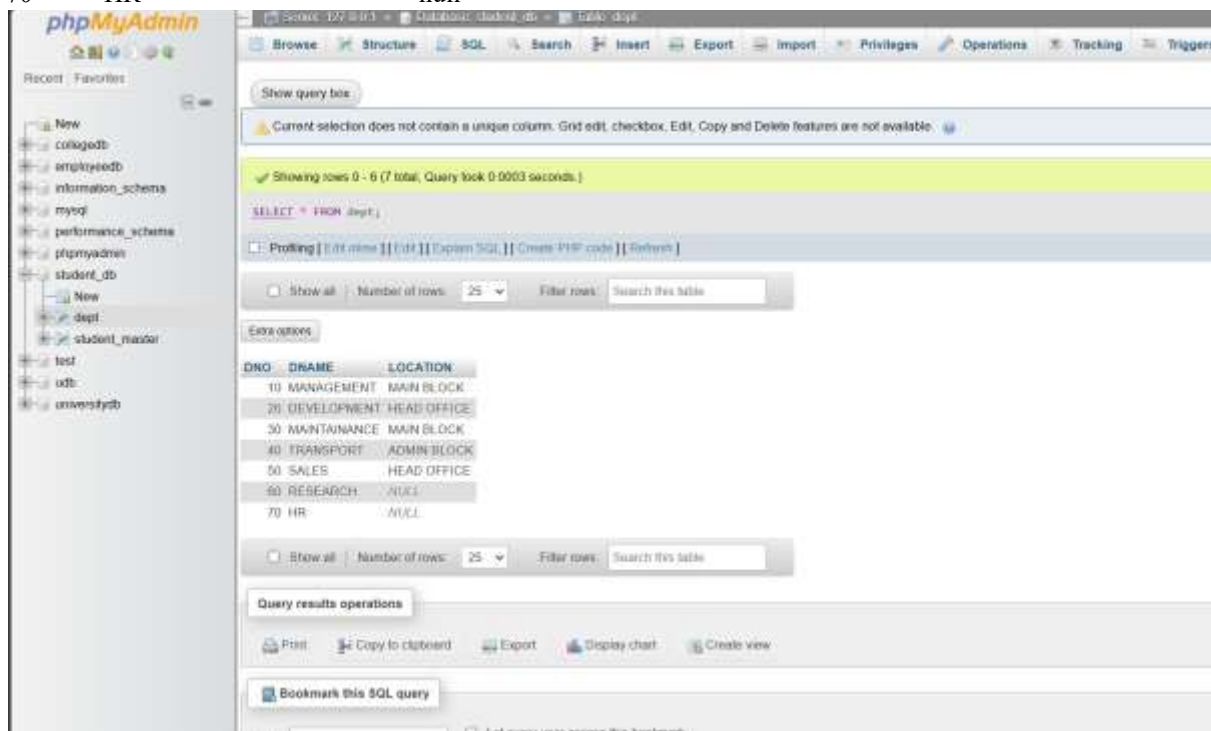
SQL >insert into dept (DNO,DNAME) values (70,'HR');

1 row created

To see the content of the dept table,

SQL >Select * from dept;

DNO	DNAME	LOCATION
-----	-----	-----
10	MANAGEMENT	MAIN BLOCK
20	DEVELOPMENT	HEAD OFFICE
30	MAINTAINANCE	MAIN BLOCK
40	TRANSPORT	ADMIN BLOCK
50	SALES	HEAD OFFICE
60	RESEARCH	null
70	HR	null



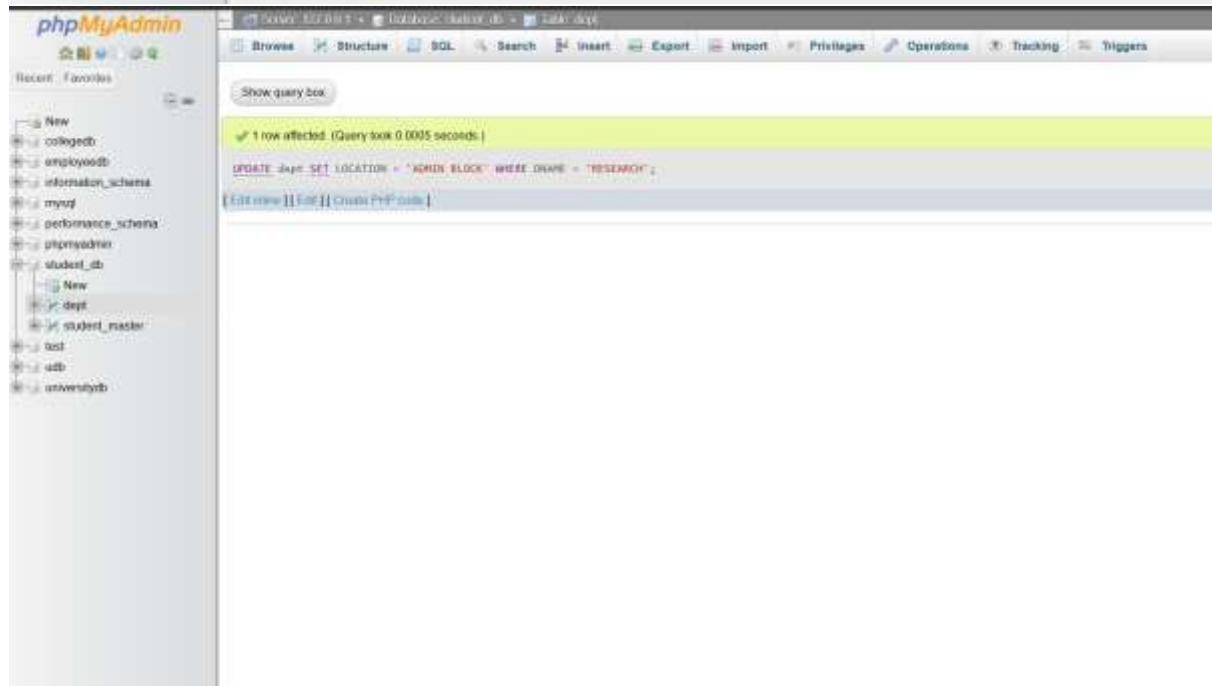
Problem 2: Update the dept table to assign location of research department to admin block.

SQL>update dept set location='admin block' where dname='research';

1 row updated

SQL>Select * from dept;

DNO	DNAME	LOCATION
-----	-----	-----
10	MANAGEMENT	MAIN BLOCK
20	DEVELOPMENT	HEAD OFFICE
30	MAINTAINANCE	MAIN BLOCK
40	TRANSPORT	ADMIN BLOCK
50	SALES	HEAD OFFICE
60	RESEARCH	ADMIN BLOCK
70	HR	null



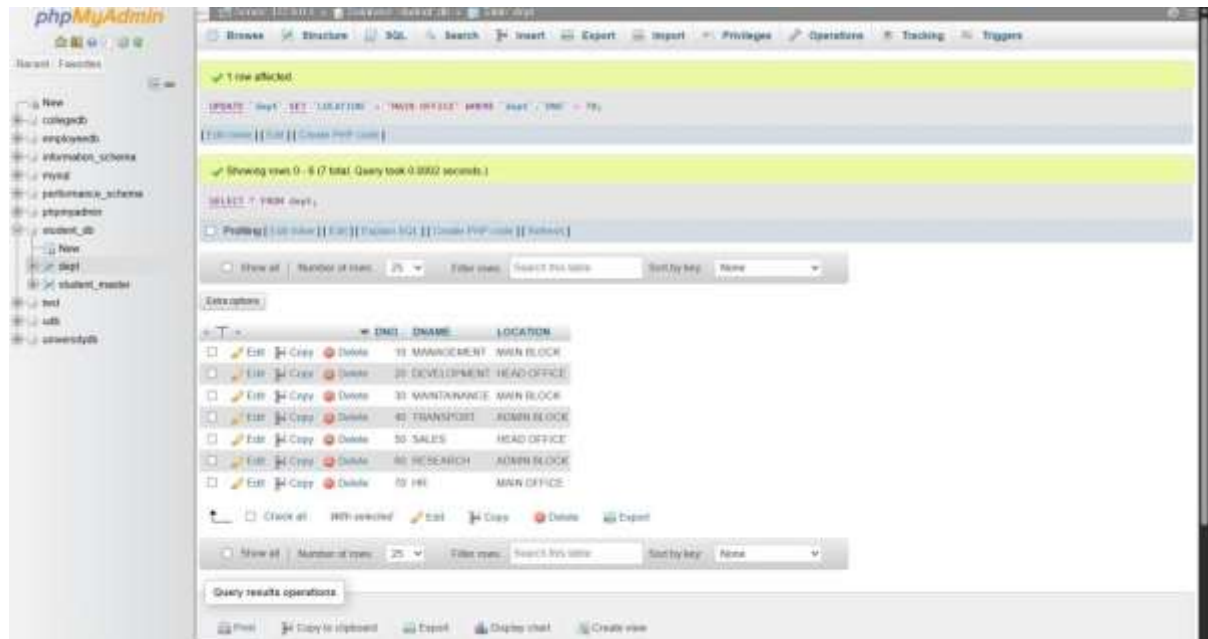
Problem 3: Delete hr department from the table.

SQL>delete from dept where dname='hr';

1 row deleted

SQL>Select * from dept;

DNO	DNAME	LOCATION
-----	-----	-----
10	MANAGEMENT	MAIN BLOCK
20	DEVELOPMENT	HEAD OFFICE
30	MAINTAINANCE	MAIN BLOCK
40	TRANSPORT	ADMIN BLOCK
50	SALES	HEAD OFFICE



RESULT:

Thus the Data Manipulation Language (DML) commands were studied and executed.

Exp.No:3 CREATION OF A DATABASE AND WRITING SQL QUERIES TO RETRIEVE INFORMATION FROM THE DATABASE

Date:

AIM:

To Create a Database and Writing SQL Queries to Retrieve Information From The Database.

PROCEDURE:

We will discuss the following select commands in this experiment.

- BASIC SELECT COMMANDS
- AGGREGATE FUNCTIONS
- SELECT COMMANDS USING SUBQUERIES
- SET OPERATIONS

BASIC SELECT COMMANDS

Consider the employee table which has the following columns and the values are also inserted one of the first two methods,

```
INSERT INTO EMP (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO) VALUES (7369, 'SMITH', 'CLERK', 7902, '1980-12-17', 800, NULL, 20), (7499, 'ALLEN', 'SALESMAN', 7698, '1981-02-20', 1600, 300, 30), (7521, 'WARD', 'SALESMAN', 7698, '1981-02-22', 1250, 500, 30), (7566, 'JONES', 'MANAGER', 7839, '1981-04-02', 2975, NULL, 20), (7654, 'MARTIN', 'SALESMAN', 7698, '1981-09-28', 1250, 1400, 30), (7698, 'BLAKE', 'MANAGER', 7839, '1981-05-01', 2850, NULL, 30), (7782, 'CLARK', 'MANAGER', 7839, '1981-06-09',
```

2450, NULL, 10), (7788, 'SCOTT', 'ANALYST', 7566, '1987-04-19', 3000, NULL, 20), (7839, 'KING', 'PRESIDENT', NULL, '1981-11-17', 5000, NULL, 10), (7844, 'TURNER', 'SALESMAN', 7698, '1981-09-08', 1500, 0, 30), (7876, 'ADAMS', 'CLERK', 7788, '1987-05-23', 1100, NULL, 20), (7900, 'JAMES', 'CLERK', 7698, '1981-12-03', 950, NULL, 30), (7902, 'FORD', 'ANALYST', 7566, '1981-12-03', 3000, NULL, 20), (7934, 'MILLER', 'CLERK', 7782, '1982-01-23', 1300, NULL, 10);

Showing rows 0 - 13 (14 total; Query took 0.0003 seconds.)

SELECT * FROM 'emp'

☐ Profiling: [Edit table](#) [\[\[Edit \]\]](#) [Explain SQL](#) [\[\[Create PHP code \]\]](#) [\[\[Refresh \]\]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
☐	7369	SMITH	CLERK	7902	1980-12-17	800	NULL	20
☐	7369	ALLEN	SALESMAN	7900	1981-02-20	1600	300	30
☐	7521	WARD	SALESMAN	7900	1981-02-22	1250	500	30
☐	7566	JONES	MANAGER	7939	1981-04-02	2975	NULL	20
☐	7654	MARTIN	SALESMAN	7900	1981-09-28	1250	1400	30
☐	7698	BLAKE	MANAGER	7939	1981-05-01	2850	NULL	30
☐	7702	CLARK	MANAGER	7939	1981-06-09	2450	NULL	10
☐	7788	SCOTT	ANALYST	7566	1987-04-19	3000	NULL	20
☐	7839	KING	PRESIDENT	NULL	1981-11-17	5000	NULL	10
☐	7844	TURNER	SALESMAN	7698	1981-09-08	1500	0	30
☐	7876	ADAMS	CLERK	7788	1987-05-23	1100	NULL	20
☐	7900	JAMES	CLERK	7698	1981-12-03	950	NULL	30
☐	7902	FORD	ANALYST	7566	1981-12-03	3000	NULL	20
☐	7934	MILLER	CLERK	7782	1982-01-23	1300	NULL	10

☐ Check all | With selected | [Edit](#) | [Copy](#) | [Delete](#) | [Export](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

phpMyAdmin

Recent Favorites

- New
- collegedb
- employees
- information_schema
- mysql
- performance_schema
- phpmyadmin
- student_db
- New
- pc_dept
- pc_emp
- pc_student_master
- test
- edu
- universitydb

Structure SQL Search Insert Export Import Privileges Operations Tracking Triggers

Show query box

14 rows inserted (Query took 0.0004 seconds.)

```

INSERT INTO emp (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO) VALUES ((7369,'SMITH','CLERK',7902,'1980-12-17',800,NULL,20), (7369,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30), (7521,'WARD','SALESMAN',7900,'1981-02-22',1250,500,30), (7566,'JONES','MANAGER',7939,'1981-04-02',2975,NULL,20), (7654,'MARTIN','SALESMAN',7900,'1981-09-28',1250,1400,30), (7698,'BLAKE','MANAGER',7939,'1981-05-01',2850,NULL,30), (7702,'CLARK','MANAGER',7939,'1981-06-09',2450,NULL,10), (7788,'SCOTT','ANALYST',7566,'1987-04-19',3000,NULL,20), (7839,'KING','PRESIDENT',NULL,'1981-11-17',5000,NULL,10), (7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30), (7876,'ADAMS','CLERK',7788,'1987-05-23',1100,NULL,20), (7900,'JAMES','CLERK',7698,'1981-12-03',950,NULL,30), (7902,'FORD','ANALYST',7566,'1981-12-03',3000,NULL,20), (7934,'MILLER','CLERK',7782,'1982-01-23',1300,NULL,10));

```

[\[\[Edit \]\]](#) [\[\[Create PHP code \]\]](#)

SQL> select * from emp;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTN
7369	SMITH	CLERK	7902	17-DEC-80	800		20
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	20
7698	BLAKE	MANAGER	7839	01-MAY-81	2850		30
7782	CLARK	MANAGER	7839	09-JUN-81	2450		10
7788	SCOTT	ANALYST	7566	19-APR-87	3000		20
7839	KING	PRESIDENT		17-NOV-81	5000		10
7844	TURNER	SALESMAN	7698	08-SEP-81	1500	0	30
7876	ADAMS	CLERK	7788	23-MAY-87	1100		20
7900	JAMES	CLERK	7698	03-DEC-81	950		30
7902	FORD	ANALYST	7566	03-DEC-81	3000		20
7934	MILLER	CLERK	7782	23-JAN-82	1300		10

14 rows selected.

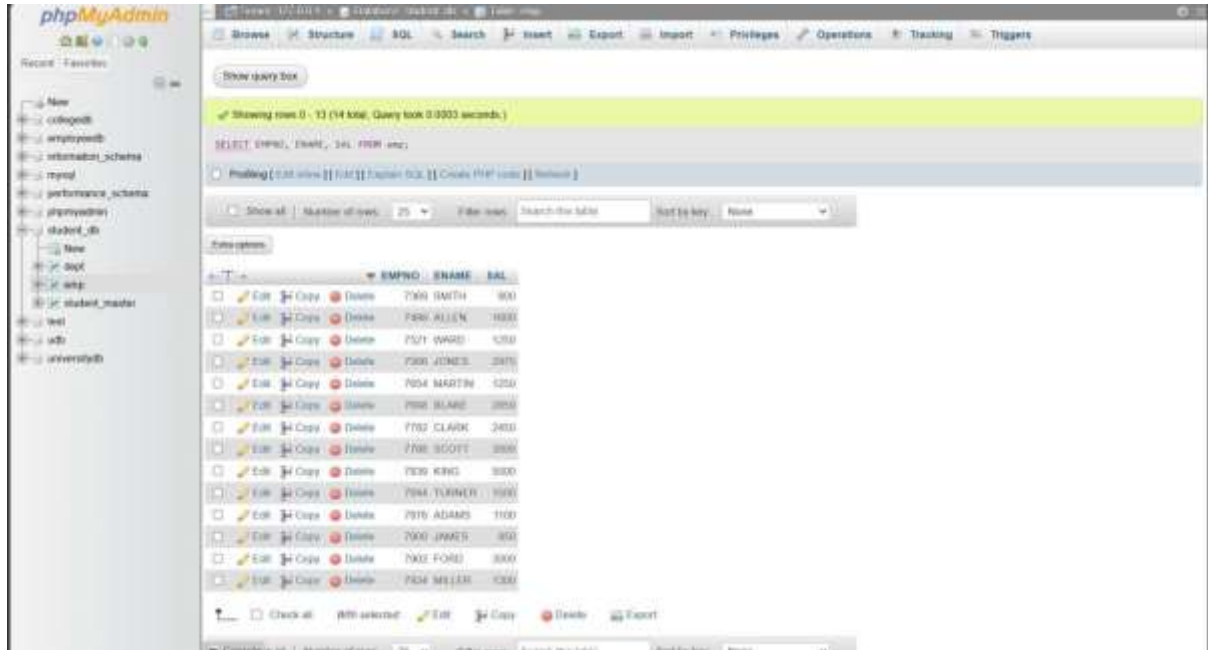
Problem 1: Display all employees' number, name and salary.

SQL> select empno,ename,sal from emp;

EMPNO	ENAME	SAL
7369	SMITH	800
7499	ALLEN	1600
7521	WARD	1250
7566	JONES	2975
7654	MARTIN	1250
7698	BLAKE	2850

7782 CLARK	2450 7788
SCOTT	3000
7839 KING	5000 7844
TURNER	1500
7876 ADAMS	1100
7900 JAMES	950
7902 FORD	3000
7934 MILLER	1300

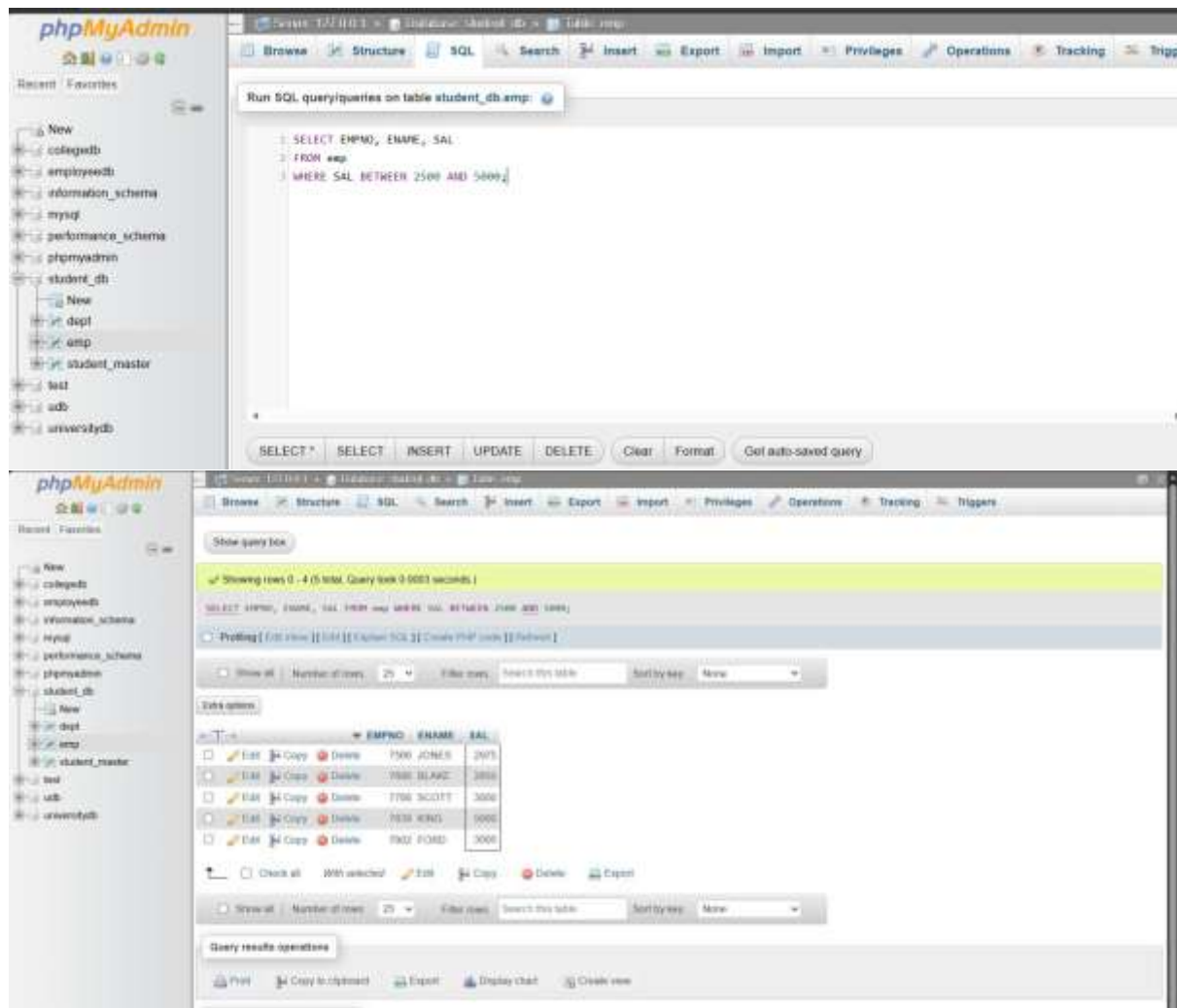
14 rows selected.



Problem 2: Display employees' number, name and salary of the range from 2500 to 5000.

SQL> select empno,ename,sal from emp where sal between 2500 and 5000;
 SELECT empno, ename, sal FROM emp WHERE sal BETWEEN 2500 AND 5000;

EMPNO	ENAME	SAL
7566	JONES	2975
7698	BLAKE	2850
7788	SCOTT	3000
7839	KING	5000
7902	FORD	3000

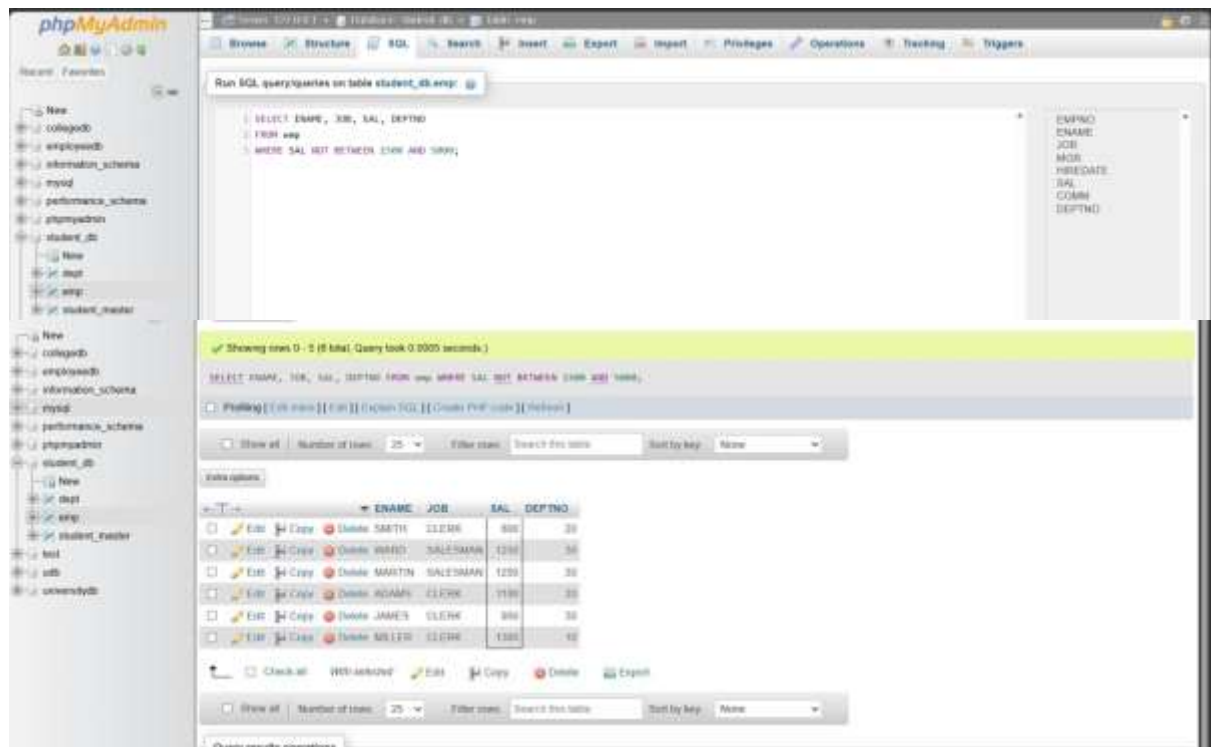


Problem 3: Display employees' name, job, department no and salary of the range not from 2500 to 5000.

SQL>select ename,job,sal,deptno from emp where sal not between 1500 and 5000;
 SELECT ename, job, sal, deptno FROM emp WHERE sal NOT BETWEEN 1500 AND 5000;

ENAME	JOB	SAL	DEPTNO
SMITH	CLERK	800	20 WARD
SALESMAN	1250	30	MARTIN
SALESMAN	1250	30	
ADAMS	CLERK	1100	20
JAMES	CLERK	950	30
MILLER	CLERK	1300	10

6 rows selected.

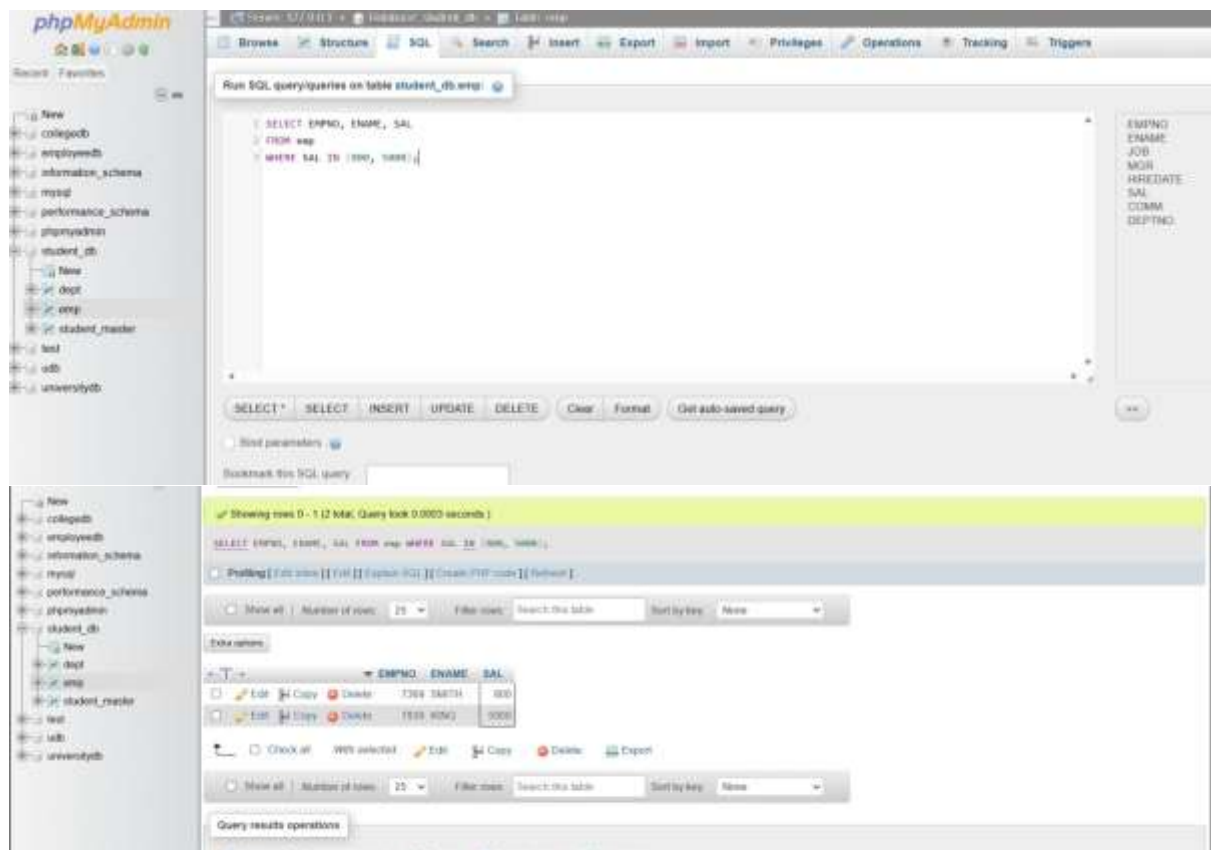


Problem 4: Find the employees' number, name and salary become 800 and 5000.

SQL> select empno,ename,sal from emp where sal in (800,5000);

SELECT empno, ename, sal FROM emp WHERE sal IN (800, 5000);

```
EMPNO ENAME    SAL
-----
7369 SMITH      800
7839 KING       5000
2 rows selected.
```

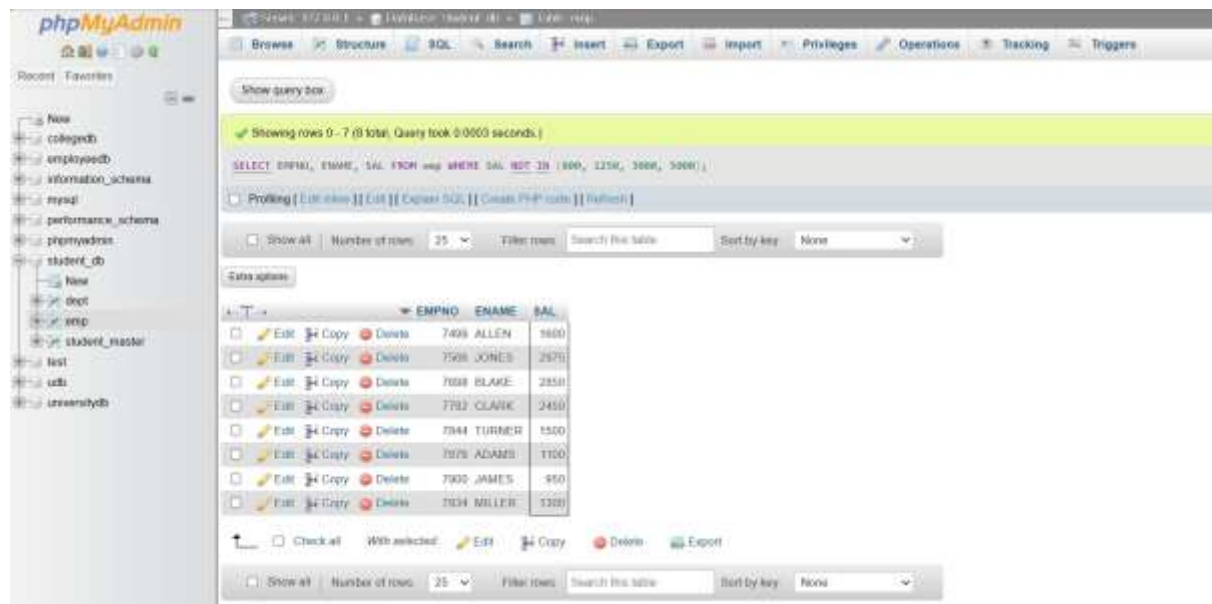


Problem 5: Find the employees' number, name and salary not become 800, 1250, 3000 and 5000

SQL> select empno,ename,sal from emp where sal not in(800,1250,3000,5000);
SELECT empno, ename, sal FROM emp WHERE sal **NOT IN** (800, 1250, 3000, 5000);

EMPNO	ENAME	SAL
7499	ALLEN	1600
7566	JONES	2975
7698	BLAKE	2850
7782	CLARK	2450
7844	TURNER	1500
7876	ADAMS	1100
7900	JAMES	950
7934	MILLER	1300

8 rows selected.



Problem 6: Find the employees' number, name and salary whose commission value is zero.

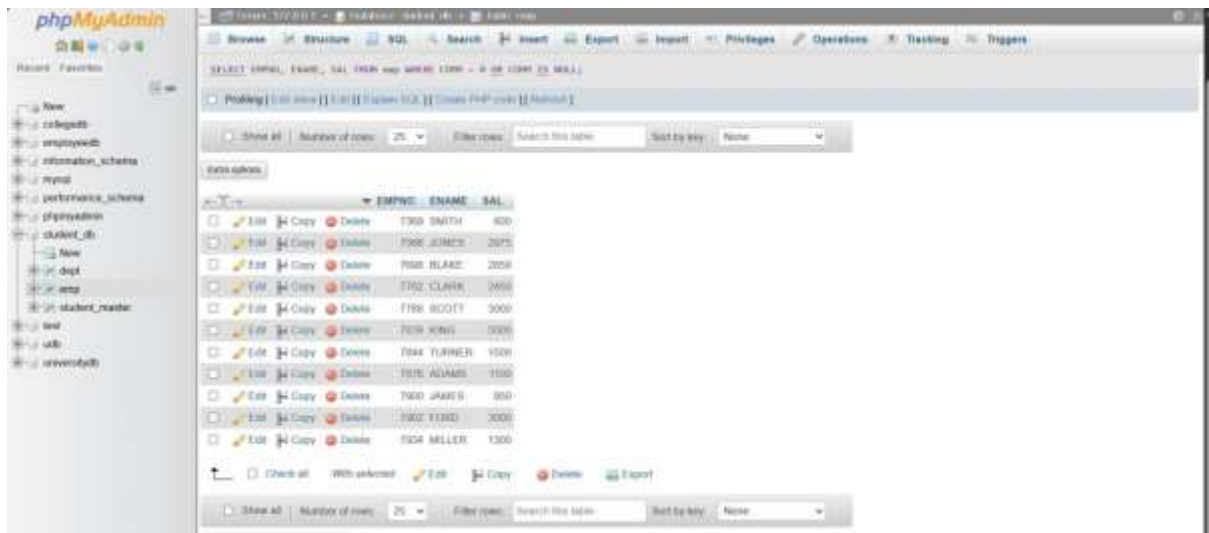
SQL> select empno,ename,sal from emp where comm is null;
SELECT empno, ename, sal FROM emp WHERE comm = 0 OR comm IS NULL;

EMPNO	ENAME	SAL
7369	SMITH	800
7566	JONES	2975
7698	BLAKE	2850
7782	CLARK	2450
7788	SCOTT	3000
7839	KING	5000
7876	ADAMS	1100
7900	JAMES	950
7902	FORD	3000
7934	MILLER	1300

10

rows

selected.

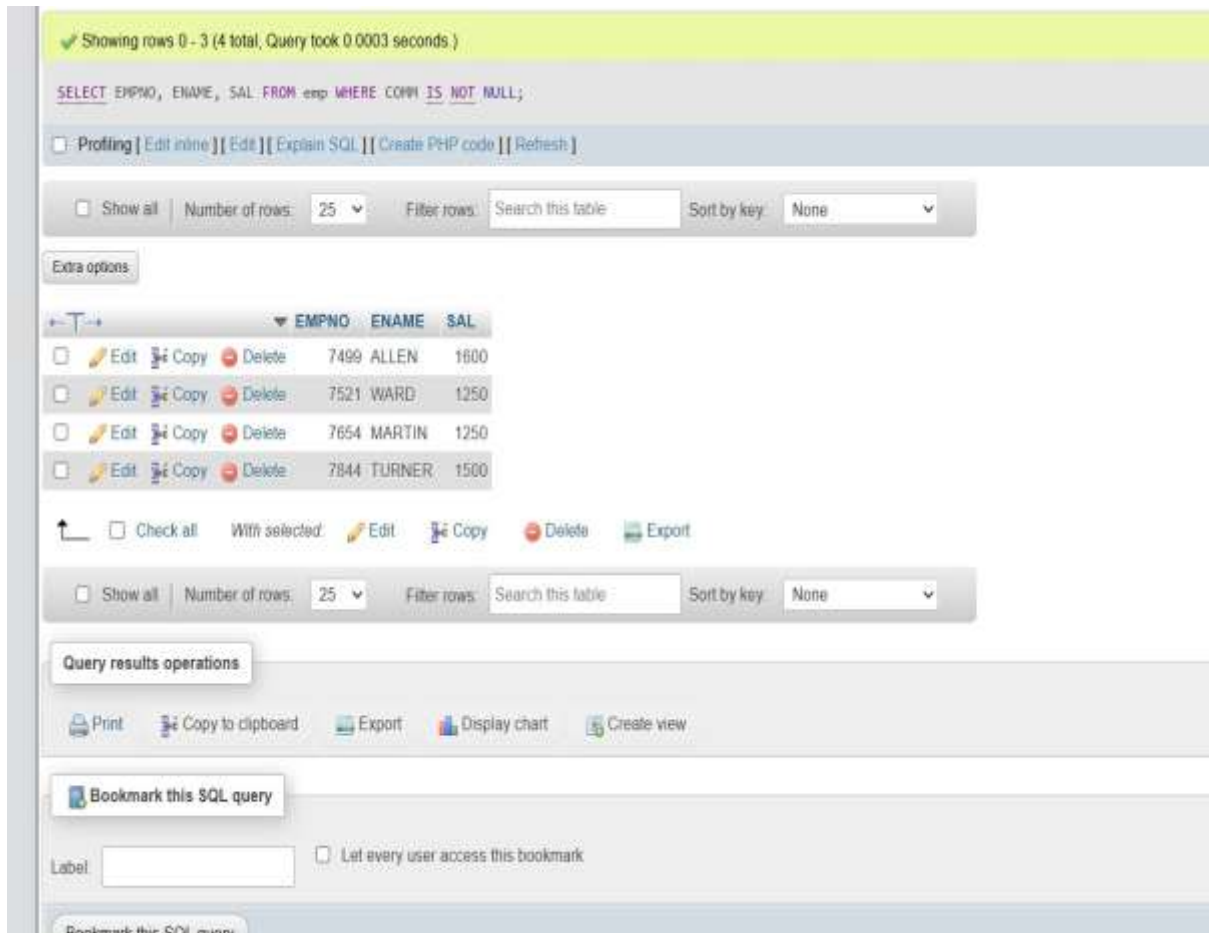


Problem 7: Find the employees' number, name and salary whose commission value other than zero.

SQL> select empno,ename,sal from emp where comm is not null;
 SELECT empno, ename, sal FROM emp WHERE comm **IS NOT NULL**;

EMPNO	ENAME	SAL
7499	ALLEN	1600
7521	WARD	1250
7654	MARTIN	1250
7844	TURNER	1500

rows selected.

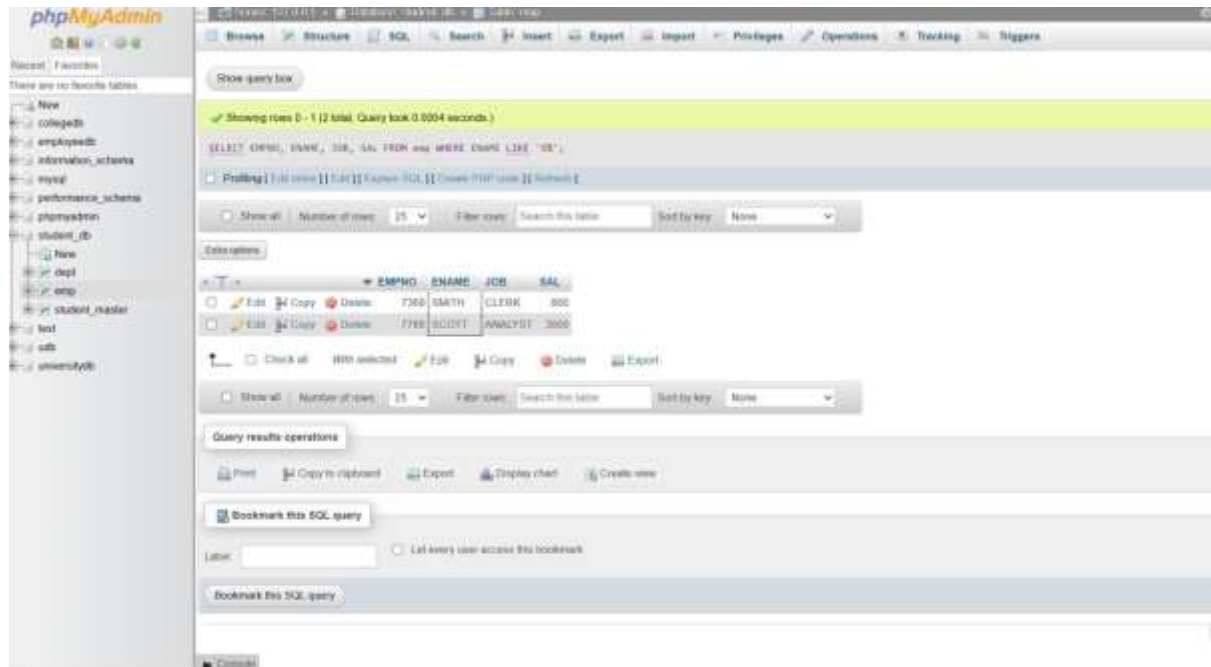


Problem 8: Find the employees' number, name, job and salary whose name starting with s.

SQL> select empno,ename,job,sal from emp where ename like'S%';
 SELECT empno, ename, job, sal FROM emp WHERE ename LIKE 'S%';

EMPNO	ENAME	JOB	SAL
7369	SMITH	CLERK	800
7788	SCOTT	ANALYST	3000

2 rows selected.

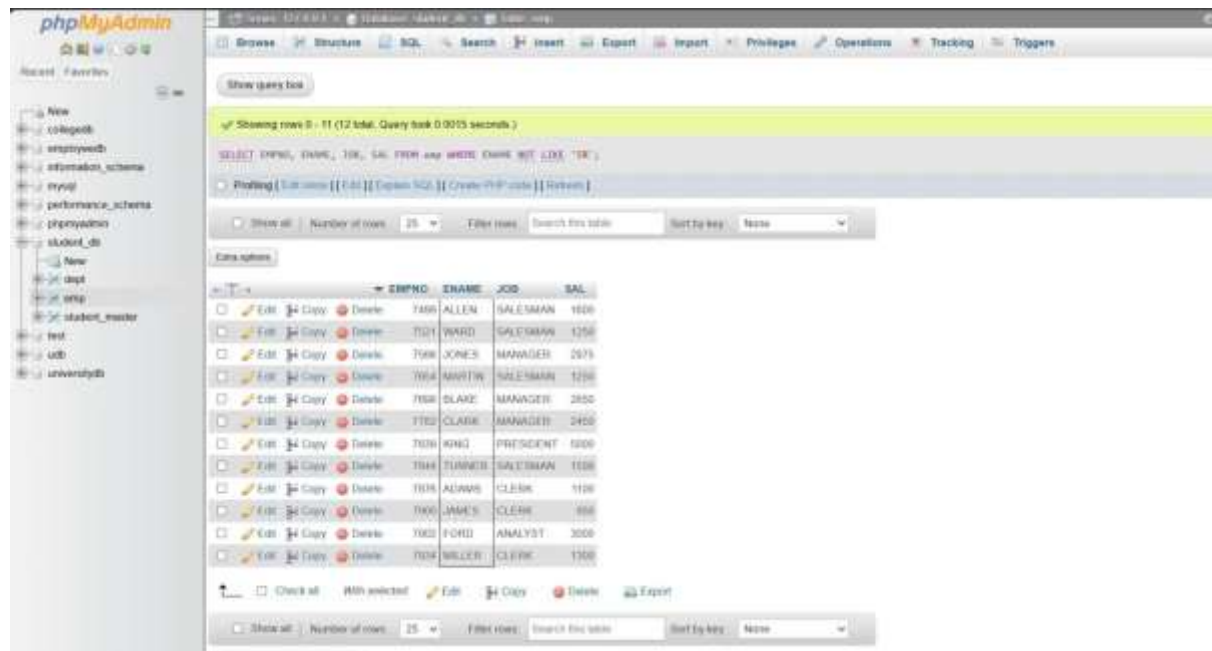


Problem 9: Find the employees' number, name, job and salary whose name not starting with s.

SQL> select empno,ename,job,sal from emp where job not like'S%';
 SELECT empno, ename, job, sal FROM emp WHERE job NOT LIKE 'S%';

EMPNO	ENAME	JOB	SAL
7369	SMITH	CLERK	800
7566	JONES	MANAGER	2975
7698	BLAKE	MANAGER	2850
7782	CLARK	MANAGER	2450
7788	SCOTT	ANALYST	3000
7839	KING	PRESIDENT	5000
7876	ADAMS	CLERK	1100
7900	JAMES	CLERK	950
7902	FORD	ANALYST	3000
7934	MILLER	CLERK	1300

rows selected.

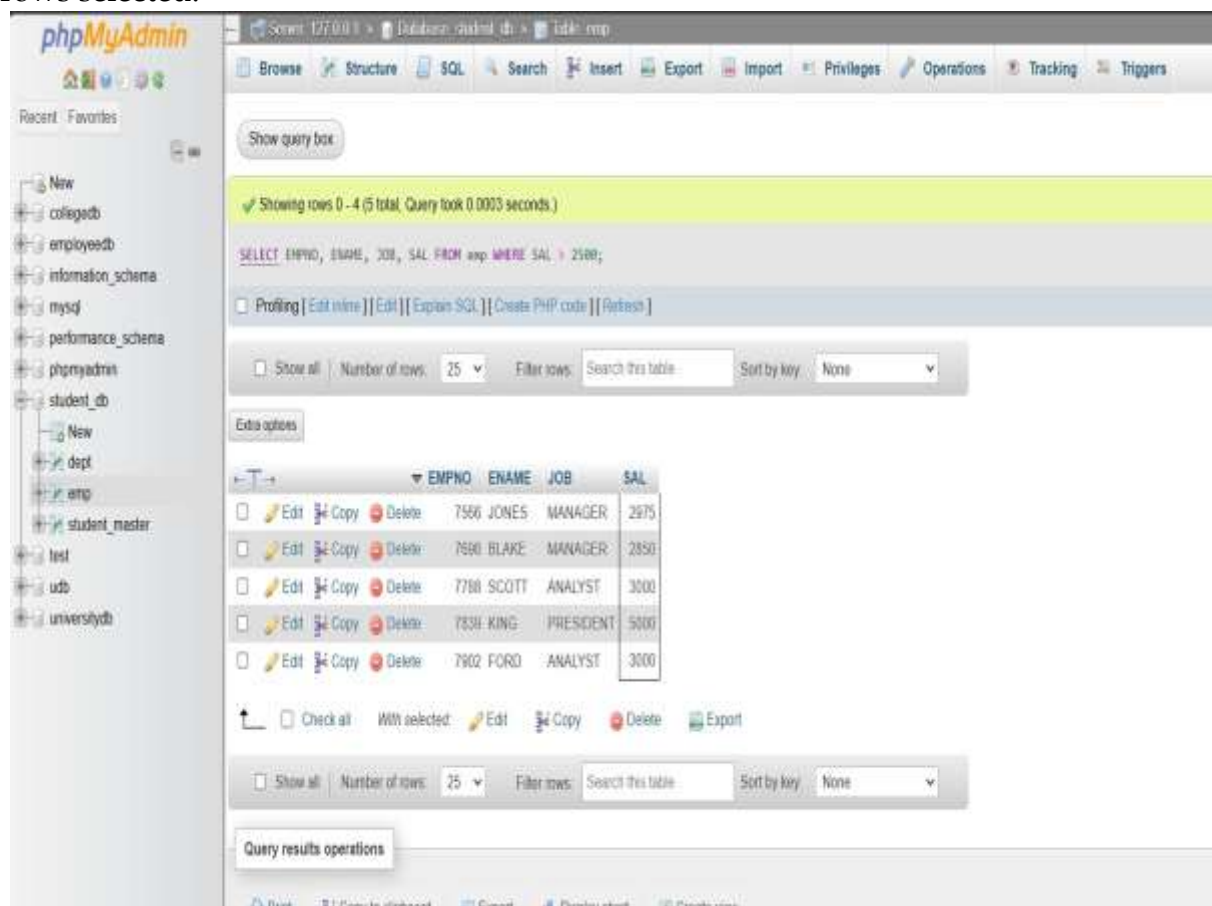


Problem 10: Find the employees' number, name, job and salary those getting more than 2500.

SQL> select ename,job,sal from emp where sal>2500;
 SELECT ename, job, sal FROM emp WHERE sal > 2500;

ENAME	JOB	SAL
JONES	MANAGER	2975
BLAKE	MANAGER	2850
SCOTT	ANALYST	3000
KING	PRESIDENT	5000
FORD	ANALYST	3000

5 rows selected.



Problem 11: Find the employees' number, name, job and salary those getting less than 2500

SQL> select ename,job,sal from emp where sal<2500;

SELECT ename, job, sal FROM emp WHERE sal < 2500;

ENAME	JOB	SAL
SMITH	CLERK	800
ALLEN	SALESMAN	1600
WARD	SALESMAN	1250
MARTIN	SALESMAN	1250
CLARK	MANAGER	2450
TURNER	SALESMAN	1500
ADAMS	CLERK	1100
JAMES	CLERK	950
MILLER	CLERK	1300

rows selected.

The screenshot shows the phpMyAdmin web interface. The left sidebar displays a database structure tree with 'student_db' selected. The main panel shows the results of the query 'SELECT EMPNO, ENAME, JOB, SAL FROM emp WHERE SAL < 2500;'. A status bar indicates 'Showing rows 0 - 8 (9 total, Query took 0.0003 seconds)'. Below the query, there are options for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (Search this table), and 'Sort by key' (None). The query results are displayed in a table with columns EMPNO, ENAME, JOB, and SAL. The table contains 9 rows of data, all of which are displayed. At the bottom, there are 'Query results operations' buttons: 'Check all', 'With selected', 'Edit', 'Copy', 'Delete', and 'Export'.

EMPNO	ENAME	JOB	SAL
7369	SMITH	CLERK	800
7499	ALLEN	SALESMAN	1600
7521	WARD	SALESMAN	1250
7654	MARTIN	SALESMAN	1250
7782	CLARK	MANAGER	2450
7844	TURNER	SALESMAN	1500
7876	ADAMS	CLERK	1100
7900	JAMES	CLERK	950
7934	MILLER	CLERK	1300

Problem 12: with respect to salary display the employee database no, name, job and salary from smallest value to largest value.

SQL> select empno,ename,job,sal from emp order by sal;
SELECT empno, ename, job, sal FROM emp ORDER BY sal;

EMPNO	ENAME	JOB	SAL
7369	SMITH	CLERK	800
7900	JAMES	CLERK	950
7876	ADAMS	CLERK	1100
7521	WARD	SALESMAN	1250
7654	MARTIN	SALESMAN	1250
7934	MILLER	CLERK	1300
7844	TURNER	SALESMAN	1500
7499	ALLEN	SALESMAN	1600
7782	CLARK	MANAGER	2450
7698	BLAKE	MANAGER	2850
7566	JONES	MANAGER	2975
7788	SCOTT	ANALYST	3000
7902	FORD	ANALYST	3000
7839	KING	PRESIDENT	5000

14 rows selected.

The screenshot shows the phpMyAdmin interface for a database named 'student_db'. The 'emp' table is selected, and the data is displayed in a table view. The columns are EMPNO, ENAME, JOB, and SAL. The data is sorted by salary in ascending order. The table contains 14 rows of data.

EMPNO	ENAME	JOB	SAL
7369	SMITH	CLERK	800
7900	JAMES	CLERK	950
7876	ADAMS	CLERK	1100
7521	WARD	SALESMAN	1250
7654	MARTIN	SALESMAN	1250
7934	MILLER	CLERK	1300
7644	TURNER	SALESMAN	1500
7499	ALLEN	SALESMAN	1600
7782	CLARK	MANAGER	2450
7698	BLAKE	MANAGER	2850
7566	JONES	MANAGER	2975
7902	FORD	ANALYST	3000
7788	SCOTT	ANALYST	3000
7839	KING	PRESIDENT	5000

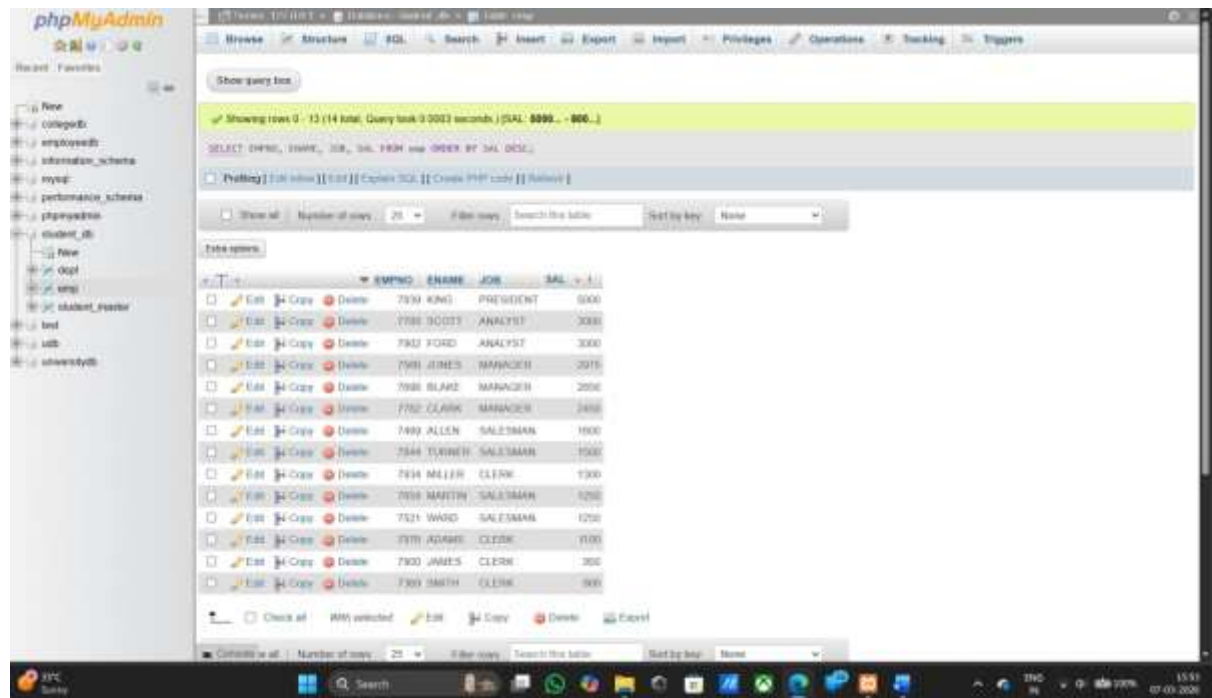
Problem 13: with respect to salary display the employee database no, name, job and salary from largest value to smallest value.

SQL> select empno,ename,job,sal from emp order by sal desc;

SELECT empno, ename, job, sal FROM emp ORDER BY sal DESC;

EMPNO	ENAME	JOB	SAL
-----	-----	-----	-----
7839	KING	PRESIDENT	5000
7788	SCOTT	ANALYST	3000
7902	FORD	ANALYST	3000
7566	JONES	MANAGER	2975
7698	BLAKE	MANAGER	2850
7782	CLARK	MANAGER	2450
7499	ALLEN	SALESMAN	1600
7844	TURNER	SALESMAN	1500
7934	MILLER	CLERK	1300
7521	WARD	SALESMAN	1250
7654	MARTIN	SALESMAN	1250
7876	ADAMS	CLERK	1100
7900	JAMES	CLERK	950
7369	SMITH	CLERK	800

14 rows selected.



AGGREGATE FUNCTIONS

Consider the below emp table

SQL> SELECT * FROM EMP;

ENO	ENAME	JOB	SAL
1001	STEVE	SALESMAN	1500
1002	ADAM	CLERK	1000
5220			
1004	JAMES	DIRECTOR	7000

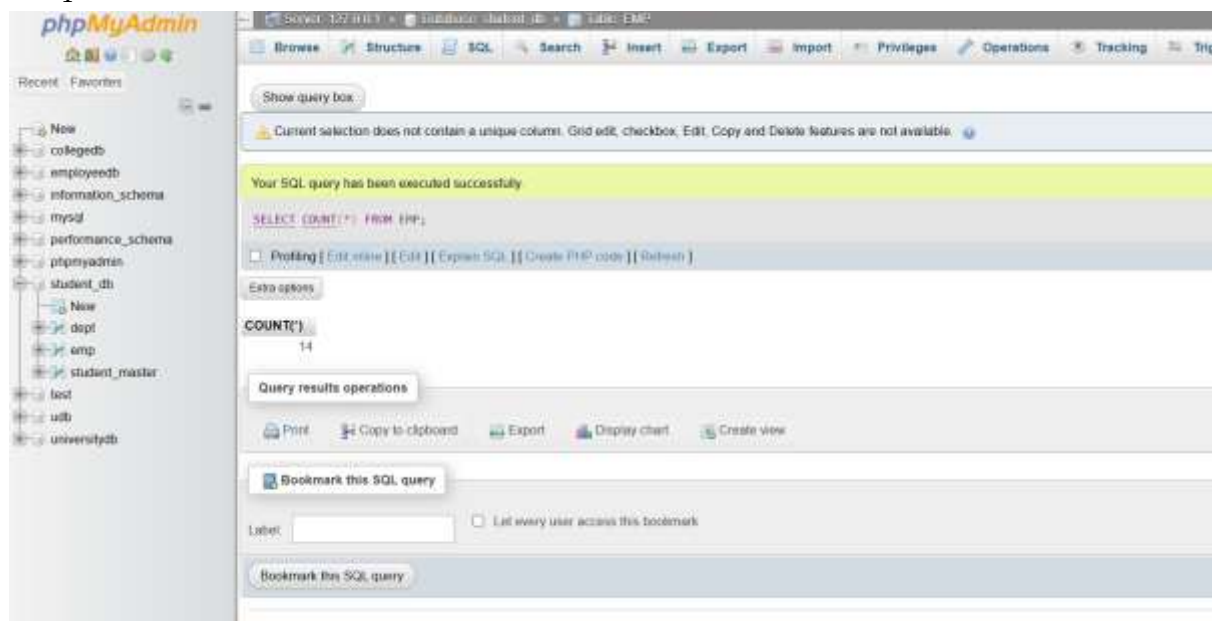
Problem 1: Number of records in the employee table.

SQL> SELECT COUNT(*) FROM EMP;

SELECT COUNT(*) FROM EMP;

COUNT(*)

4



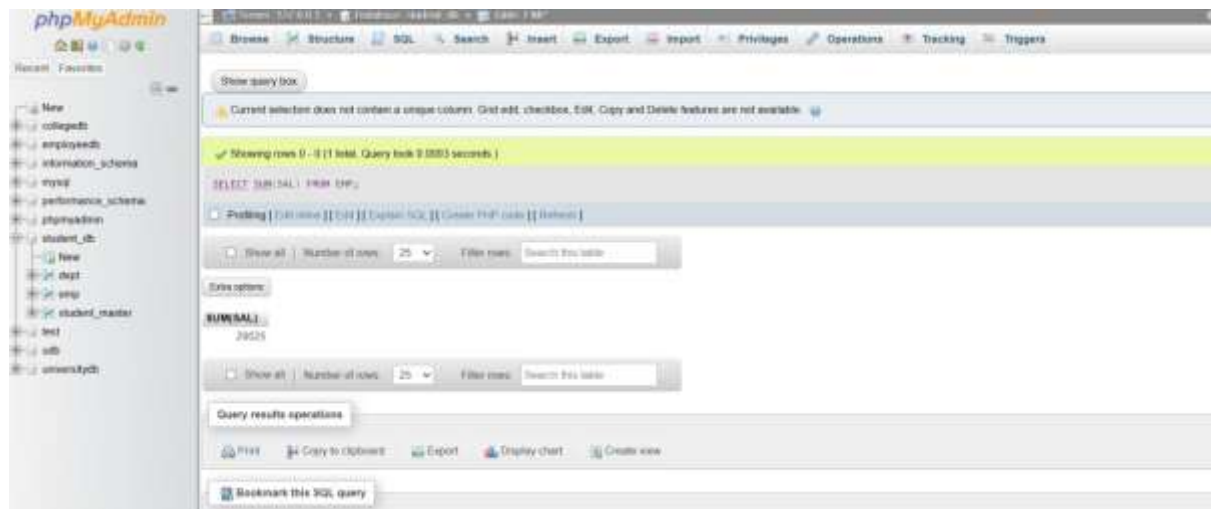
Problem 2: Find the total salary of the employee table.

SQL> SELECT SUM(SAL) FROM EMP;

SELECT SUM(SAL) FROM EMP;

SUM(SAL)

14720



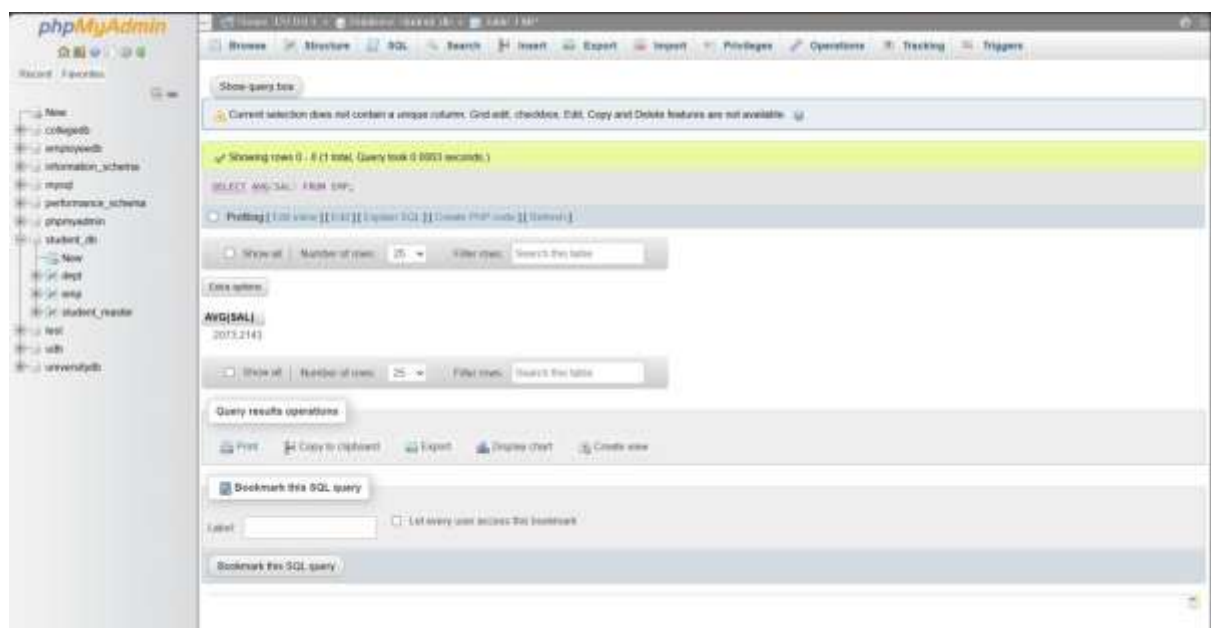
Problem 3: Find the average salary of the employee table.

SQL> SELECT AVG(SAL) FROM EMP;

SELECT AVG(SAL) FROM EMP;

AVG(SAL)

3680



Problem 4: Find the maximum salary of the employee table.

SQL> SELECT MAX(SAL) FROM EMP;

SELECT MAX(SAL) FROM EMP;

MAX(SAL)

7000

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)
[Operations](#)
[Tracking](#)
[Triggers](#)

Show query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

✓ Showing rows 0 - 0 (1 total, Query took 0.0003 seconds.)

```
SELECT MAX(SAL) FROM EMP;
```

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all
 Number of rows: 25
 Filter rows:

Extra options

MAX(SAL)
5000

☐ Show all
 Number of rows: 25
 Filter rows:

Query results operations

[Print](#)
[Copy to clipboard](#)
[Export](#)
[Display chart](#)
[Create view](#)

[Bookmark this SQL query](#)

Label:
☐ Let every user access this bookmark

Problem 5: Find the minimum salary of the employee table.

SQL> SELECT MIN(SAL) FROM EMP;

SELECT MIN(SAL) FROM EMP;

MIN(SAL)

1000

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)
[Privileges](#)
[Operations](#)
[Tracking](#)
[Triggers](#)

Show query box

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

✓ Showing rows 0 - 0 (1 total, Query took 0.0005 seconds.)

```
SELECT MIN(SAL) FROM EMP;
```

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all
 Number of rows: 25
 Filter rows:

Extra options

MIN(SAL)
800

☐ Show all
 Number of rows: 25
 Filter rows:

Query results operations

➤ APPLYING GROUP BY

Consider the below emp table,

SQL> select * from emp;

EMPNO	ENAME	JOB	SAL	DEPTNO
7369	SMITH	CLERK	800	20
7499	ALLEN	SALESMAN	1600	30
7521	WARD	SALESMAN	1250	30
7566	JONES	MANAGER	2975	20
7654	MARTIN	SALESMAN	1250	30
7698	BLAKE	MANAGER	2850	30
7782	CLARK	MANAGER	2450	10
7788	SCOTT	ANALYST	3000	20
7839	KING	PRESIDENT	5000	10
7844	TURNER	SALESMAN	1500	30
7876	ADAMS	CLERK	1100	20
7900	JAMES	CLERK	950	30
7902	FORD	ANALYST	3000	20
7934	MILLER	CLERK	1300	10

rows selected.

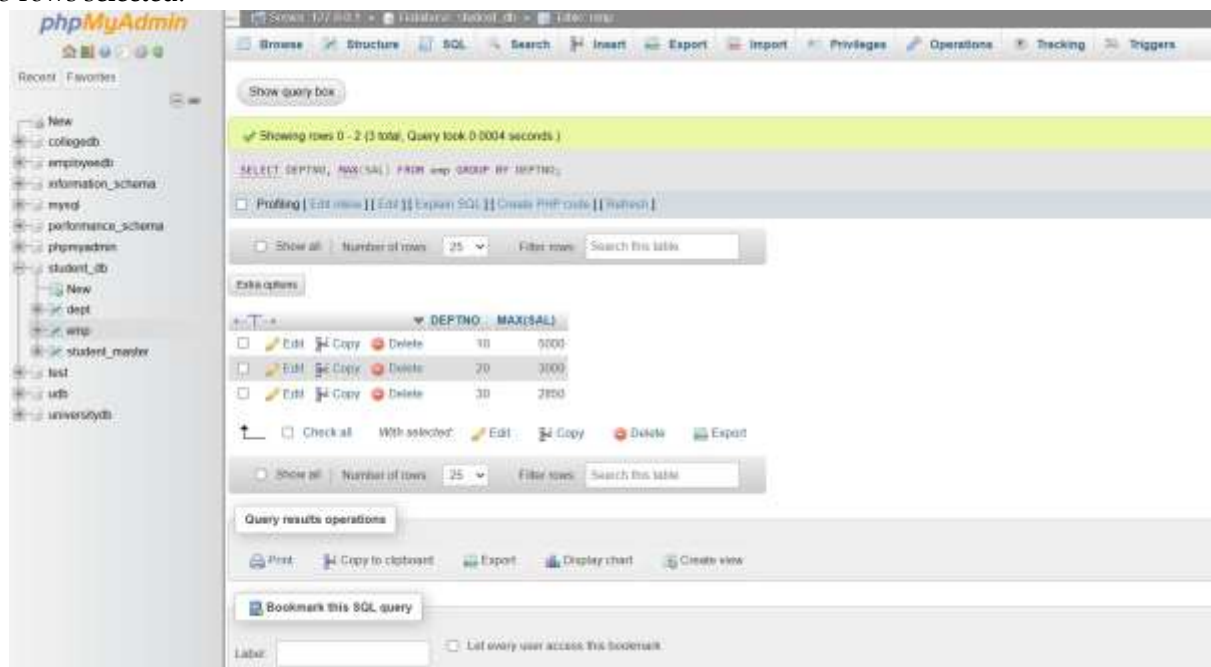
Problem 1: Find the maximum salary of each department.

SQL> select deptno,max(sal) from emp group by deptno;

SELECT deptno, MAX(sal) FROM emp GROUP BY deptno;

DEPTNO	MAX(SAL)
10	5000
20	3000
30	2850

3 rows selected.



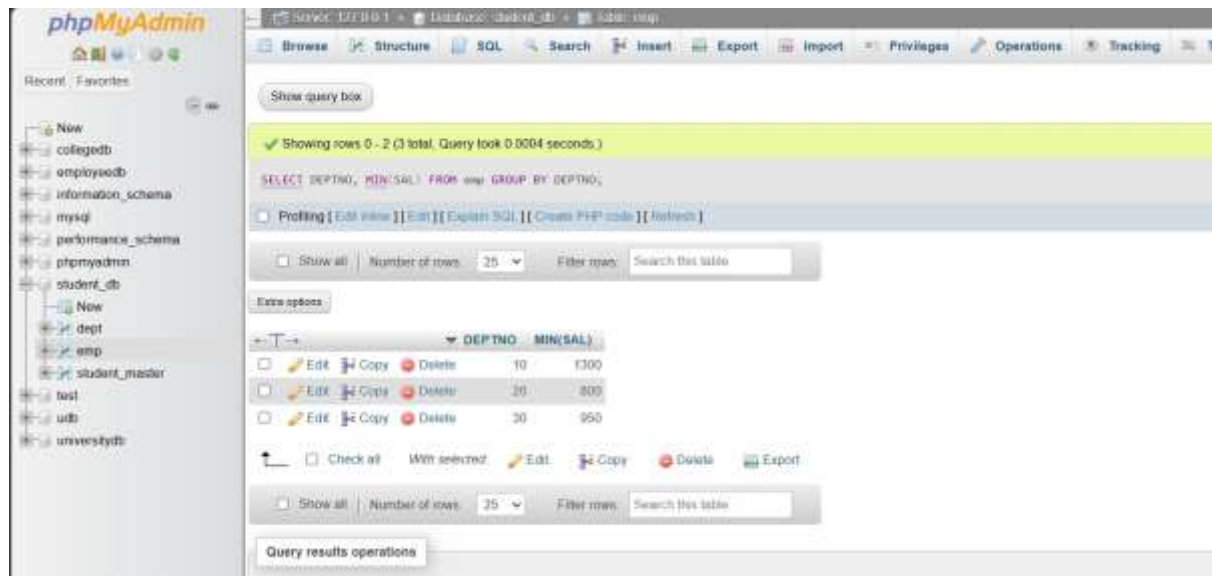
Problem 2: Find the minimum salary of each department.

SQL> select deptno,min(sal) from emp group by deptno;

SELECT deptno, MIN(sal) FROM emp GROUP BY deptno;

DEPTNO	MIN(SAL)
10	1300
20	800
30	950

3 rows selected.



Problem 3: Find the department whose maximum salary is greater than 3000.

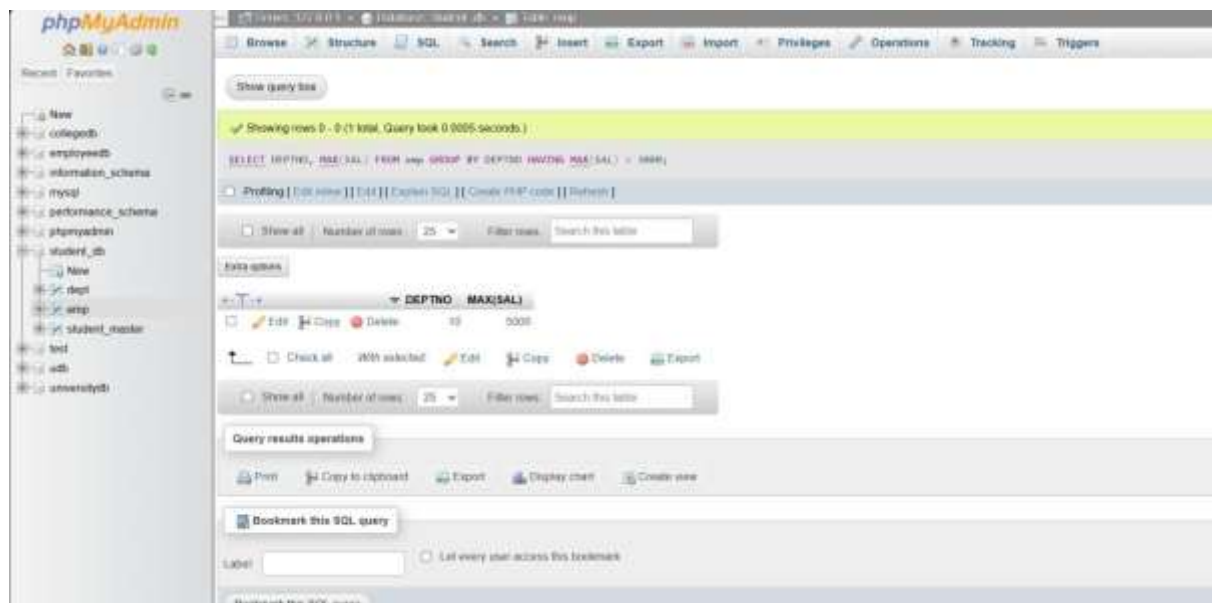
SQL> select deptno,max(sal) from emp group by deptno having max(sal)>3000;

SELECT deptno, MAX(sal) FROM emp GROUP BY deptno HAVING MAX(sal) > 3000;

DEPTNO MAX(SAL)

30 2850

1 row selected.



Problem 4: Find the department whose minimum salary is greater than 1000.

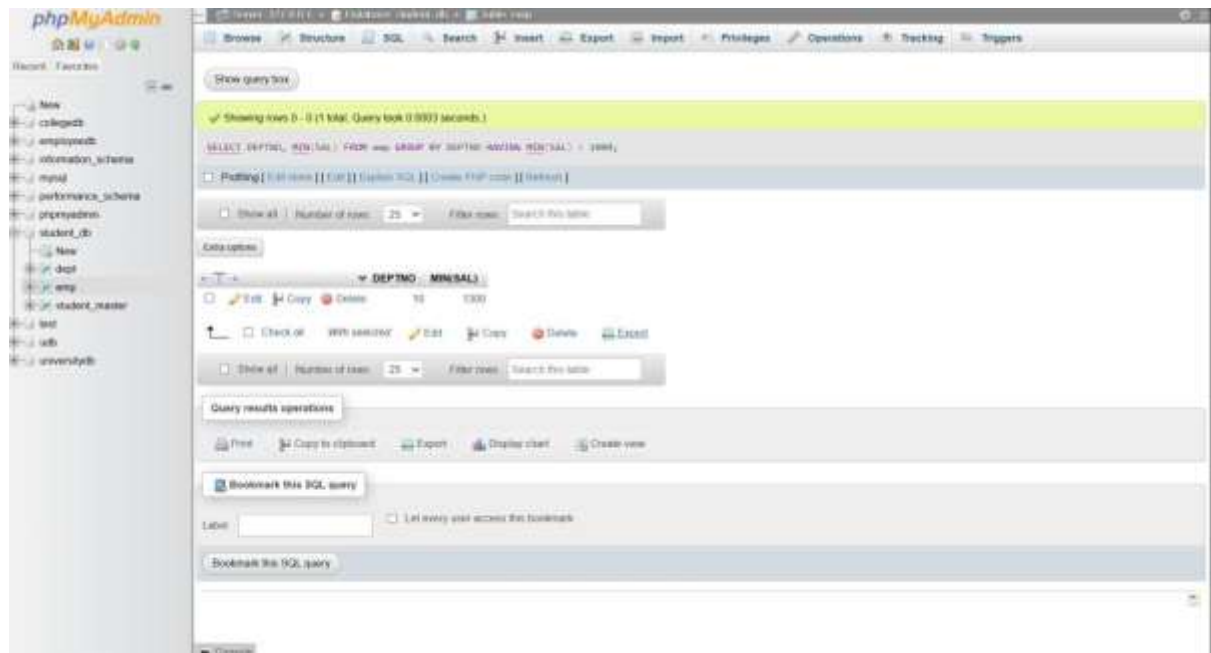
SQL> select deptno,min(sal) from emp group by deptno having min(sal)>1000; SELECT

deptno, MIN(sal) FROM emp GROUP BY deptno HAVING MIN(sal) > 1000;

DEPTNO MIN(SAL)

10 1300

1 row selected.



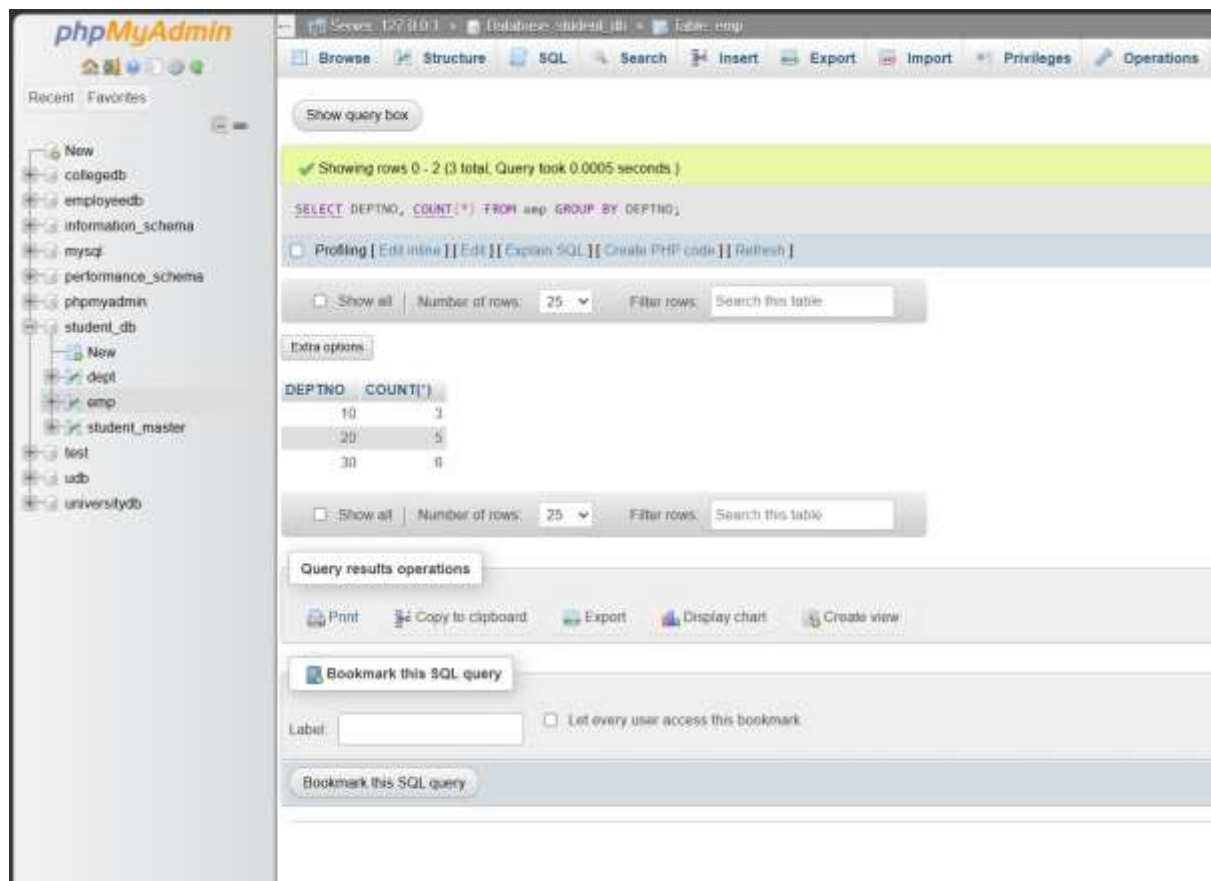
Problem 5: Find the number of employees working in each department.

SQL> select deptno,count(*) from emp group by deptno;

SELECT deptno, COUNT(*) FROM emp GROUP BY deptno;

DEPTNO	COUNT(*)
10	3
20	5
30	6

3 rows selected.



SELECT COMMANDS USING SUBQUERIES (APPLYING IN, ALL, ANY, EXISTS, NOT EXISTS)

IN – The inner query return records matched with outer query records

ALL- The inner query returns more than one record for the comparison

ANY- The inner query returns single record for the comparison

EXIST – checking the availability of tuples in the inner query

To execute these operations we can take the previous emp table and one more table called dept, and its content become

SQL>select * from dept;

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON4

4 rows selected.

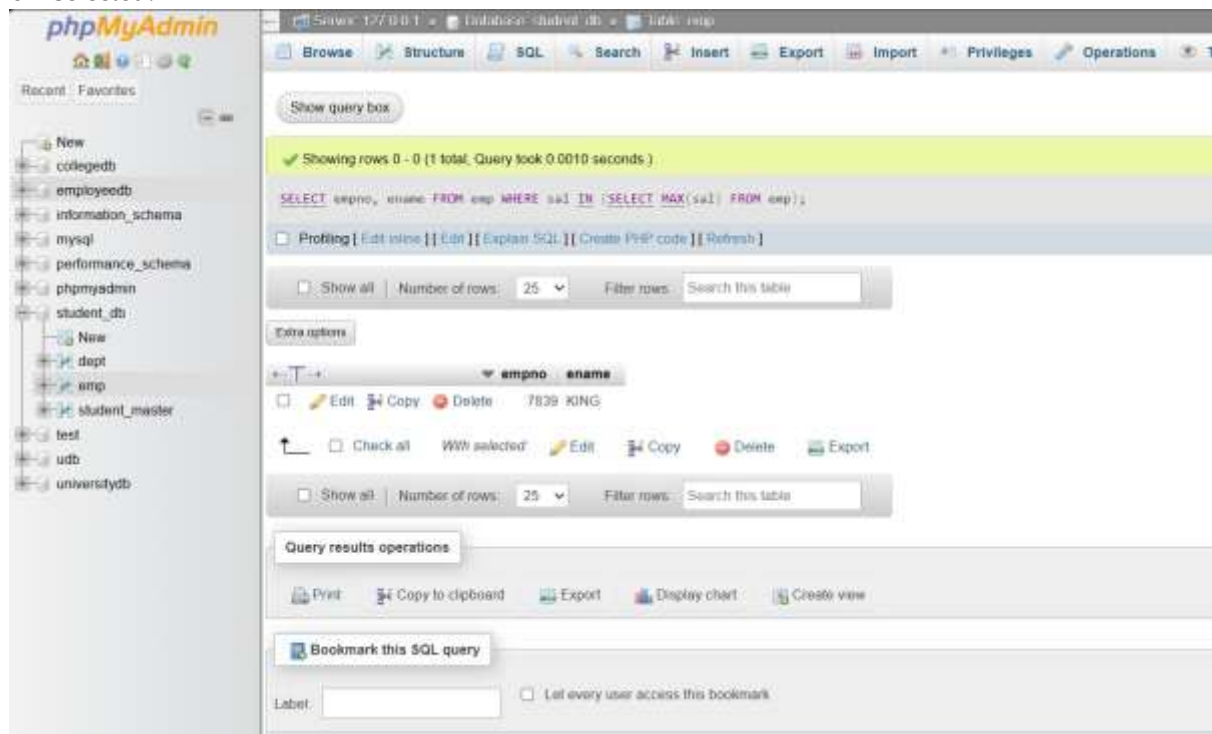
Problem 1: Find the employee who is getting highest salary.

SQL> select empno,ename from emp where sal in(select max(sal) from emp); SELECT empno, ename FROM emp WHERE sal IN (SELECT MAX(sal) FROM emp);

EMPNO ENAME

7839 KING 1

row selected.

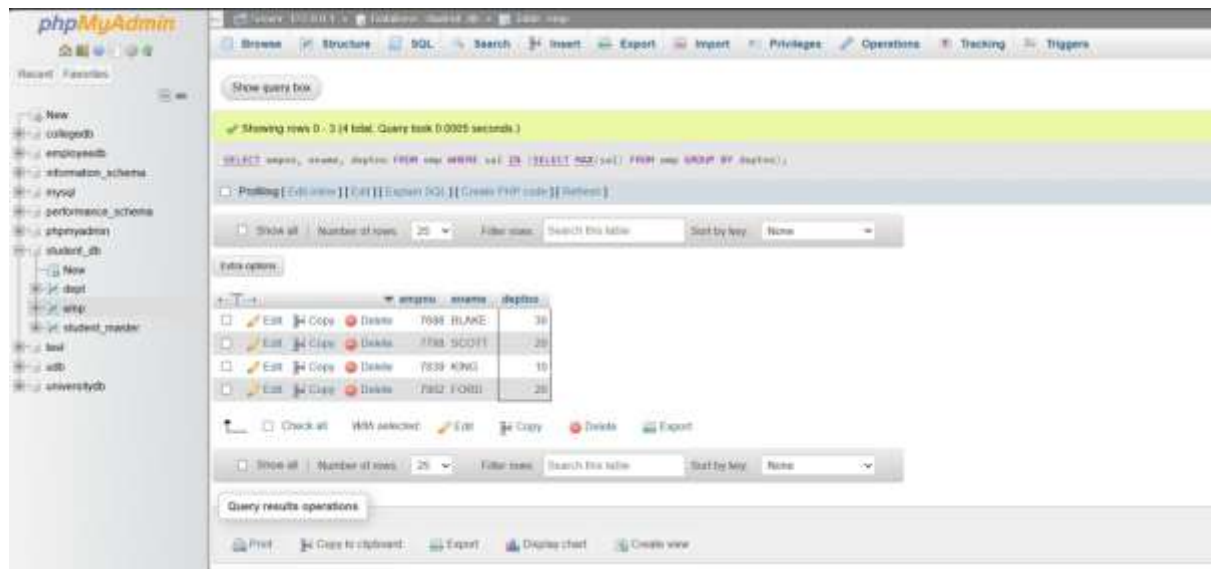


Problem 2: Find the employees who are all getting highest salary in each department.

SQL> select empno,ename,deptno from emp where sal in(select max(sal) from emp group by deptno); SELECT empno, ename, deptno FROM emp WHERE sal IN (SELECT MAX(sal) FROM emp GROUP BY deptno);

EMPNO	ENAME	DEPTNO
7698	BLAKE	30
7788	SCOTT	20
7902	FORD	20
7839	KING	10 4

rows selected.

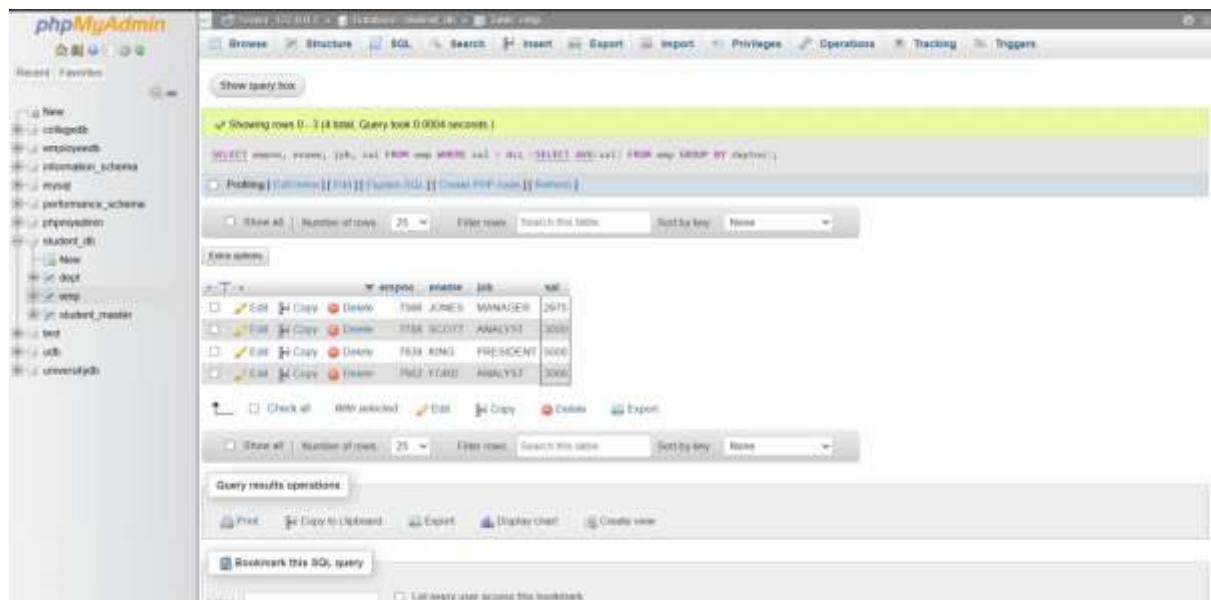


Problem 3: Find the employees who are all getting highest salary than the average salary of each department.

SQL> select empno,ename,job,sal from emp where sal>all(select avg(sal) from emp group by deptno);
SELECT empno, ename, job, sal FROM emp WHERE sal > ALL(SELECT AVG(sal) FROM emp GROUP BY deptno);

ENO	ENAME	JOB	SAL
7566	JONES	MANAGER	2975
7788	SCOTT	ANALYST	3000
7839	KING	PRESIDENT	5000
7902	FORD	ANALYST	3000

rows selected.

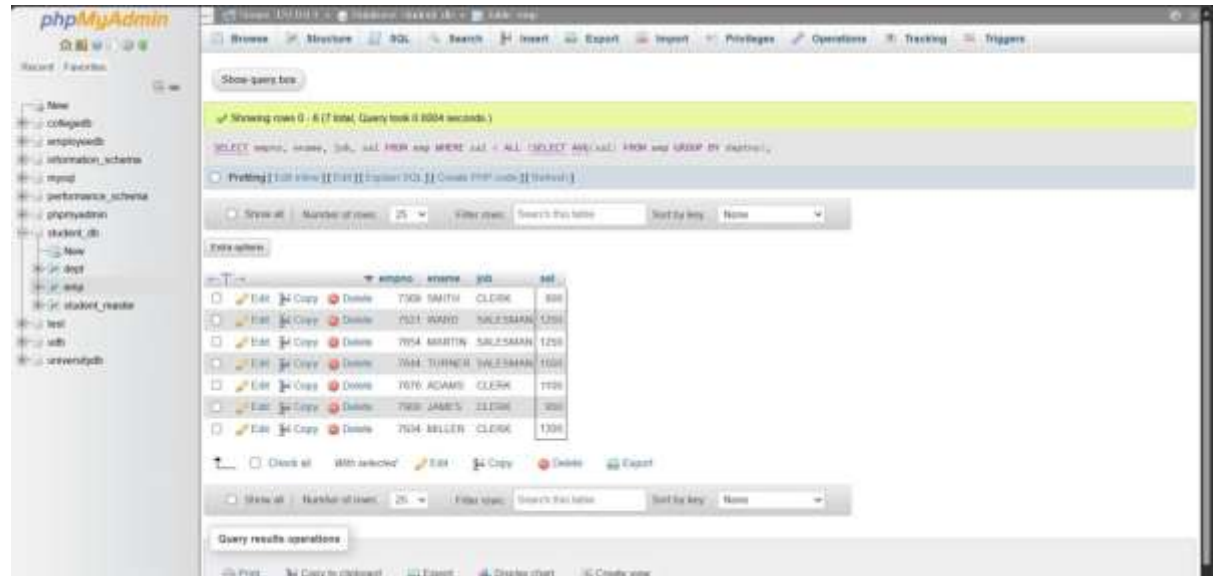


Problem 4: Find the employees who are all getting lowest salary than the average salary of each department

SQL> select empno,ename,job,sal from emp where sal<all(select avg(sal) from emp group by deptno);
SELECT empno, ename, job, sal FROM emp WHERE sal < ALL(SELECT AVG(sal) FROM emp GROUP BY deptno);

EMPNO	ENAME	JOB	SAL
7369	SMITH	CLERK	800
7521	WARD	SALESMAN	1250
7654	MARTIN	SALESMAN	1250
7844	TURNER	SALESMAN	1500
7876	ADAMS	CLERK	1100

7900 JAMES CLERK 950
7934 MILLER CLERK 1300 7
rows selected.

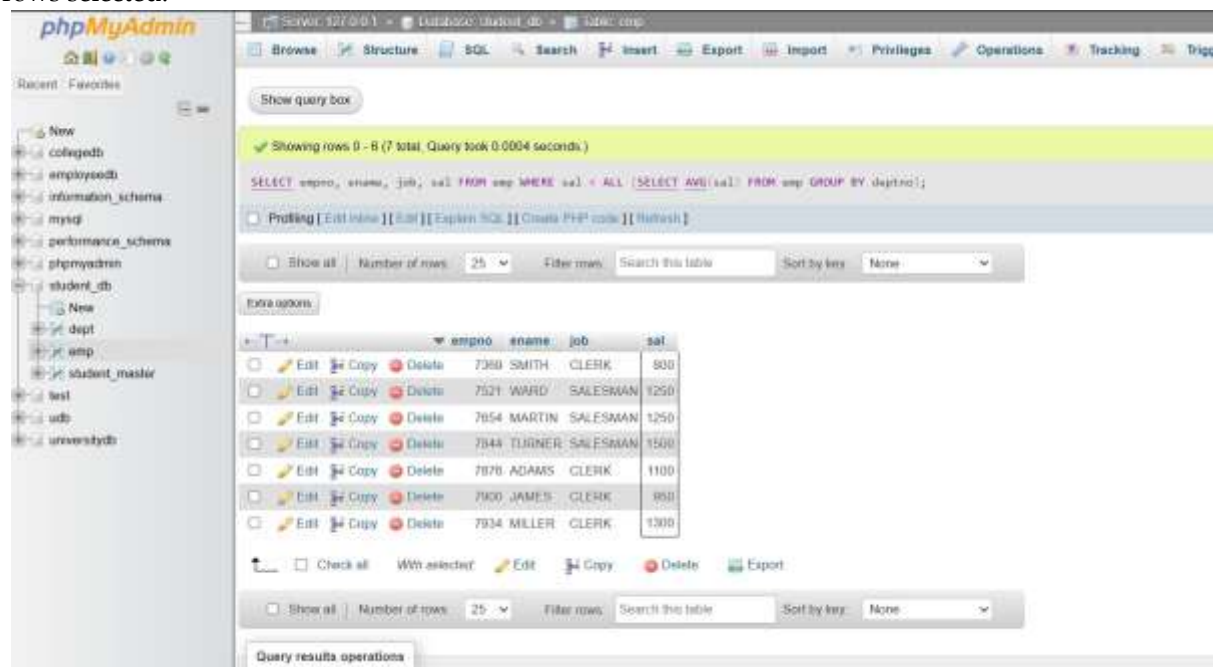


Problem 5: Find the employees who are all getting highest salary than the salary of employee working as clerk.

SQL> select ename,job,sal from emp where sal>any(select sal from emp where job='CLERK'); SELECT ename, job, sal FROM emp WHERE sal > ANY(SELECT sal FROM emp WHERE job = 'CLERK');

ENAME	JOB	SAL
ALLEN	SALESMAN	1600
WARD	SALESMAN	1250
JONES	MANAGER	2975
MARTIN	SALESMAN	1250
BLAKE	MANAGER	2850
MANAGER		2450
ANALYST		3000
KING	PRESIDENT	5000
TURNER	SALESMAN	1500
ADAMS	CLERK	1100
JAMES	CLERK	950
FORD	ANALYST	3000
MILLER	CLERK	1300
13		

rows selected.

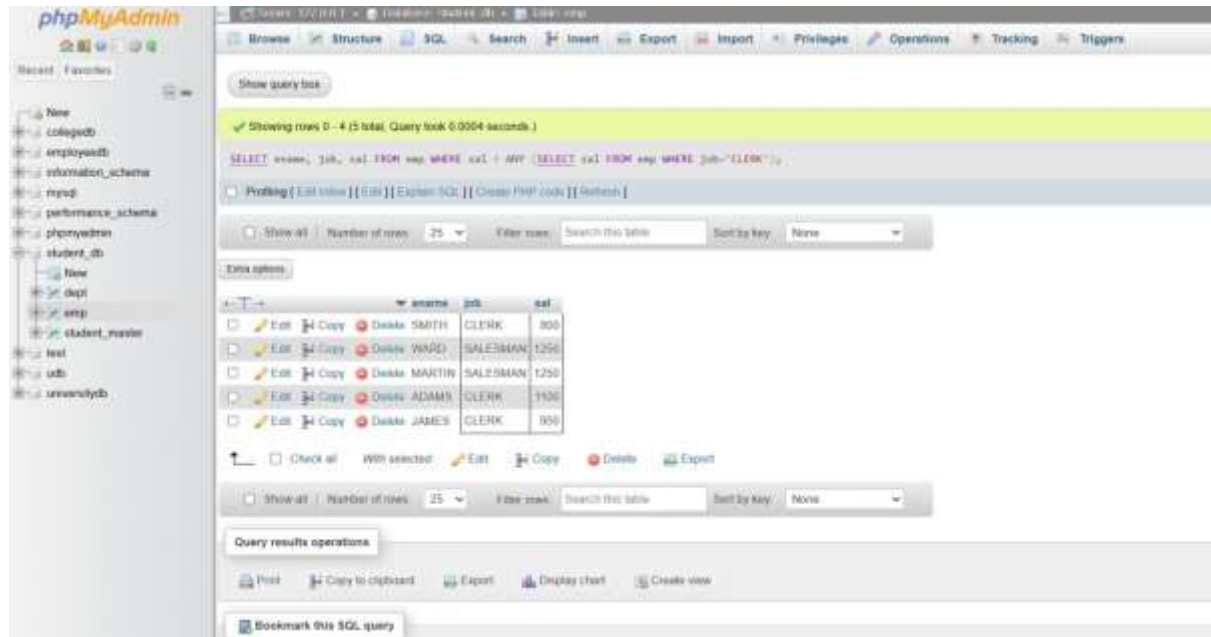


Problem 6: Find the employees who are all getting lowest salary than the salary of employee working as clerk.

SQL> select ename,job,sal from emp where sal<any(select sal from emp where job='CLERK');
SELECT ename, job, sal FROM emp WHERE sal < ANY(SELECT sal FROM emp WHERE job = 'CLERK');

ENAME	JOB	SAL
SMITH	CLERK	800
WARD	SALESMAN	1250
MARTIN	SALESMAN	1250
ADAMS	CLERK	1100
JAMES	CLERK	950

rows selected.

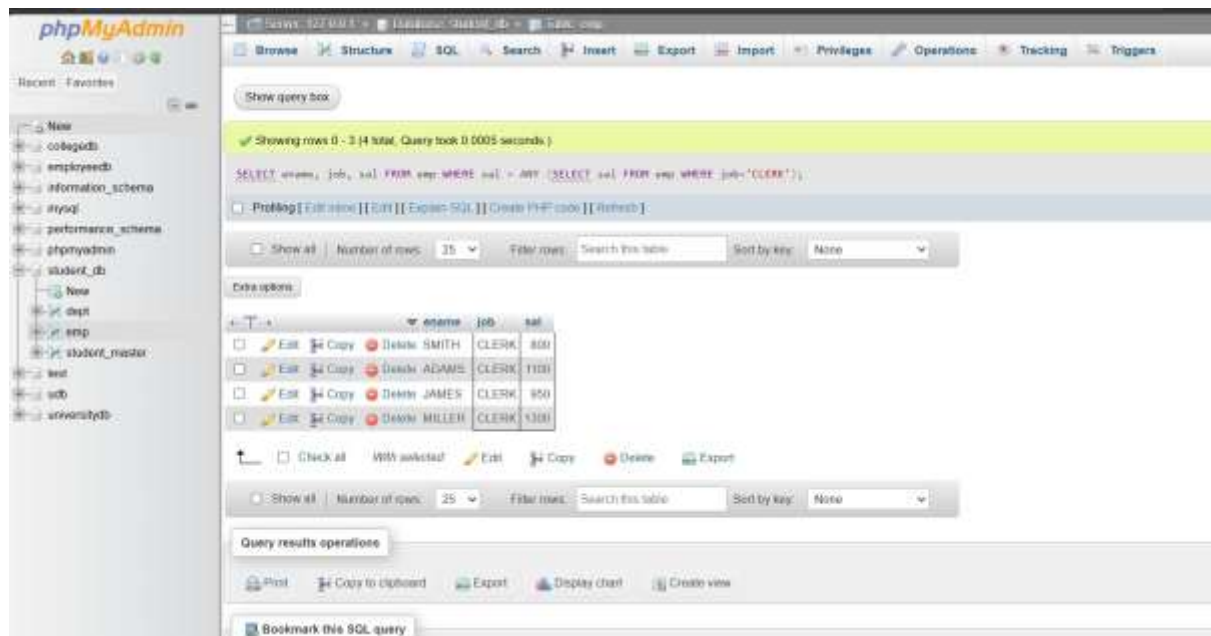


Problem 7: Find the employees who are all getting salary of employee working as clerk.

SQL> select ename,job,sal from emp where sal=any(select sal from emp where job='CLERK');
SELECT ename, job, sal FROM emp WHERE sal = ANY(SELECT sal FROM emp WHERE job = 'CLERK');
SELECT ename, job, sal FROM emp WHERE sal IN (SELECT sal FROM emp WHERE job = 'CLERK');

ENAME	JOB	SAL
SMITH	CLERK	800
JAMES	CLERK	950
ADAMS	CLERK	1100
MILLER	CLERK	1300

rows selected.



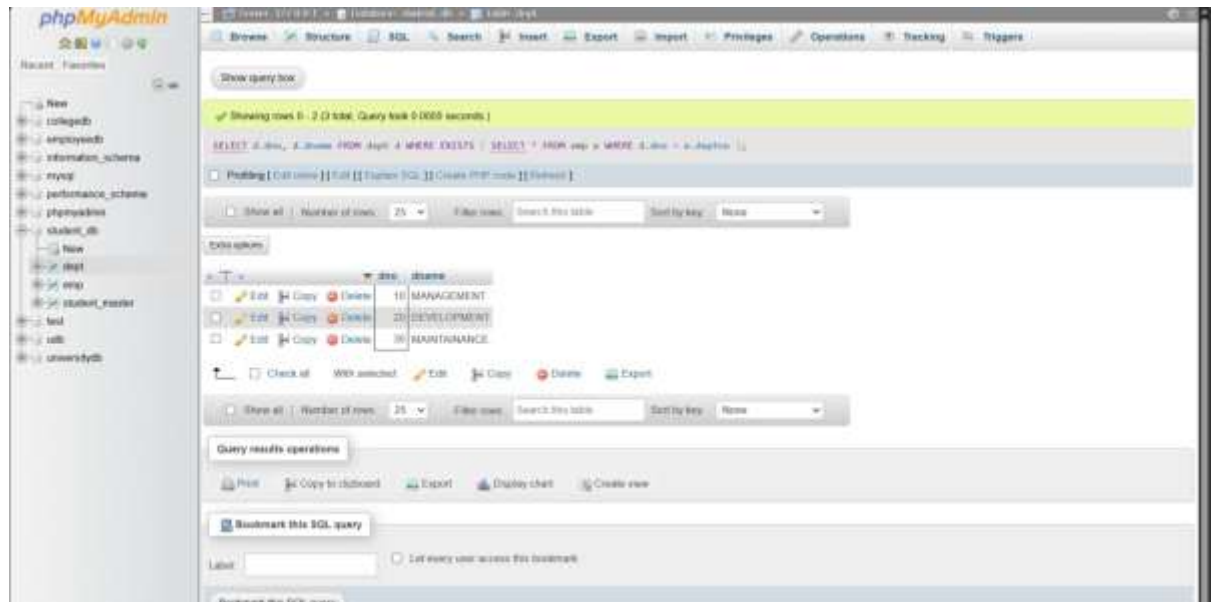
Problem 8: Find the list of departments working by the employees

SQL> select deptno,dname from dept d where exists(select * from emp e where d.deptno=e.deptno);
 SELECT deptno, dname FROM dept d WHERE EXISTS (SELECT * FROM emp e WHERE d.deptno = e.deptno);
 SELECT DISTINCT d.deptno, d.dname FROM dept d INNER JOIN emp e ON d.deptno = e.deptno;

DEPTNO DNAME

10 ACCOUNTING
 20 RESEARCH
 30 SALES 3

rows selected.

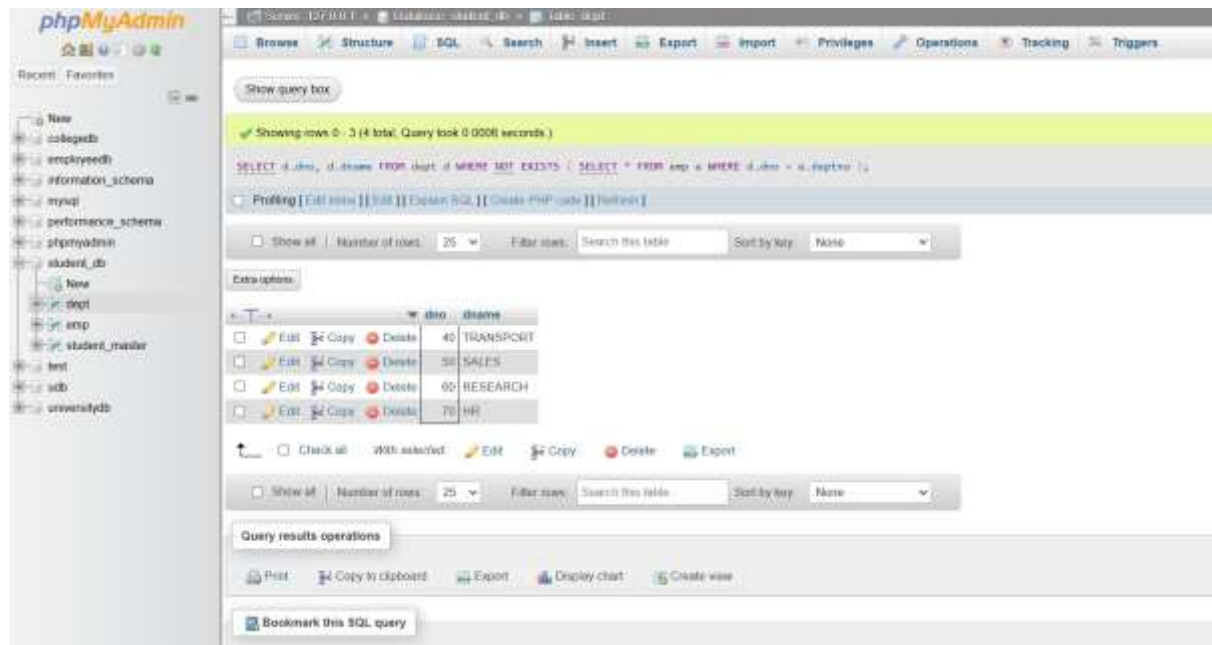


Problem 9: Find the list of departments in which no employee is working.

SQL> select deptno,dname from dept d where not exists(select * from emp e where d.deptno=e.deptno);
 SELECT deptno, dname FROM dept d WHERE NOT EXISTS (SELECT * FROM emp e WHERE d.deptno = e.deptno);
 SELECT d.deptno, d.dname FROM dept d LEFT JOIN emp e ON d.deptno = e.deptno WHERE e.deptno IS NULL;

DEPTNO DNAME

40 OPERATIONS

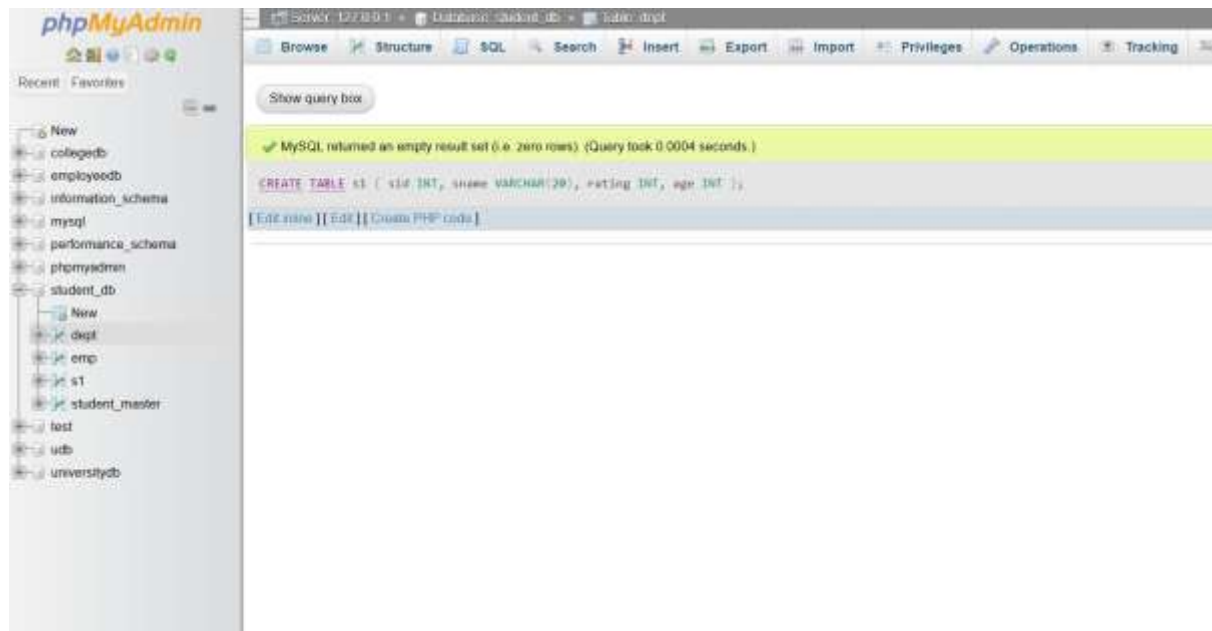


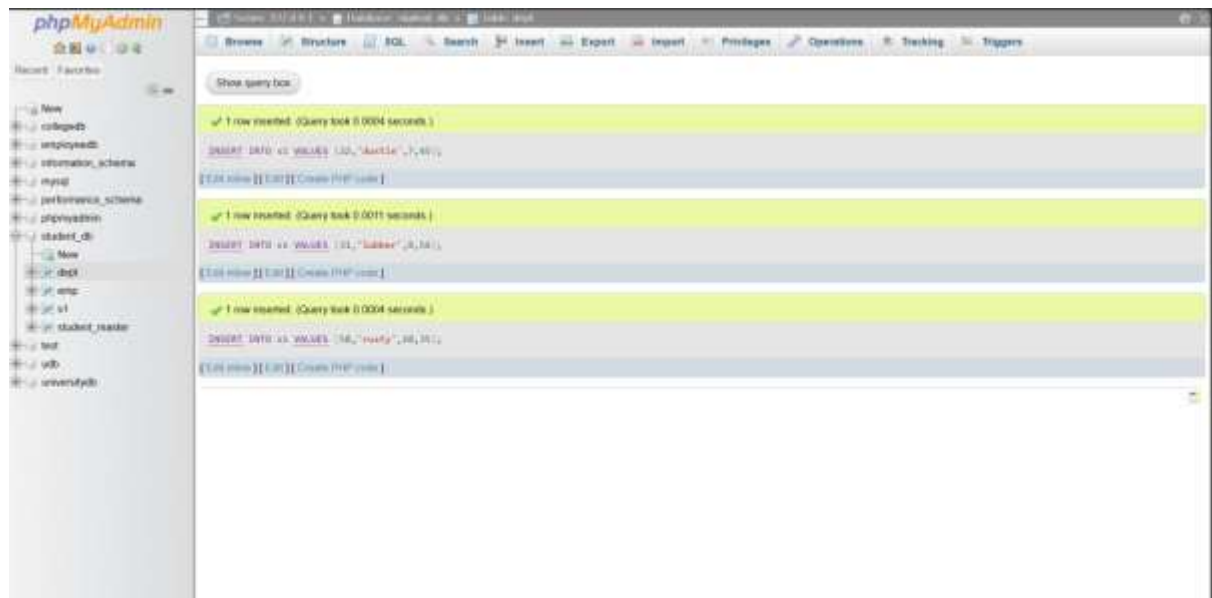
SELECT COMMANDS USING (UNION, INTERSECT, MINUS)

Need to create two tables for discussing these operations. The content of the tables become, SQL> select * from s1;

SID	SNAME	RATING	AGE
22	dustin	7	45
31	lubber	8	56
58	rusty	10	35

3 rows selected.

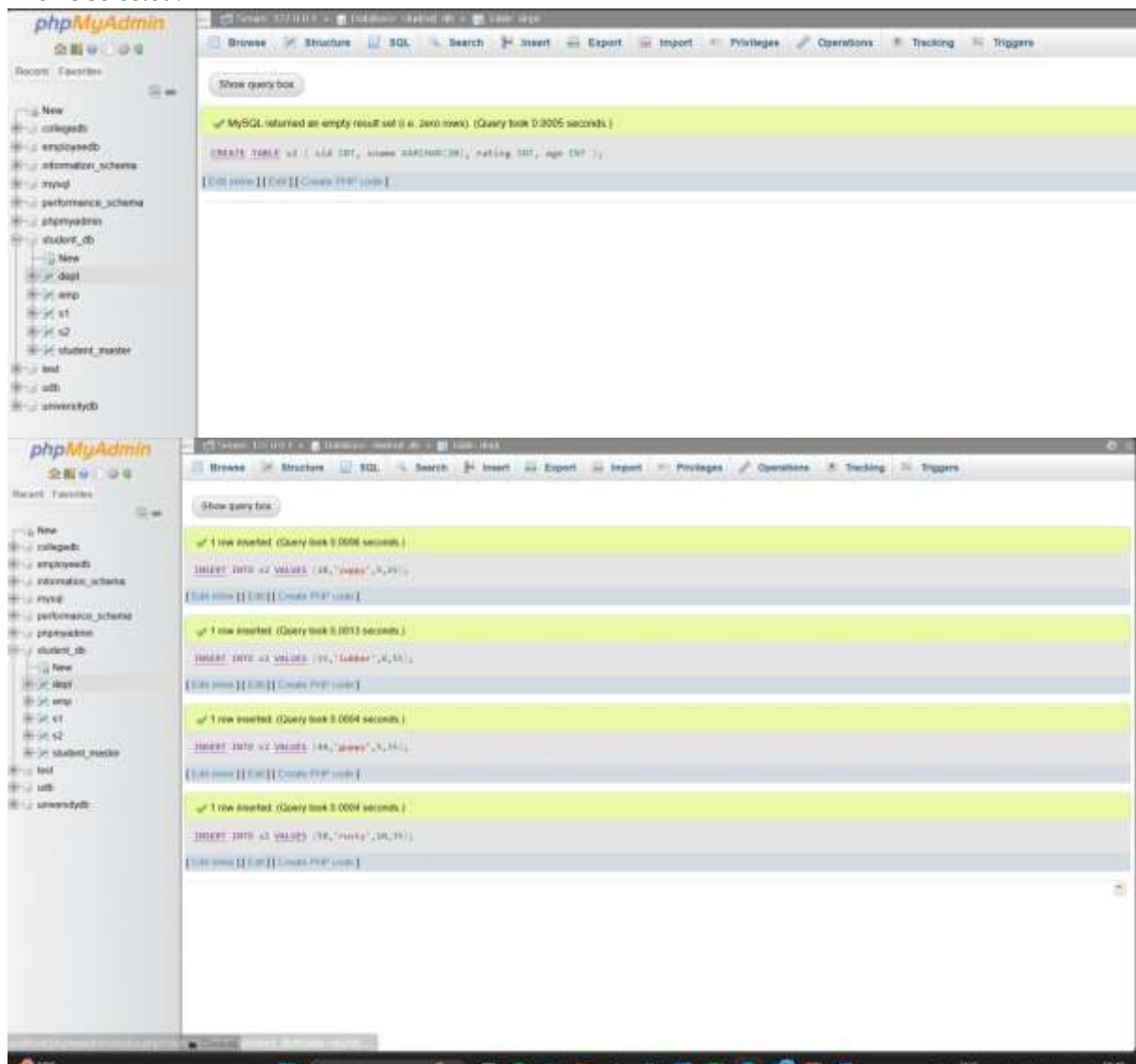




SQL> select * from s2;

SID	SNAME	RATING	AGE
28	yuppy	9	35
31	lubber	8	55
44	guppy	5	35
58	rusty	10	35

4 rows selected.



Showing rows 0 - 2 (3 total, Query took 0.0002 seconds)

SELECT * FROM s1;

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

s1d	sname	rating	age
22	dustin	7	45
31	lubber	8	58
58	rusty	10	35

☐ Show all | Number of rows: 25 | Filter rows:

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available.

New

- collegedb
- employeesdb
- information_schema
- mysql
- performance_schema
- phpmyadmin
- student_db
 - New
 - dept
 - emp
 - s1
 - s2
 - student_master
- test
- uob
- universitydb

Showing rows 0 - 3 (4 total, Query took 0.0004 seconds)

SELECT * FROM s2;

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows:

Extra options

s2d	sname	rating	age
28	yuppy	9	25
31	lubber	8	55
44	guppy	5	35
58	rusty	10	28

☐ Show all | Number of rows: 25 | Filter rows:

Query results operations

[Print](#) [Copy to clipboard](#) [Export](#) [Display chart](#) [Create view](#)

[Bookmark this SQL query](#)

Label: ☐ Let every user access this bookmark

[Bookmark this SQL query](#)

Problem 1: Find the List of students

SQL> select sname from s1 union select sname from s2;
 SELECT ENAME AS NameAndTitle FROM EMP UNION SELECT DNAME FROM DEPT;
 SELECT DEPTNO FROM EMP UNION ALL SELECT DEPTNO FROM DEPT;
 SNAME

 dustin
 guppy
 lubber rusty
 yuppy
 5 rows selected.



Problem 2: Find the list of students in both the set.

SQL> select sname from s1 intersect select sname from s2;

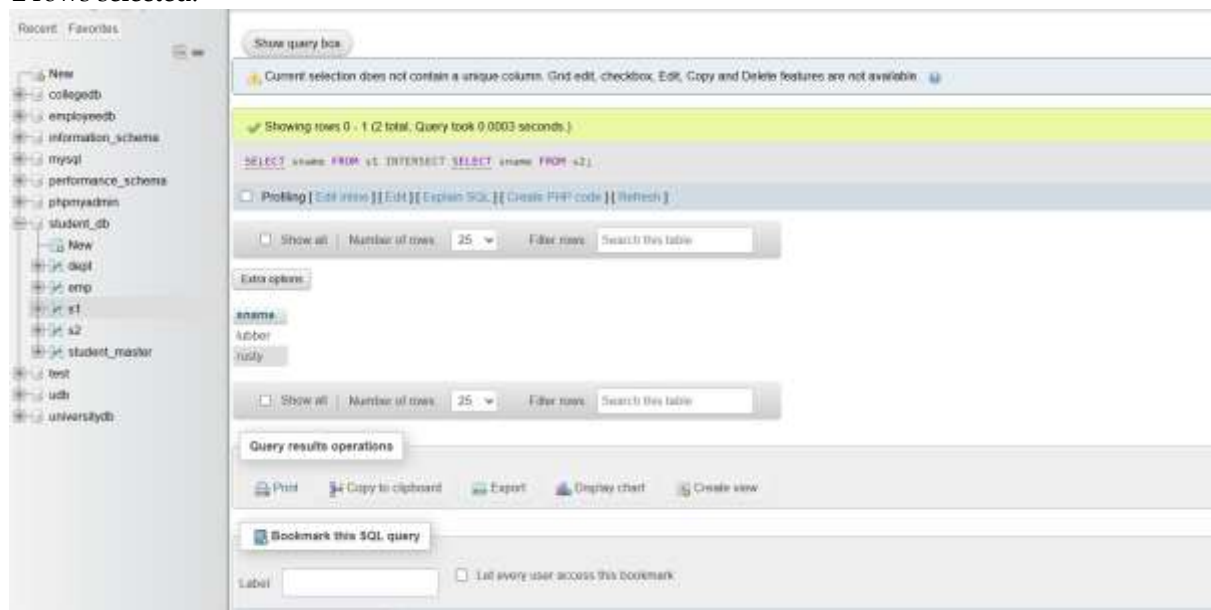
SELECT DISTINCT sal FROM EMP WHERE sal IN (SELECT sal FROM BONUS);

SNAME

lubber

rusty

2 rows selected.



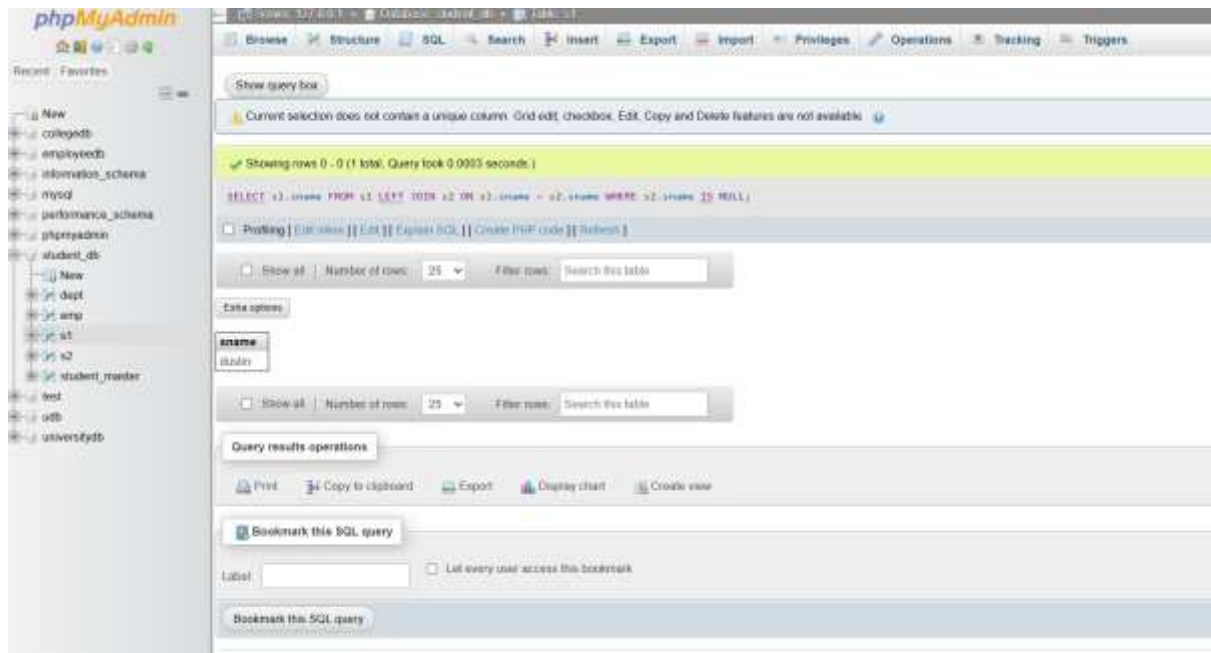
Problem 3: Find the list of students in set1 not in set2.

SQL> select sname from s1 minus select sname from s2;

SELECT d.deptno, d.dname FROM DEPT d LEFT JOIN EMP e ON d.deptno = e.deptno WHERE e.deptno IS NULL;
SNAME

dustin

1 row selected.



RESULT:

Thus the Program Creation of a Database and Writing SQL Queries to Retrieve Information from the Database is executed successfully. **Exp.No: 4**

BUILT IN FUNCTIONS

Date:

AIM:

To execute built in functions available in Oracle by using SQL.

PROCEDURE:

The following are the built in functions in Oracle. All of its operations were discussed in this experiment.

- CONVERSION FUNCTION
- STRING FUNCTION
- DATE FUNCTION ➤
- NUMBER FUNCTION

CONVERSION FUNCTIONS

SQL> SELECT TO_CHAR (SYSDATE,'DAY') FROM DUAL;

MONDAY

[SQL Worksheet]*      Aa 

```
1 SELECT TO_CHAR(SYSDATE, 'DAY') FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	TO_CHAR(SYSDATE
1	SATURDAY

SQL> SELECT TO_CHAR(SYSDATE, 'MONTH') FROM DUAL;

JANUARY

[SQL Worksheet]*      Aa 

```
1 SELECT TO_CHAR(SYSDATE, 'MONTH') FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	TO_CHAR(SYSDATE
1	MARCH

SQL> SELECT TO_CHAR(SYSDATE,'YEAR')FROM DUAL;

TWO THOUSAND NINE

[SQL Worksheet]*

```
1 SELECT TO_CHAR(SYSDATE,'YEAR') FROM DUAL;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.002 seconds

	Column "TO_CHAR(SYSDATE,'YEAR')\" row 1
1	TWENTY-TWENTY-SIX

STRING FUNCTIONS

SQL>SELECT CONCAT ('ORACLE','CORPORATION') FROM DUAL;

ORACLECORPORATION

[SQL Worksheet]*

```
1 SELECT CONCAT('ORACLE','CORPORATION') FROM DUAL;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.002 seconds

	Column "CONCAT('ORACLE','CORPORATION')\" row 1
1	ORACLECORPORATION

Close

SQL>SELECT LPAD ('ORACLE', 15,'*') FROM DUAL;

*****ORACLE

The screenshot shows an SQL Worksheet interface. At the top, there is a toolbar with icons for running queries, saving, and other functions. Below the toolbar, the SQL query is entered: `1: SELECT LPAD('ORACLE',15,'*') FROM DUAL;`. The query is executed, and the results are displayed in a table. The table has one column titled **Column "LPAD('ORACLE',15,'*')" row 1** and one row containing the text `*****ORACLE`. The interface also shows tabs for "Query result", "Script output", "DBMS output", "Explain Plan", and "SQL history". A "Download" button and "Execution time: 0.003 seconds" are visible. A "Close" button is at the bottom right of the results area.

SQL>SELECT RPAD ('ORACLE', 15,'*') FROM DUAL;

ORACLE*****

[SQL Worksheet]*      Aa 

```
1 SELECT RPAD('ORACLE',15,'*') FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.002 seconds

1	Column "RPAD('ORACLE',15,'*')" row 1
	ORACLE*****

SQL>SELECT LTRIM ('SSMITHSS','S') FROM DUAL;

MITHSS

[SQL Worksheet]*      Aa   

```
1 SELECT LTRIM('SSMITHSS','S') FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.001 seconds

1	Column "LTRIM('SSMITHSS','S')" row 1
	MITHSS

Close

SQL>SELECT RTRIM ('SSMITHSS','S') FROM DUAL;

SSMITH

[SQL Worksheet]*      Aa 

```
1 SELECT RTRIM('SSMITHSS','S') FROM DUAL;  
2
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.003 seconds

	Column "RTRIM('SSMITHSS','S')"
1	SSMITH

Close

SQL>SELECT LOWER ('DBMS') FROM DUAL;

dbms

[SQL Worksheet]*      Aa 

```
1 SELECT LOWER ('DBMS') FROM DUAL;  
2  
3
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.001 seconds

	Column "LOWER('DBMS')"
1	dbms

SQL>SELECT UPPER ('dbms') FROM DUAL;
DBMS

The screenshot shows a web-based SQL editor interface. At the top, there's a toolbar with icons for running, saving, and other functions. Below the toolbar, the SQL code 'SELECT UPPER('dbms') FROM DUAL;' is entered in a text area. The interface then displays the 'Query result' tab, which shows the execution time as 0.003 seconds. Below this, a table with one row and one column is shown. The column header is 'Column "UPPER('DBMS')" row 1' and the value in the row is 'DBMS'.

Column "UPPER('DBMS')" row 1
DBMS

SQL>SELECT INITCAP ('ORACLE','CORPORATION') FROM DUAL;
Oracle Corporation

The screenshot shows the same web-based SQL editor interface. The SQL code 'SELECT INITCAP('ORACLE','CORPORATION') FROM DUAL;' is entered. The 'Query result' tab shows an execution time of 0.002 seconds. Below this, a table with one row and one column is shown. The column header is 'Column "INITCAP('ORACLECORPORATION')" row 1' and the value in the row is 'Oracle Corporation'.

Column "INITCAP('ORACLECORPORATION')" row 1
Oracle Corporation

Close

SQL>SELECT LENGTH ('DATABASE') FROM DUAL;

8

The screenshot shows a web-based SQL editor interface. At the top, there's a toolbar with icons for running, saving, and other functions. Below the toolbar, the SQL query `SELECT LENGTH('DATABASE') FROM DUAL;` is entered on two lines. The interface then shows the 'Query result' tab selected. Below this, there's a 'Download' button and the text 'Execution time: 0.001 seconds'. The result is displayed in a table with one row and one column. The column header is `LENGTH('DATABASE')` and the value in the row is `8`.

	LENGTH('DATABASE')
1	8

SQL>SELECT SUBSTR ('ABCDEFGHIJ'3, 4) FROM DUAL;

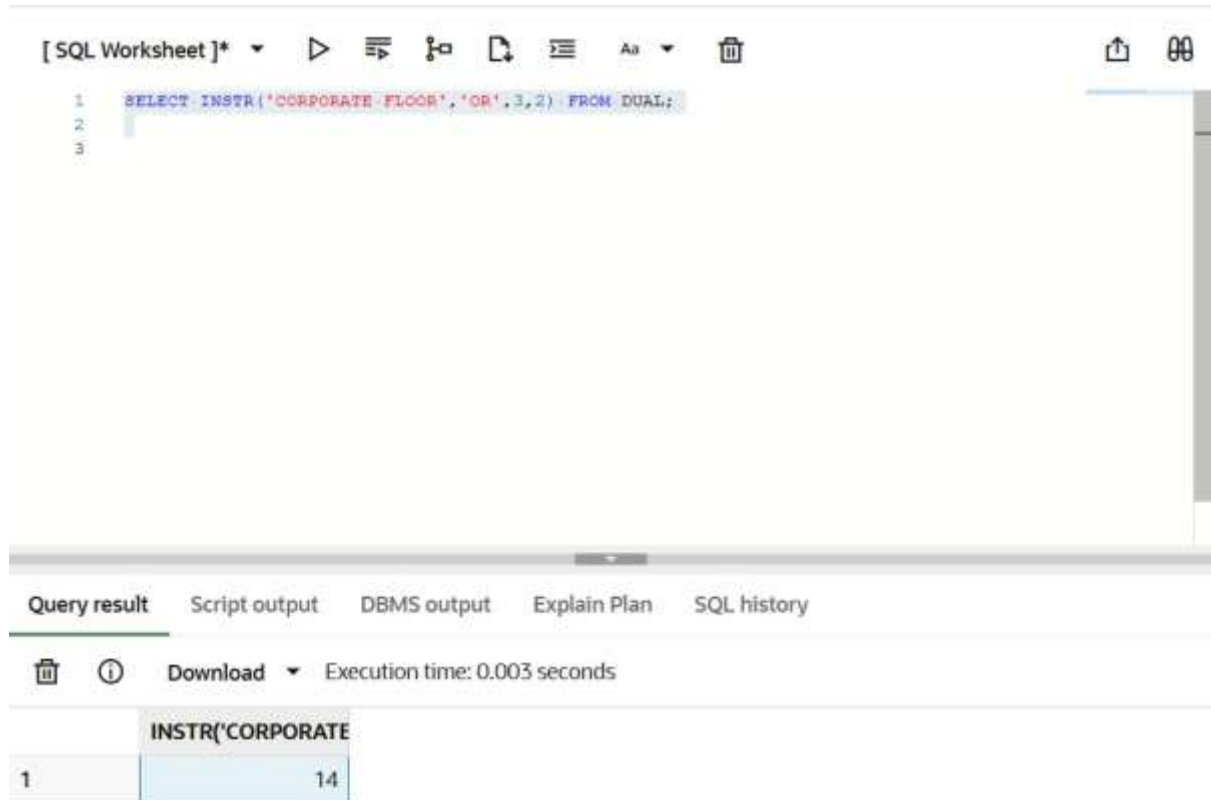
CDEF

The screenshot shows the same SQL editor interface. The query `SELECT SUBSTR('ABCDEFGHIJ',3,4) FROM DUAL;` is entered. The 'Query result' tab is selected, showing 'Execution time: 0.003 seconds'. The result is displayed in a table. The first row has a column header `Column "SUBSTR('ABCDEFGHIJ',3,4)" row 1` and a value `CDEF`.

	Column "SUBSTR('ABCDEFGHIJ',3,4)" row 1
1	CDEF

SQL>SELECT INSTR ('CORPORATE FLOOR','OR', 3, 2) FROM DUAL;

14



The screenshot shows an SQL Worksheet interface. At the top, there is a toolbar with icons for running, saving, and other functions. Below the toolbar, the SQL query is entered in a text area: `SELECT INSTR('CORPORATE FLOOR','OR', 3, 2) FROM DUAL;`. The query is highlighted in blue. Below the query area, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is selected. Below the tabs, there is a section for the query result. It shows a table with one row and one column. The column is labeled 'INSTR('CORPORATE' and the row is labeled '1'. The value in the cell is '14'. The execution time is shown as '0.003 seconds'.

	INSTR('CORPORATE
1	14

SQL>SELECT TRANSLATE ('JACK','J','B') FROM DUAL;
BACK

[SQL Worksheet]*      Aa   

```
1 SELECT TRANSLATE('JACK','J','B') FROM DUAL;  
2  
3
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.001 seconds

1	Column "TRANSLATE('JACK','J','B')" row 1
	BACK

SQL>SELECT REPLACE ('JACK AND JUE','J','BL') FROM DUAL;
BLACK AND BLUE

[SQL Worksheet]*      Aa   

```
1 SELECT REPLACE('JACK AND JUE','J','BL') FROM DUAL;  
2  
3
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.003 seconds

1	Column "REPLACE('JACKANDJUE','J','BL')" row 1
	BLACK AND BLUE

DATE FUNCTIONS

SQL>SELECT SYSDATE FROM DUAL;

29-DEC-08

The screenshot shows an SQL Worksheet interface. At the top, there is a toolbar with icons for running queries, saving, and other functions. Below the toolbar, the SQL query `SELECT SYSDATE FROM DUAL;` is entered in the editor. The interface has tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Query result' tab is active, showing a single row of results. The result is displayed as a date and time: '15:49:52.000000000'. A calendar widget is also visible, showing 'March 2026'.

[SQL Worksheet]* Aa

```
1 SELECT SYSDATE FROM DUAL;
2
3
4
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download ▾ Execution time: 0.001 seconds

1 **Column "SYSDATE" row 1**

Date:	Time:
< March 2026 > S M T W T F S 1 2 3 4 5 6 7	15:49:52.000000000

SQL>SELECT NEXT_DAY (SYSDATE,'WED') FROM DUAL;

05-JAN-09

[SQL Worksheet]*

```
1 SELECT NEXT_DAY(SYSDATE,'WEDNESDAY') FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.001 seconds

1 **Column "NEXT_DAY(SYSDATE,'WEDNESDAY')" row 1**

Date:	Time:
< March 2026 > S M T W T F S	16:00:52.000000000

Close

SQL>SELECT ADD_MONTHS (SYSDATE, 2)FROM DUAL;

28-FEB-09

[SQL Worksheet]*

```
1 SELECT ADD_MONTHS(SYSDATE,2) FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.001 seconds

1 **Column "ADD_MONTHS(SYSDATE,2)" row 1**

Date:	Time:
< May 2026 > S M T W T F S	16:01:30.000000000

SQL>SELECT LAST_DAY (SYSDATE) FROM DUAL;

31-DEC-08

[SQL Worksheet]*

```

1 SELECT MONTHS_BETWEEN(SYSDATE, DATE '2009-02-12') FROM DUAL;
2

```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.001 seconds

	MONTHS_BETWEEN 02-12')
1	05.1183516278375

SQL>SELECT MONTHS_BETWEEN (SYSDATE,'12-feb-09') FROM DUAL;

2

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.003 seconds

1 **Column "LEAST(DATE'2007-01-10',DATE'2007-10-12')"** row 1

Date:	Time:
< January 2007 > S M T W T F S	00:00:00.000000000

Close

SQL>SELECT LEAST ('10-JAN-07','12-OCT-07') FROM DUAL;

10-JAN-07



12-OCT-07

[SQL Worksheet]*      Aa   

```
1 SELECT GREATEST (DATE '2007-01-10', DATE '2007-10-12') FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.001 seconds

1 **Column "GREATEST (DATE '2007-01-10', DATE '2007-10-12') " row 1**

Date:	Time:
< October 2007 > S M T W T F S	00:00:00.000000000

Close

NUMBER FUNCTIONS

SQL> SELECT ROUND (12.36), ROUND (14.63) FROM DUAL;

ROUND (12.36)	ROUND (14.63)
-----	-----
12	15

[SQL Worksheet]*      Aa  

```
1 SELECT ROUND(12.36), ROUND(14.63) FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	ROUND(12.36)	ROUND(14.63)
1	12	15

SQL> SELECT FLOOR (12.87), FLOOR (11.23) FROM DUAL;

FLOOR (12.87) FLOOR (11.23)

12 11

[SQL Worksheet]*      Aa  

```
1 SELECT FLOOR(12.87), FLOOR(11.23) FROM DUAL;
2
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	FLOOR(12.87)	FLOOR(11.23)
1	12	11

SQL> SELECT CEIL (16.23), CEIL (12.78) FROM DUAL;

17 13



56

[SQL Worksheet]*      Aa 

```
1 SELECT TRUNC(56.63) FROM DUAL;  
2  
3  
4
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	TRUNC(56.63)
1	56

SQL> SELECT MOD (11, 4) FROM DUAL;

MOD (11, 4)

3

[SQL Worksheet]*      Aa  

```

1  SELECT MOD(11,4) FROM DUAL;
2
3
4

```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.002 seconds

	MOD(11,4)
1	3

SQL> SELECT POWER (2, 3) FROM DUAL;

POWER (2,3)

8

[SQL Worksheet]*      Aa  

```

1  SELECT POWER(2,3) FROM DUAL;

```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	POWER(2,3)
1	8

SQL> SELECT SIGN (0), SIGN (34), SIGN (-56) FROM DUAL;

SIGN (0) SIGN (34) SIGN (-56)

0

1

-1

[SQL Worksheet]*      Aa 

```
1 SELECT SIGN(0), SIGN(34), SIGN(-56) FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.002 seconds

	SIGN(0)	SIGN(34)	SIGN(-56)
1	0	1	-1

SQL> SELECT ABS (12), ABS (-89) FROM DUAL;

ABS (12) ABS (-89)

12 89

[SQL Worksheet]*      Aa 

```
1 SELECT ABS(12), ABS(-89) FROM DUAL;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	ABS(12)	ABS(-89)
1	12	89

SQL> SELECT SQRT (25) FROM DUAL;

SQRT (25)

[SQL Worksheet]*      Aa 

1 SELECT SQRT(25) FROM DUAL;

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.003 seconds

	SQRT(25)
1	5

RESULT:

Thus the Program Built in functions is executed successfully.

Date:**AIM:**

To implement and execute view, synonyms, sequence, indexes in Oracle using Structured Query Language commands

SYNTAX:

The following are the SQL Objects and their syntax.

➤ VIEW:

```
CREATE VIEW <VIEWNAME> AS SELECT * FROM <TABLENAME> WHERE
<CONDITION>;
```

➤ SYNONYMS

```
CREATE SYNONYM <SYNONYMS NAME> FOR <TABLENAME>;
```

➤ SEQUENCE

```
CREATE SEQUENCE<SEQUENCENAME> START WITH <VALUE> INCREMENT
BY <VALUE>;
```

➤ INDEXES

```
CREATE INDEX < INDEX_NAME> ON TABLENAME (COLUMNNAME
[, COLUMNNAME...]);
```

VIEW:

We have a source table student from that some of the unrestricted fields are taken and form a view called studentdetail. Now the view (studentdetail) is a temporary table, any changes happened in this view will actually updates the source relation student.

```
SQL> create table student(regno number(6),name char(5),mark1 number(2),mark2
number(2),total number(3));
```

Table created.

After insertion the content of the table become,

```
SQL> select * from student130;
```

REGNO	NAME	MARK1	MARK2	TOTAL
130	ajay	90	90	180
126	aldo	95	96	191
76	guru	90	95	185

[SQL Worksheet]*      As   

```
1 SELECT * FROM student;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.091 seconds

	REGNO	NAME	MARK1	MARK2	TOTAL
1	130	AJAY	90	90	180
2	126	ALDO	95	96	191
3	76	GURU	90	95	185

SQL> create view studentdetail as select regno,name,mark1,mark2 from student;
View created.

By inserting values into view,

SQL> insert into studentdetail values (®no,'&name',&mark1,&mark2);

Enter value for regno: 547

Enter value for name: shgh

Enter value for mark1: 80

Enter value for mark2: 80

Query result Script output DBMS output Explain Plan SQL history

```
SQL> INSERT INTO studentdetail VALUES(547,'SHAH',80,80)
```

1 row inserted.
Elapsed: 00:00:00.003

```
SQL> INSERT INTO studentdetail VALUES(67,'JEGAN',70,80)
```

1 row inserted.
Elapsed: 00:00:00.002

[SQL Worksheet]*  

```
1 SELECT * FROM studentdetail;
```

Query result Script output DBMS output Explain Plan SQL history

  Download Execution time: 0.007 seconds

	REGNO	NAME	MARK1	MARK2
1	130	AJAY	90	90
2	126	ALDO	95	96
3	76	GURU	90	95
4	547	SHAH	80	80
5	67	JEGAN	70	80

old 1: insert into studentdetail130 values(®no,'&name',&mark1,&mark2)

new 1: insert into studentdetail130 values(547,'shah',80,80) 1 row created.

SQL> /

Enter value for regno: 67

Enter value for name: jdgh

Enter value for mark1: 70

Enter value for mark2: 80

old 1: insert into studentdetail130 values(®no,'&name',&mark1,&mark2) new

1: insert into studentdetail130 values(67,'jegan',70,80)

1 row created.

SQL> select * from studentdetail;

REGNO	NAME	MARK1	MARK2
-------	------	-------	-------

130	ajay	90	90
-----	------	----	----

126	aldo	95	96
-----	------	----	----

76	guru	90	95
----	------	----	----

547	shah	80	80
-----	------	----	----

67	jegan	70	80
----	-------	----	----

SQL> select * from student;

REGNO NAME		MARK1	MARK2	TOTAL	
-----	----	-----	-----	-----	
130	ajay	90	90	180	126
aldo	95	96	191		
76	guru	90	95	185	
547	shah	80	80		
67	jegan	70	80		

[SQL Worksheet]*
▶
▶
≡
🔗
📄
≡
Aa
▼
🗑️
📶
🔍

1
SELECT * FROM student;

Query result
Script output
DBMS output
Explain Plan
SQL history

🗑️
📘
Download
▼
Execution time: 0.002 seconds

	REGNO	NAME	MARK1	MARK2	TOTAL
1	130	AJAY	90	90	180
2	126	ALDO	95	96	191
3	76	GURU	90	95	185
4	547	SHAH	80	80	(null)
5	67	JEGAN	70	80	(null)

SYNONYMS

```
CREATE TABLE product (product_name VARCHAR2(25) PRIMARY KEY,  
product_price NUMBER(4,2), quantity_on_hand NUMBER(5,0), last_stock_date  
DATE);
```

Table created.

The screenshot displays an SQL IDE interface. At the top, a toolbar contains icons for file operations, execution, and formatting. Below the toolbar, a code editor shows the SQL statement for creating the 'product' table. The statement is as follows:

```
1 CREATE TABLE product (  
2   product_name VARCHAR2(25) PRIMARY KEY,  
3   product_price NUMBER(4,2),  
4   quantity_on_hand NUMBER(5),  
5   last_stock_date DATE  
6 );
```

Below the code editor, a tabbed interface shows the 'Script output' tab selected. This tab contains the following information:

- Icons for deleting and downloading the output.
- A message: '1 row inserted.'
- Elapsed time: 00:00:00.002
- The executed SQL command: `SQL> CREATE TABLE product (
product_name VARCHAR2(25) PRIMARY KEY,
product_price NUMBER(4,2),
quantity_on_hand NUMBER(5),...`
- A link: [Show more...](#)
- A confirmation message: 'Table PRODUCT created.'
- Elapsed time: 00:00:00.020

[SQL Worksheet]*
▶
≡
🔗
📄
≡
Aa
🗑️

1 INSERT INTO product VALUES ('Product1',99,1,DATE '2003-01-15');
2 INSERT INTO product VALUES ('Product2',75,1000,DATE '2002-01-15');
3 INSERT INTO product VALUES ('Product3',50,100,DATE '2003-01-15');
4 INSERT INTO product VALUES ('Product4',25,10000,DATE '2003-01-14');
5 INSERT INTO product VALUES ('Product5',9.95,1234,DATE '2004-01-15');
6 INSERT INTO product VALUES ('Product6',45,1,DATE '2008-12-31');

Query result
Script output
DBMS output
Explain Plan
SQL history

🗑️
📄

SQL> INSERT INTO product VALUES ('Product5',9.95,1234,DATE '2004-01-15')

1 row inserted.
Elapsed: 00:00:00.002

SQL> INSERT INTO product VALUES ('Product6',45,1,DATE '2008-12-31')

1 row inserted.
Elapsed: 00:00:00.002

AFTER INSERTING THE RECORDS TO PRODUCT TABLE

SQL> SELECT * FROM product;

PRODUCT_NAME	PRODUCT_PRICE	QUANTITY_ON_HAND	LAST_STOC
Product 1	99	1	15-JAN-03
Product 2	75	1000	15-JAN-02
Product 3	50	100	15-JAN-03
Product 4	25	10000	14-JAN-03
Product 5	9.95	1234	15-JAN-04
Product 6	45	1	31-DEC-08

6 rows selected.

```
1 SELECT * FROM product;
```

Query result

Script output

DBMS output

Explain Plan

SQL history

  Download ▾ Execution time: 0.012 seconds

	PRODUCT_NAME	PRODUCT_PRICE	QUANTITY_ON_HAI	LAST_STOCK_DATE
1	Product1	99	1	1/15/2003, 12:00:00
2	Product2	75	1000	1/15/2002, 12:00:00
3	Product3	50	100	1/15/2003, 12:00:00
4	Product4	25	10000	1/14/2003, 12:00:00
5	Product5	9.95	1234	1/15/2004, 12:00:00
6	Product6	45	1	12/31/2008, 12:00:00

```
SQL> SELECT * FROM prod;
      SELECT * FROM prod
      *
ERROR at line 1:
ORA-00942: table or view does not exist
```

```
1 CREATE SYNONYM prod FOR product;
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> INSERT INTO product VALUES ('Product6',45,1,DATE '2008-12-31')
```

1 row inserted.

Elapsed: 00:00:00.002

```
SQL> CREATE SYNONYM prod FOR product
```

Synonym PROD created.

Elapsed: 00:00:00.011

[Terms and Conditions](#) | [Your Privacy Rights](#) | [Delete Your FreeSQL Account](#) | [Cookie Preferences](#)

SQL> **CREATE SYNONYM** prod FOR product;

Synonym created.

SQL> **SELECT * FROM** prod;

PRODUCT_NAME TOC	PRODUCT_PRICE	QUANTITY_ON_HAND	LAST_S
-----Product 1	99	1	15-
JAN-03			
Product 2	75	1000	15-JAN-
02			
Product 3	50	100	15-JAN-
03			
Product 4	25	10000	14-JAN-

[SQL Worksheet]* ▼



Aa ▼



1 DROP SYNONYM prod;

Query result

Script output

DBMS output

Explain Plan

SQL history



SQL> CREATE SYNONYM prod FOR product

Synonym PROD created.

Elapsed: 00:00:00.011

SQL> DROP SYNONYM prod

Synonym PROD dropped.

Elapsed: 00:00:00.015

SQL> drop table product;

Table dropped.

[SQL Worksheet]*
▶
≡
🔑
📄
≡
Aa
▼
🗑️

1 DROP TABLE product;

Query result
Script output
DBMS output
Explain Plan
SQL history

🗑️
📄

SQL> DROP SYNONYM prod

Synonym PROD dropped.
Elapsed: 00:00:00.015

SQL> DROP TABLE product

Table PRODUCT dropped.
Elapsed: 00:00:00.038

SEQUENCE

Create a sequence and design the student table with the given attributes.

```
SQL> create table student(student_id number, name varchar2(10),result varchar2(10));
```

```
SQL> desc student;
```

Name	Null?	Type
-----	-----	-----
STUDENT_ID		NUMBER
NAME		VARCHAR2(10)
RESULT		VARCHAR2(10)

Sequence Creation

[SQL Worksheet]*



Aa ▼



```
1 CREATE TABLE student(  
2 student_id NUMBER,  
3 name VARCHAR2(10),  
4 result VARCHAR2(10)  
5 );
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Table STUDENT dropped.

Elapsed: 00:00:00.023

```
SQL> CREATE TABLE student(  
      student_id NUMBER,  
      name VARCHAR2(10),  
      result VARCHAR2(10)...
```

[Show more...](#)

Table STUDENT created.

Elapsed: 00:00:00.012

[SQL Worksheet]*

Aa

1DESC student;

Query resultScript outputDBMS outputExplain PlanSQL history

SQL> DESC student

Name	Null?	Type
STUDENT_ID		NUMBER
NAME		VARCHAR2(10)
RESULT		VARCHAR2(10)

SQL> create sequence student_seq start with 100 increment by 1;

Sequence created.

[SQL Worksheet]*

```
1 CREATE SEQUENCE student_seq
2 START WITH 100
3 INCREMENT BY 1;
```

Query result **Script output** DBMS output Explain Plan SQL history

🗑️ ⬇️

Name	Null?	Type
STUDENT_ID		NUMBER
NAME		VARCHAR2(10)
RESULT		VARCHAR2(10)

SQL> CREATE SEQUENCE student_seq
START WITH 100
INCREMENT BY 1

Sequence STUDENT_SEQ created.

Elapsed: 00:00:00.006

SQL> insert into student values(student_seq.nextval,'raja','pass'); 1 row created.

SQL> insert into student values(student_seq.nextval,'ravi','pass'); 1 row created.

[SQL Worksheet]*

Aa

1

2

3

INSERT INTO student VALUES (student_seq.NEXTVAL, 'RAJA', 'PASS');

INSERT INTO student VALUES (student_seq.NEXTVAL, 'RAVI', 'PASS');

Query resultScript outputDBMS outputExplain PlanSQL history

SQL> INSERT INTO student VALUES(student_seq.NEXTVAL, 'RAJA', 'PASS')

1 row inserted.

Elapsed: 00:00:00.014

SQL> INSERT INTO student VALUES(student_seq.NEXTVAL, 'RAVI', 'PASS')

1 row inserted.

Elapsed: 00:00:00.002

SQL> select * from student;

	STUDENT_ID	NAME
100	raja	pass
101	ravi	pass

[SQL Worksheet]*      Aa 

```
1 SELECT * FROM student;
```

Query result Script output DBMS output Explain Plan SQL history

  Download  Execution time: 0.011 seconds

	STUDENT_ID	NAME	RESULT
1	100	RAJA	PASS
2	101	RAVI	PASS

INDEXES

To create index on the Last Name column of the Employee table.

```
SQL> create table Employee(ID VARCHAR2(4 BYTE) NOT NULL, first_name
VARCHAR2(10 BYTE), Last_Name VARCHAR2(10 BYTE), Start_Date DATE,
End_Date DATE, salary Number(8,2));
```

Table created.

```
SQL> select * from Employee
```

ID	FIRST_NAME	LAST_NAME	START_DATE	END_DATE	SALARY
01	Jason	Martin	25-JUL-96	25-JUL-06	1234.56
02	Alison	Mathews	21-MAR-76	21-FEB-86	6661.78
03	James	Smith	12-DEC-78	15-MAR-90	6544.78
04	Celia	Rice	24-OCT-82	21-APR-99	2344.78
05	Robert	Black	15-JAN-84	08-AUG-98	2334.78

6 rows selected.

```
SQL> CREATE INDEX LastNameIndex ON Employee (Last_Name);
```

Index Created.

SQL> drop index LastNameIndex;
Index Dropped.

[SQL Worksheet]*      Aa 

```
1 CREATE TABLE Employee(  
2 ID VARCHAR2(4),  
3 first_name VARCHAR2(10),  
4 Last_Name VARCHAR2(10),  
5 Start_Date DATE,  
6 End_Date DATE,  
7 salary NUMBER(8,2)  
8 );
```

Query result Script output DBMS output Explain Plan SQL history

1 row inserted.
Elapsed: 00:00:00.002

SQL> CREATE TABLE Employee(
ID VARCHAR2(4),
first_name VARCHAR2(10),
Last_Name VARCHAR2(10),...
[Show more...](#)

Table EMPLOYEE created.
Elapsed: 00:00:00.010



```
1 TO Employee VALUES ('01','Jason','Martin',DATE '1996-07-25',DATE '2006-07-25',1234.56);
2 TO Employee VALUES ('02','Alison','Mathews',DATE '1976-03-21',DATE '1986-02-21',6661.78);
3 TO Employee VALUES ('03','James','Smith',DATE '1978-12-12',DATE '1990-03-15',6544.78);
4 TO Employee VALUES ('04','Celia','Rice',DATE '1982-10-24',DATE '1999-04-21',2344.78);
5 TO Employee VALUES ('05','Robert','Black',DATE '1984-01-15',DATE '1998-08-08',2334.78);
```

Query result

Script output

DBMS output

Explain Plan

SQL history



```
SQL> INSERT INTO Employee VALUES('04','Celia','Rice',DATE '1982-10-24',DATE '1999-04-21',2344.78)
```



```
1 row inserted.
```

```
Elapsed: 00:00:00.002
```

```
SQL> INSERT INTO Employee VALUES('05','Robert','Black',DATE '1984-01-15',DATE '1998-08-08',2334.78)
```



```
1 row inserted.
```

```
Elapsed: 00:00:00.001
```



```
1 SELECT * FROM Employee;
```

Query result

Script output

DBMS output

Explain Plan

SQL history



Download ▾

Execution time: 0.012 seconds

	ID	FIRST_NAME	LAST_NAME	START_DATE	END_DATE	S
1	01	Jason	Martin	7/25/1996, 12:00:00	7/25/2006, 12:00:00	
2	02	Alison	Mathews	3/21/1976, 12:00:00	2/21/1986, 12:00:00	
3	03	James	Smith	12/12/1978, 12:00:00	3/15/1990, 12:00:00	
4	04	Celia	Rice	10/24/1982, 12:00:00	4/21/1999, 12:00:00	
5	05	Robert	Black	1/15/1984, 12:00:00	8/8/1998, 12:00:00	

```
1 CREATE INDEX LastNameIndex
2 ON Employee (Last_Name);
```

Query result Script output DBMS output Explain Plan SQL history



```
SQL> INSERT INTO Employee VALUES('05','Robert','Black',DATE '1984-01-15',DATE '1998-08-08',2334.78)
```

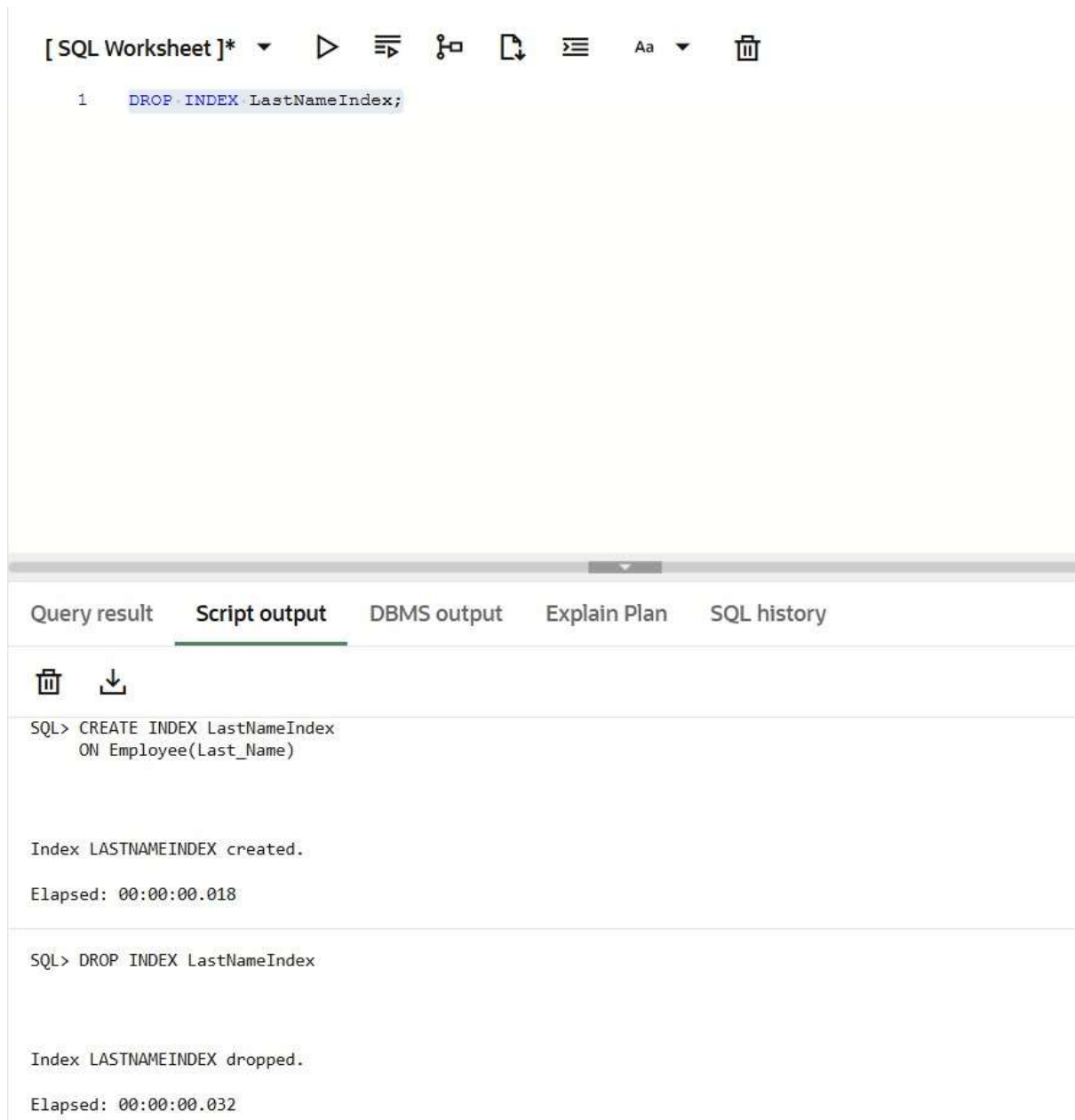
```
1 row inserted.
```

Elapsed: 00:00:00.001

```
SQL> CREATE INDEX LastNameIndex
ON Employee(Last_Name)
```

Index LASTNAMEINDEX created.

Elapsed: 00:00:00.018



RESULT:

Thus the Program Creation of View, Synonym, Sequence, and Index is executed successfully.

Exp.No.6 CREATING A STUDENT DATABASE TO SET VARIOUS CONSTRAINTS

Date:

AIM:

To Creating a Student database to set various constraints in Oracle using Structured Query Language commands

VARIOUS CONSTRAINTS

Constraint	Constraint Type	Meaning
CHECK	C	Specifies a certain condition for a column, or group of columns.
NOT NULL	C	Not null column
PRIMARY KEY	P	primary key
FOREIGN KEY	R	foreign key
UNIQUE	U	unique
CHECK OPTION	V	Specifies that DML operations on a view must satisfy the sub query.
READ ONLY	O	Specifies that a view may only be read from.

[SQL Worksheet]*

Aa

```

1 CREATE TABLE studentdb(
2   student_no NUMBER PRIMARY KEY,
3   student_name VARCHAR2(30) NOT NULL,
4   student_email VARCHAR2(20) UNIQUE,
5   student_percentage NUMBER CHECK(student_percentage <= 100)
6 );

```

Query result
Script output
DBMS output
Explain Plan
SQL history

Index LASTNAMEINDEX dropped.

Elapsed: 00:00:00.032

```

SQL> CREATE TABLE studentdb(
      student_no NUMBER PRIMARY KEY,
      student_name VARCHAR2(30) NOT NULL,
      student_email VARCHAR2(20) UNIQUE,...
Show more...

```

Table STUDENTDB created.

Elapsed: 00:00:00.023

TABLE NAME: STDUDENTDB

Column name	Data Type	Constraint
STUDENT_NO	NUMBER	PRIMARY KEY

STUDENT_NAME	VARCHAR2(30)	NOT NULL
STUDENT_EMAIL	VARCHAR2(30)	UNIQUE
STUDENT_PERCENTAGE	NUMBER	CHECK

[SQL Worksheet]*

1 DESC studentdb;

Query result
Script output
DBMS output
Explain Plan
SQL history

Table STUDENTDB created.
Elapsed: 00:00:00.023

SQL> DESC studentdb

Name	Null?	Type
STUDENT_NO	NOT NULL	NUMBER
STUDENT_NAME	NOT NULL	VARCHAR2(30)
STUDENT_EMAIL		VARCHAR2(20)
STUDENT_PERCENTAGE		NUMBER

V

STUDENT DATABASE

create table studentdb(student_no number primary key,student_name varchar2(30) not null,student_email varchar2(20) unique,student_percentage number check(student_percentage<=100))

Table Created

SQL> desc studentdb;

Name	Null?	Type
STUDENT_NO	NOT NULL	NUMBER
STUDENT_NAME	NOT NULL	VARCHAR2(30)
STUDENT_EMAIL		VARCHAR2(20)
STUDENT_PERCENTAGE		NUMBER

PRIMARY KEY

CASE 1: (Redundant value cannot be accepted by the column)

SQL> insert into studentdb values(3123101,'rajan','rajan@gmail.com',76);

1 row created.

SQL> insert into studentdb values(3123101,'rajan','rajan@gmail.com',76);
insert into studentdb values(3123101,'rajan','rajan@gmail.com',76) *

ERROR at line 1:

ORA-00001: unique constraint (SCOTT.SYS_C006201) violated

The screenshot shows the SQL Developer interface. At the top, the 'Script output' tab is selected. The main area displays the following SQL statement and its result:

```
1  INSERT INTO studentdb VALUES(3123101,'RAJAN','rajan@gmail.com',76);
```

Below the script, the 'Query result' tab shows the table structure for 'STUDENTDB':

STUDENT_NO	STUDENT_NAME	STUDENT_EMAIL	STUDENT_PERCENTAGE
NOT NULL NUMBER	NOT NULL VARCHAR2(30)	VARCHAR2(20)	NUMBER

Below the table structure, the SQL statement is repeated:

```
SQL> INSERT INTO studentdb VALUES(3123101,'RAJAN','rajan@gmail.com',76)
```

The result of the insert operation is shown as:

```
1 row inserted.
```

The elapsed time for the operation is 00:00:00.023.